

Slot-Chain：基于槽位签名链的预确认协议

v2 设计规范

草案 v1.48

2026 年 8 月

摘要 本文给出 Taiko 预确认协议的后继设计 (v2)。L2 时间被切成 1 秒的槽位 (slot)，提前两个 epoch 公布的排班表 (lookahead) 为每个槽位指定唯一构建者；构建者出块并签名，签名覆盖父块哈希，未上链的块因此串成一条签名链 (signature chain)。把这条链连同有效性证明提交上 L1 的动作称为落地 (landing)：常态由拍卖产生的聚合者执行，聚合者掉线时任何人都可以执行——块的权威来自构建者签名，不来自落地者。最终性由证明系统与确定性的结算窗口共同给出：窗口内最重的已证明候选于收盘时最终化。本文同时给出该设计的信任模型、活性核算、强制包含底线、罚没规则与参数几何，并如实标注全部已接受的残留风险与仍开放的待定项。

目录

0	一页摘要	3
1	设计目标与非目标	4
2	角色	6
3	时间与排班	6
3.1	slot 与 epoch	6
3.2	排班表 (lookahead)	7
4	构建者	9
4.1	注册表 (registry)	9
4.2	出块与签名	9
4.3	双签罚没 (L1 直接验证)	12
5	签名链：合法性、正统性、预确认语义	13
5.1	单块合法性 (validity, 电路规则)	13
5.2	正统性 (canonicity) 与分叉选择 (fork choice)	14
5.3	缺口 (gap)	15
5.4	删除 vs 缺口：签名链修复了什么、剩下什么 (明说)	16
5.5	签头不发体 (withhold-body) —— 已知的非罚没不当行为	17
5.6	结算窗口与最重已证明批次最终性 (settlement-window finality) —— 机械、无 DA 的尾巴保护	18
6	落地 (landing)	20
6.1	批次 (batch)	20
6.2	节奏	21
6.3	落地义务与无许可兜底 (这是聚合者唯一的义务)	22
6.4	停摆与缺口恢复 (recovery) —— 普通出块经落地头一跳接管, 无 episode	25
6.5	聚合者席位与经济	28
7	强制包含 (forced inclusion)：保留，规则简化为逐块义务	28
8	锚定与桥 (L1 → L2)	33
9	各角色掉线时会发生什么 (活性核算，一张表说清)	37
10	罚没与保证金总表	38

11 与 v15 的详细对照（给读过 v15 的读者）	38
12 待定项（open items）	39
参数总表	42
附录 A：与所有者建议的分歧点	44
附录 B：术语对照表	45
附录 C：结算窗口可执行参考模型（normative pseudocode + 性质验证）	46
附录 D：版本变更历史（设计过程记录）	49
附录 E：评审注记索引	57

文档定位。这是 *Taiko* 预确认协议的后继设计 (*successor design*)，取代此前的 *v15* 设计线 (永续拍卖 + *epoch* 判定)^[1]。设计源自所有者 2026-08-25 的指令：采用亚 *epoch* 级 (*sub-epoch*) 故障切换粒度，由轮换的构建者 (*builder*) 按 *slot* 出块并签名、签名串成链、由证明系统 (*proof system*) 验证。本文用中文撰写，特定名词首次出现时在括号内标注英文。

如何读这份文档。正文 (§0-§12 与附录 A-C) 是规范本体，按顺序读即可；急着抓主干的读者读 §0 一页摘要与 §9 活性核算表就能建立全貌。正文中大量形如“^[2]”“ (独立审核第 4 轮发现 2) ”的括注是设计过程的审计线索——标注某条规则由哪一轮对抗评审促成——第一遍阅读可以直接跳过它们，不影响理解；需要追溯某条规则来龙去脉时再回来查。完整的逐版本变更历史^[3] 收在附录 D，不再置于正文之前。

0 一页摘要

一句话概括：L2 每一秒一个槽位 (*slot*)，提前两个 *epoch* 公布“这一秒谁有权出块”的排班表 (*lookahead*)。轮到的构建者出块并签名，签名覆盖父块哈希——于是未上链的块串成一条签名链 (*signature chain*)。把这条链打包上 L1、附上有效性证明 (*validity proof*) 的动作叫落地 (*landing*)：常态由拍卖产生的聚合者 (*aggregator*) 负责，但聚合者掉线时任何人都可以做——因为块的权威来自构建者的签名，不来自落地者。

从出块到最终的完整通路：

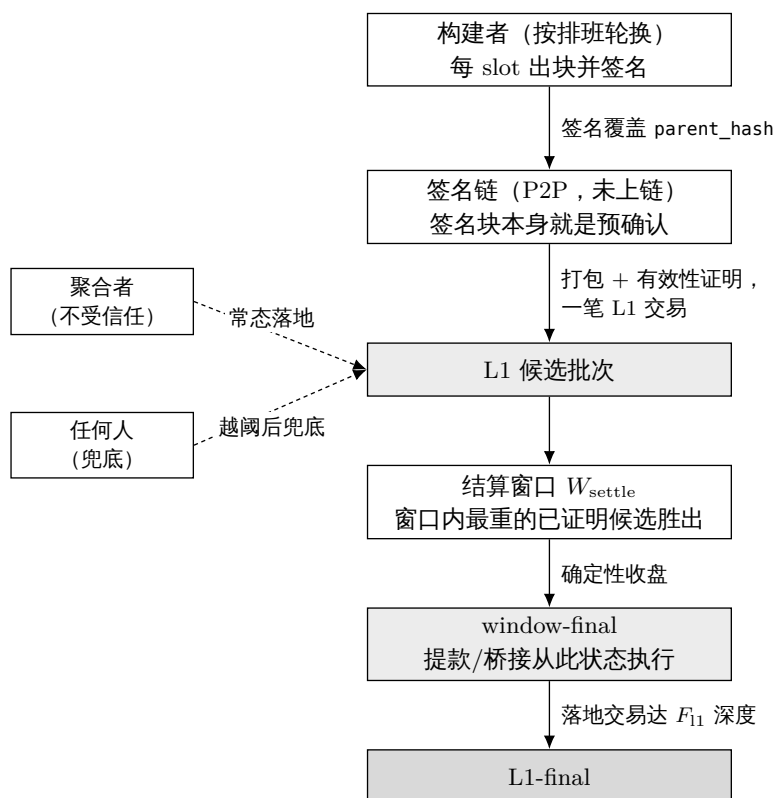


图 1: 从出块到最终的完整通路：构建者签名产生未上链的签名链，任何人都可以把它连同有效性证明落到 L1 成为候选，结算窗口收盘后依次达到 window-final 与 L1-final；落地环节无特权，聚合者与兜底者可互相替代。

这个结构一次解决三件事：

1. 秒级活性。一个构建者掉线只损失它自己的 slot（约 1 秒），下一个构建者接着最后可见的链头继续出块。不存在”一个人掉线、链停几十分钟”的窗口。
2. 不需要 L2 共识。”这个 slot 该谁出块”由排班表决定，”这个块是否合法”由证明电路（circuit）里的纯函数规则决定（验签名、验排班、验链接）——任何人读 L1 都算出同一个答案，没有投票，没有仲裁者。
3. 落地无特权，聚合者不是单点。签名链自带权威，落地只是搬运。指定聚合者超时不落地，任何人可以把签名链捡起来落地并从聚合者的保证金（bond）领取报酬。

保留什么、删掉什么（相对 v15）：

保留	删除
聚合者席位的常驻拍卖（perpetual auction）骨架	epoch 边界承诺（EBC）与每 epoch 一次的不可逆判定
强制包含队列（forced inclusion queue），规则大幅简化（§7）	默认派生规则（§6.8 default derivation）
”过错方付费、无许可替补”模式（fault-paid permissionless fall-back）	完全无政府模式（Total Anarchy，§9）——落地本身已可无许可兜底，不再需要
分级保证金/罚没（slashing）思想	K_empty、可用性证书（AC）、H_cancel 取消级联等 epoch 判定配套机制
逐块锚定（per-block anchor）的 L1→L2 桥接方式	交接/跳槽仲裁类机制（由电路规则取代）

时间线示意（slot = 1 秒，epoch = 384 slot）——掉线只留缺口，链不停：



图 2: 单个构建者掉线只在时间线上留下一个 slot 的缺口，下一个构建者以最后可见的块为父继续出块，链不停摆。

落地与兜底的关系（细节见 §6）：任何人都可以把一段连续签名链 + 有效性证明用一笔交易落上 L1 成为候选；结算窗口 w_{settle} 内最重的已证明候选收盘即最终（§5.6）。聚合者超时未落地时，任何人可落地，报酬从聚合者保证金支付（§6.3）。

1 设计目标与非目标

目标：

- [G1] 秒级排序连续性。任何单一参与者（构建者或聚合者）掉线，用户可感知的排序服务中断不超过若干个 slot（秒级），而不是若干个 epoch（分钟到小时级）。
- [G2] 无 L2 共识。排班、合法性、正统性全部是 L1 数据与电路规则的纯函数（pure function），L2 节点之间不投票、不仲裁。继承 v15 的 I1 精神：派生（derivation）是 L1 数据的确定性函数。

- [G3] 落地无特权。把已签名的链搬上 L1 不需要任何身份；指定聚合者只是效率安排（避免 gas 竞争与重复证明），不是权力安排。
- [G4] 证明 + 确定性窗口即最终^[4]。落地批次附带有效性证明、L1 当场验证成为候选；结算窗口收盘时最重的已证明候选最终化——仍无乐观期/欺诈证明/仲裁，没有“已提议未证明”的中间状态，因此不需要围绕这种中间状态的全部机制（封存截止、取消级联、计时豁免的大部分）。
- [G5] 保留包含底线。强制包含队列保留：即使全部构建者串谋审查，用户与桥接消息仍有 L1 强制路径 (§7)。“无条件”目前带一个阻塞级缺口^[5]：强制条目用普通 L2 nonce，审查方可接受一笔同 nonce 冲突交易使其被弃置 (§7 的 nonce 抢占，§12 第 12 项)。在该待定项按 §7 三候选修法之一定形前，强制路径的“无条件”仅对不可被 nonce 抢占的条目成立——由桥合约构造、不走用户 nonce 的桥接/退出消息属此类，普通用户的强制交易暂不属。§0/§1/§7 凡称“无条件底线”处均按本注解读。
- [G6] 进入门槛低。构建者保证金按“单个 slot 的可提取价值”量级定价，远低于 v15 中一个席位的整套抵押。

信任模型 (trust model, 规范性) ^[6]：

- 构建者：诚实多数 (honest majority)。假设每个排班窗口 (epoch 量级) 内，大部分排班构建者诚实遵守协议（按时出块、广播块头与块体、接最高可用链头）。
- 聚合者：不受信任 (untrusted)。聚合者可以任意作恶——拖延、截断、择叉、与少数构建者合谋——协议必须在此情形下仍保持 §9 表给出的界。界的辖域^[7]：§9 的界是活性界（最终性推进、席位更替）；聚合者与恶意构建者合谋对预确认安全的敞口——无罚没孤立，单块深跳过深度 $\leq G_{\max} - 1$ 、少数派联盟接力稀疏分支深度可达未落地尾巴长度 ($\approx \Delta_{\text{lag}}$ + 兜底响应)——是 §5.4 明码标价的安全残留，不在“有界”承诺之内。
- L1：安全且活。所有 L2 的标准假设。外加一条本设计显式承认的【有界纳入假设】^[8]：一笔付足费用的候选/落地交易在 $T_{\text{include,max}}$ 个 L1 块内被纳入，且参数满足 $T_{\text{include,max}} < \min(W_{\text{settle}} - P_{\text{prove,max}} - \text{余量}, D_{\text{anchor,max}} - D_{\text{anchor}} - \Delta_{\text{lag}} - P_{\text{prove,max}})$ (§12 设置器关系)。超出该界的 L1 定向审查属于模型外——此时更优候选可能错过收盘、恢复块 anchor 可能过期，凡依赖“临时审查下仍稳健”的表述（含 §6.4 恢复界）一律条件于本假设。这是把 v1.28–v1.40 各层隐含的同一时序依赖收拢成一条明说的假设，不是新引入的信任。

诚实多数是经济/社会假设，不是协议强制的^[9]：抽签按保证金加权、 $w_{\max}$ 防不住女巫 (§3.2)，所以一个资本方原则上可以买下多数权重、把系统推出模型——与任何 PoS 链的诚实多数假设同类。本设计不声称对构建者角色完全无信任、完全无许可；这是所有者 (2026-08-25) 明确接受的信任模型，换取的是不为全员合谋支付协议级 DA 的成本（成本恒等式见附录 A-3）。各项承诺按此校准：模型内的界见 §9 表，模型外只保证强制路径。验收基准注记^[10]：若验收标准是“任何角色均不可信、完全无许可”，本设计不满足且不试图满足——验收基准是本节所有者接受的信任模型；面向用户的承诺分级见 §5.4 的三档表述。

各机制对假设的实际用量（写明，防止假设被悄悄加码）：

- 最终性活性（兜底落地始终有可落的链）只需要每窗口 ≥ 1 个诚实构建者 (§6.3)——弱于假设，有余量；
- 预确认被孤立（跳过、扣体）的频率由不诚实构建者的占比约束——这里用到“多数”，且它

是承重的：跳过对跳过者可获利^[11]，不是靠“无利可图”自灭的行为；

- 构建者全员与聚合者整体合谋（块体私享、链无人可落地）在模型之外：协议不为这种情形提供最终性保证^[12]。但强制路径（§7）被保留为模型外的纵深防御（defense in depth）——即使假设被打破，强制交易与桥接消息仍只依赖 L1 数据独立运行，用户的退出权不随模型失效；
- 尾巴预确认安全额外依赖落地者的视图完整^[13]：一个诚实但被 eclipse/致盲的落地者会落错尾巴、撤销一条它没看到的更重尾巴的预确认。化解靠“落地深度 $\geq$ 传播沉淀”（§5.2），把该风险压到“双签（可罚没）或 eclipse（P2P 层）”；与“诚实多数是经济假设”“Byzantine 聚合者活性条件于兜底经济可行”同属明码的非纯协议依赖。

非目标（本版不做）：

- 交易级公平交换（per-transaction fair exchange）：预确认凭证是块级的（§5）。
- 用户赔付（restitution）：罚没是威慑，不是保险。
- 抗 L1 深度重组：与任何 L2 相同，L1 深度重组可回滚 L2 状态。
- 内建证明者市场（prover market）：聚合者自带证明能力或自行采购。
- 构建者全员 + 聚合者的整体合谋：按上述信任模型属模型外情形，仅保留强制路径作纵深防御，不提供最终性界。

2 角色

- 构建者（builder）：在 L1 合约注册、缴纳保证金的地址集合。按排班表轮到自己的 slot 时出块、签名、在 P2P 网络广播。构建者就是预确认者（preconfer）：它签出的块就是对块内交易的预确认承诺。
- 聚合者（aggregator）：通过常驻拍卖持有唯一席位的服务方。职责只有一件：把签名链打包成批次、生成有效性证明、落地上 L1。不构建、不排序、没有内容裁量权（discretionary power）——唯一残留的裁量是“落地哪个前缀、何时落地”，两者都被截止时间与无许可兜底约束（§6）。
- 兜底落地者（fallback lander）：任何人。聚合者超时后出现，按过错方付费模式获得报酬。不是一个注册角色。
- 用户：向构建者提交交易换取预确认；被审查时走强制包含队列。
- L1 合约：构建者注册表、聚合者拍卖、强制队列、批次入口（验证证明）、罚没入口。
- 证明系统：验证“这一段链合法且状态转换正确”的电路。它是本设计的裁判。

3 时间与排班

3.1 slot 与 epoch

- slot = 1 秒。L2 的时间单位。每个 slot 至多一个块；块的时间戳（timestamp）恒等于 slot 的起始秒——时间戳不是谁选的，是排班表定的（消除 v15 附录 C 里时间戳来源的整类问题）。
- epoch = 384 slot（沿用现值，与 L1 的 32×12 秒对齐）。epoch 在本设计中只是计量单位

(排班窗口、费用结算的粒度)，不再承载”每 epoch 一次判定”的机制含义。

3.2 排班表 (lookahead)

- 排班表是一个映射 $\text{lookahead}(\text{slot}) \rightarrow \text{builder}$ 地址，对每个未来 slot 给出唯一的出块权持有者。
- 计算方式^[14]： $\text{lookahead}(\text{slot}) = \text{抽签}(\text{hash}(\text{seed}(\text{窗口}), \text{slot}), \text{注册表快照}(\text{窗口}))$ ——种子按窗口取一次，不按 slot 取：
 - 窗口划分^[15]：窗口是固定对齐分区： $W(\text{slot}) = \text{floor}(\text{slot} / W_{\text{size}})$ ， $W_{\text{size}} = 768$ (两个 L2 epoch)；窗口 W 的起点 $\text{slot} = W \times W_{\text{size}}$ ，其”起点对应的 L1 slot”按创世映射 $L1_slot(s) = \text{GENESIS_L1} + \text{floor}(s / 12)$ 换算 (L2 slot 1 秒、L1 slot 12 秒)。任一 slot 的排班对所有观察者、在所有时刻同值。
 - 唯一快照高度：窗口 W 的快照高度 $H_{\text{snap}}(W) = W$ 起点对应的 L1 slot - D_{snap} ，其中 $D_{\text{snap}} = \text{前瞻视界 } H_{\text{look}}(2 \text{ L1 epoch}) + \text{最终性距离 } F_{\text{final}}(2 \text{ L1 epoch}) + \text{余量}(1 \text{ L1 epoch}) = 5 \text{ L1 epoch}$ ^[16]。设置器不变量： $D_{\text{snap}} \geq H_{\text{look}} + F_{\text{final}} + \text{余量}$ 。由此对排班需要已定的任意时刻都有： $\text{randao_source}(W) = H_{\text{snap}}(W) \leq \text{最终化的 L1 头} < W$ 起点——种子只有一个，对窗口内所有 slot 同时成立，排班在被使用前已定、且不受 L1 重组影响。
 - 种子： $H_{\text{snap}}(W)$ 所在 L1 epoch 的 RANDAO (经 EIP-4788 信标根电路内可证)；slot 间差异由 $\text{hash}(\text{seed}, \text{slot})$ 派生，不需要逐 slot 的独立熵源。
 - 注册表快照：同一 $H_{\text{snap}}(W)$ 的注册表状态；进出延迟 (§4.1) 保证快照覆盖整个窗口。窗口内所有块的电路排班验算一律用 $H_{\text{snap}}(W)$ ，与各块自己的锚定块无关^[17]。
 - 抽签：按保证金加权的确定性抽样 (每 slot 独立)。单地址权重上限 w_{max} (初始 20%)。诚实定性^[18]： w_{max} 只防单地址集中，防不了拆分保证金的女巫 (Sybil)——大资本拆成多个地址即可重夺权重，且不多花一分钱。真正的集中度上界是经济性的 (买权重重要真锁保证金)，与 v15 接受”最高出价者可以持续赢”同一取舍。 w_{max} 的作用是运维卫生，不承诺去中心化。
- 上述计算的完整代码化 (每一步的约束在注释里；可运行版本含性质测试见 `lookahead-model.py`，抽样算子是 §12 第 18 项 (b) 的定形候选)：

```
W_SIZE = 768          # 排班窗口 = 768 个 L2 slot (固定对齐分区，非滑动视界)
H_LOOK = 768          # 前瞻视界
D_SNAP_L1 = 5 * 32    # 快照延迟 (L1 slot)；不变量  $D_{\text{snap}} \geq H_{\text{look}} + F_{\text{final}} + \text{余量}$ 
W_MAX = 0.20          # 单地址有效权重上限 (运维卫生；防不了拆分保证金的女巫)
```

```
def window_of(slot):
    # 固定对齐分区：同一 slot 在所有时刻、对所有观察者映射到同一窗口 (r10-5)
    return slot // W_SIZE
```

```
def snapshot_height(w):
    # 唯一快照高度 = 窗口起点对应的 L1 slot -  $D_{\text{snap}}$ 。
    #  $D_{\text{snap}}$  的三项构成保证：排班被使用前，该高度已 L1 最终化——
    # 种子与注册表都不受 L1 重组影响 (r8-1)
    return l1_slot_of(w * W_SIZE) - D_SNAP_L1
```

```
def seed(w):
```



```

# 种子 = 快照高度所在 L1 epoch 的 RANDAO (EIP-4788 信标根电路内可证)。
# 按窗口取一次、不按 slot 取：窗口内所有 slot 共享同一熵源 (r7-1)
return hash("seed", randao_at(snapshot_height(w)))

def effective_weights(registry):
    # 有效权重 = min(保证金, w_max × 总保证金)。单次封顶、不迭代重归一——
    # 超额部分作废不重分配，一遍线性扫描、电路友好
    total = sum(registry.values())
    return {a: min(b, W_MAX * total) for a, b in registry.items()}

def lookahead(slot):
    # lookahead(slot) = 抽签 (hash(seed(窗口), slot), 注册表快照 (窗口))。
    # 注册表快照与种子取同一高度 (r1-9 唯一性)；slot 间差异由 hash 派生
    w = window_of(slot)
    reg = registry_at(snapshot_height(w))
    r = hash(seed(w), slot)
    return weighted_pick(reg, r)

def weighted_pick(registry, r):
    # 确定性加权抽样 (§12-18(b) 定形候选算子)：
    # 1. 地址按注册号固定排序 (快照内在序，防注册名 grinding)；
    # 2. capped 权重前缀和 (定点整数 wei, 无浮点)；
    # 3. x = r mod 总权重, x 落进哪个前缀区间就选谁。
    # 被选概率 = 有效权重占比；电路内 = 一遍前缀和 + 一次取模 + 一次定位
    eff = effective_weights(registry)
    addrs = sorted_by_registration_index(registry)
    x = r % sum(eff[a] for a in addrs)
    acc = 0
    for a in addrs:
        acc += eff[a]
        if x < acc:
            return a

```

- 前瞻可用性：任一时刻，至少未来 $H_{\text{look}} = 768$ 个 slot (约 12.8 分钟) 的排班表已经确定且任何人可算 (当前窗口剩余部分 + 下一整个窗口； D_{snap} 的视界项保证下一窗口的快照已最终化)。快照、窗口与前瞻的时间几何：

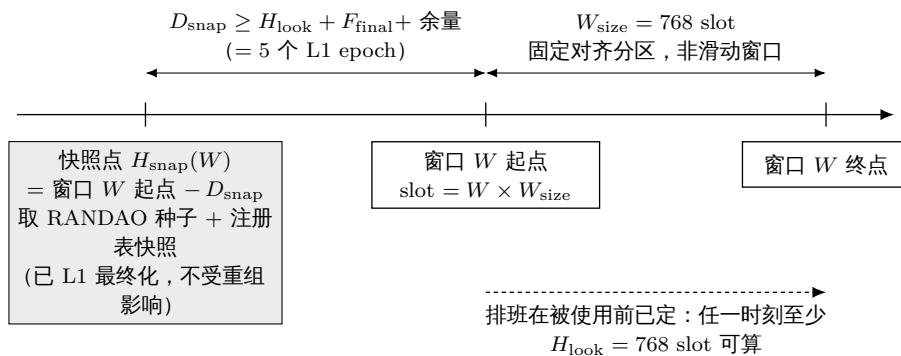


图 3: 快照点、排班窗口与前瞻视界的时间几何：熵与注册表在窗口起点前 D_{snap} 处一次性冻结，因而任一时刻均有至少一个完整窗口的排班可被任何人计算。

- 为什么不混入”最近证明的哈希”做熵^[19]：同一个陈述可以生成无数个字节不同的合法证

明（证明生成自带随机性），所以证明哈希是出证明者可零成本重磨（grind）的熵——混入它等于把排班的部分控制权送给聚合者。纯 L1 信标随机数虽然可被 L1 提议者做每人约 1 bit 的偏置，但对排班这种用途足够，且没有引入新的可控方。原则：熵只来自 L1 共识层，且取值时点早于排班窗口。

- 已知代价，明说：排班公开意味着“下一秒谁出块”是公开的，定向 DoS 一个即将出块的构建者在理论上可行。缓解：slot 只有 1 秒（打击窗口极小）、构建者可用哨兵/私有网络、被 DoS 的 slot 只是变成缺口（§5.3）。以太坊自身也带着同样的公开排班运行。

4 构建者

4.1 注册表（registry）

- L1 合约维护。注册需缴纳保证金 B_{builder} ，分两个用途（一个账户，两条红线）：
 - 双签罚没额度 L_{eq} (ETH)。定价基数是窗口累积敞口，不是单个 slot^[20]：一个构建者在被首次罚没并生效（§4.3 的 δ_{slash} ）之前，可以对它在本窗口排到的全部 slot 批量预签双叉，所以 $L_{\text{eq}} \geq$ 该构建者在一个敞口视界内的最大排班 slot 数 $\times$ 单 slot 最坏可提取价值 $\times$ 安全系数，其中敞口视界 = 剩余排班窗口 + δ_{slash} + 检测与提交时延余量^[21]。按 $w_{\text{max}} = 20\%$ 、窗口 768 slot 计，量级在一两百个 slot 的累积敞口。压低这个数字的正路是缩短罚没生效延迟与检测时延，不是假装敞口只有一个 slot。具体估价方法是 §12 第 2 项的校准内容。
 - 滋扰缓冲：小额，覆盖零星违规的处理成本。
 - L_{eq} 的诚实上限^[22]： L_{eq} 按协议可见的可提取价值估计定价；外部价值（贿赂、跨域头寸、桥接下游价值）协议无法给出上界——所以双签威慑是“按估计定价的威慑”，不是对任意外部激励的保证。依赖超出保证金规模的承诺方必须自行定价该残余（与 v15 §5.2“威慑非赔付”同一诚实条款；用户赔付本就是非目标，§1）。证据幂等：同一构建者的多份双签证据只触发一次罚没（ L_{eq} 全额一次性没收，其后证据为无效操作）——单个构建者的总威慑上限就是 L_{eq} ，这正是 §4.1 按敞口视界整体定价、§4.3 用 δ_{slash} 尽快把其余 slot 冻结为缺口的原因。
- 进出延迟与保证金保留期^[23]：退出申请后，凡是已排班给它的 slot 仍然有效（出块或成为缺口）——已进入某窗口快照 $H_{\text{snap}}(W)$ 的排班项在该窗口内一律有效，即使该地址此后（快照高度之后）退出注册也不失效（此前笼统写“排班表永不含已不在册地址”有歧义：快照后退出的地址确实仍出现在已定的排班里，正确表述是“其排班 slot 仍然有效、按出块或缺口处理”）；且保证金的实际解锁必须等到该构建者的全部罚没敞口过期——设置器不变量：退出延迟 $\geq$ 至其最后一个已排班 slot 的距离 + δ_{slash} + 检测与提交时延余量，与上文 L_{eq} 的敞口视界同一公式。否则构建者可在临近退出时批量双签、赶在证据提交生效前取回保证金，威慑清零。
- 注册表容量上限 N_{max} （初始 64）：够分散、又让电路里的排班验证保持便宜。超额按保证金排序竞争席位（细则待定，§12）。

4.2 出块与签名

- 轮到 slot s 的构建者构造块头（block header）：

```
header = (chainId, slot, parent_hash, anchor, final_ref(仅档 (ii) 块, 否则置零),
          txs_root, state_root_claim(可选), forced_root, coinbase, gas_used, ...)
sig     = 构建者对 keccak(header) 的签名
```

关键字段：

- slot：本块的槽位号；timestamp 由它唯一决定，不单独存在。
- anchor：本块引用的 L1 区块（新鲜度基准与 L1→L2 消费点，§8）^[24]。
- parent_hash：父块头的哈希。签名覆盖 parent_hash 是本设计的核心一环：父块的选择（包括“跳过谁”）是构建者签了字的显式行为，不是事后可抵赖的模糊状态^[25]。链接用父块头哈希而非父块签名——哈希是结构必需（定义链），把父块签名再放进子块属冗余（电路反正逐块验签），可作为 P2P 层的证据打包习惯，但不是共识规则。
- forced_root：本块包含的强制条目承诺（§7）。
- 签名域唯一性不变量^[26]：块头的规范化摘要（canonical digest）与签名方案必须是一个共享对象——L1 罚没合约直验（§4.3）和电路合法性验证（§5.1）验的是完全相同的字节、相同的哈希、相同的曲线。若为降低电路成本改用 BLS（走 EIP-2537 预编译）或 SNARK 友好哈希，必须两侧同时切换。密钥体系选型因此是阻塞级待定项（§12 第 6 项），先于其余参数落定。
- 构建者把（块头 + 签名 + 块体）在 P2P 广播。这个签名块本身就是预确认：给用户的回执 = （块头，构建者签名，交易的包含证明）——回执含块头哈希与 txs_root，是公开可转发的证据（§5.5 会用到）。
- 两档父块规则的判定流程（正文与例外详见下条）：
 - 父块规则——两档，判据是【缺口大小】而非父块落地状态^[27]：parent.slot < s，且父块满足以下其一。判据是缺口 s - parent.slot，在签名链结构（缺口 + parent_hash）上求值——签名时、证明时、落地时三处求值一致，不依赖父块此刻”是否已落地/是否已 final”这个随时间漂移的状态：
 - (i) 有界缺口档（正常出块）：s - parent.slot ≤ G_max（缺口上限，初始 64 slot）。父块只需是签名链里一个真实前块（parent_hash 指向它），其落地状态无关紧要——未落地、已落地未 final、乃至已 final 都可。正常出块（含每次落地后的下一块）永远走这一档：落地后下一块以小缺口接在（已落地的）链头上，缺口 ≤ G_max 即合法，“已落地但未 final”因此不构成任何空档^[28]。这一档防深重组：G_max 封顶单块回退深度，恶意构建者要孤立更深的未落地尾巴需接力稀疏分支（见下残留）。
 - (ii) 无界缺口档（恢复）：s - parent.slot 任意——但父块必须同时是【window-final】（其所在候选已在 §5.6 结算窗口收盘）^[29] 且【L1-final】（L1-final，其落地 L1 交易达 F_l1 深度；术语区分，评审 r38 DeepSeek 警告 3：这里的“final”专指 L1-final = L1 达 F_l1 深度，区别于 §6.1 的 landed-final = L2 批次经结算窗口收盘的 window-final 最终性^[30]——两档父块规则一律以 L1-final 为准；全文术语统一见 §12 待定项 18(c)）。仅在缺口 > G_max（档 (i) 不可用，即真停摆）时进入。为什么大缺口可无上限：final 落地块按 §5.2 不可再 reorg，建在它上面不可能 reorg 任何已落地块；于是它天然安全，不需要 G_max。这一档是恢复的唯一机制（取代旧的整套重锚 episode，见 §6.4）。求值时点与 F_l1 等待^[31]：父块”是 final 且是当前 canonical 落地尖端”由 §7 落地规则在落地时校验，非签名时刻。由此若停摆恰在一次落地后不久开始、当前落地尖端尚未达 F_l1，

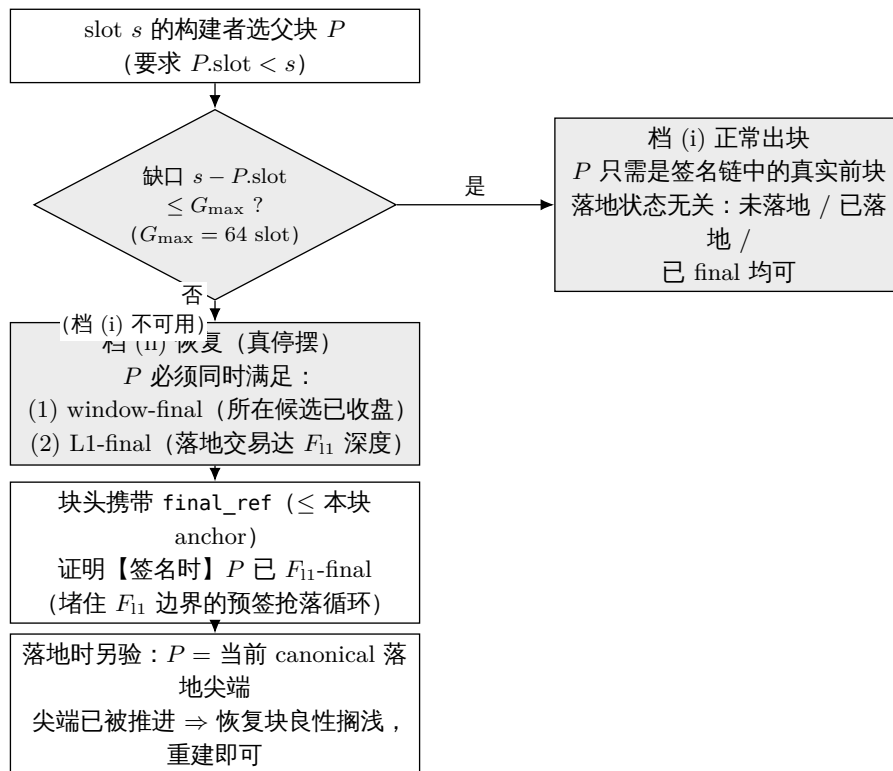


图 4: 两档父块规则的判定流程：判据是签名链上的缺口大小而非父块的落地状态，故签名时、证明时、落地时三处求值一致；只有缺口超过 $G_{\max}$ 的真停摆才进入需要 final 父块的恢复档。

恢复块须等当前尖端 final (额外 $\leq F_{l1}$, 分钟级) 方能落——建在更旧的 final 头上会 fork 掉较新尖端、被 §7 拒。稳态灾难停摆 (上次落地早已 final) 无此等待。确定性同样由落地规则兜底：批次只能延伸当前 canonical 落地尖端 (§7)，建在旧尖端上的恢复块若尖端已被推进则良性搁浅、重建即可 (无 episode/押金可乱；真停摆时尖端不动，恢复块必落)。F_l1 使“final 落地块”这一谓词在浅层 L1 重组下稳定 (挂钩 §12 L1 重组)。

- 档 (ii) 块头须携【签名时父已 F_l1-final】的 L1 引用——堵 F_l1 边界预签循环^[32]：只在落地时校验父 final，给了聚合者一个循环：在尖端尚未 final 时就预签、预证一个档-(ii) 块，等尖端刚到 final 的第一个 L1 块抢落，如此每 F_l1 一循环、lag 永不越阈、诚实内容还没老到 Δ_{prop} 无法挑战 → 永久只落稀疏仅强制链。修复：档 (ii) 块头新增一个字段 final_ref = 一个 L1 块引用，电路/L1 入口校验“该 L1 块已见证父块达 F_l1 深度”，且 final_ref $\leq$ 本块 anchor (签名时可得)。于是父块必须在签名时就已 F_l1-final，不能预签未 final 尖端；抢落循环被打断 (要落档 (ii) 必须等父真 final 后才能签，那时诚实内容尾巴已有时间被证明并作为更重候选落进结算窗口，按 §5.6 直接胜出稀疏仅强制候选)。正常恢复不受影响：恢复块本就在父 final 后才出。

- $G_{\max_landed} = \infty$ 的取值论证^[33]：档 (ii) 不设上限，是因为经落地头的 reorg 敞口天然被未落地尾巴长度封住，而尾巴长度被 Δ_{lag} 封住——超过 Δ_{lag} 的尾巴已被兜底落地、最终、不可再 reorg。所以“无限”不增大最坏 reorg 深度 (仍 $\leq \Delta_{\text{lag}}$ ，与 §5.4 既有残留同界)，却让任意时长停摆一跳恢复 (见 §9)。有限值反而两头输：小于证明时延则永远追不上长停摆，等于 Δ_{lag} 则收紧是幻觉 (自然界已是 Δ_{lag})——论证详见附录 D v1.31 条。
- “尾巴 $\leq \Delta_{\text{lag}}$ ” 是条件性的，不是无条件^[34]：上面“尾巴长度被 Δ_{lag} 封住”只在无许

可兜底能在 $\approx \Delta_{\text{lag}}$ 内把老尾巴落地时成立, 与 §6.3^[35] 对 Byzantine 聚合者活性界的同一个条件性完全一致。若兜底在经济上不可行 (无人愿兜底) 或兜底落地交易本身被 L1 审查 (§6.3/§5.2 已列), 未落地尾巴会随停摆无界增长、 $> \Delta_{\text{lag}}$; 此时档 (ii) 的 $G_{\text{max_landed}} = \infty$ 让一个恶意构建者一块即可孤立这条超长尾巴 (不再需要档 (i) 那种随尾长变奇的联盟排班密度), 合谋落地者落它即孤立远超 Δ_{lag} 的诚实预确认。诚实结论: 档 (ii) 的最坏 reorg 深度 $\leq$ 实际未落地尾巴长度, 它 $\approx \Delta_{\text{lag}}$ 当且仅当兜底按期落地; 兜底停摆时深度随停摆增长, 与 §6.3 最终性可无限停滞是同一条件、同一根因 (无协议级 DA + 兜底经济性/L1 抗审查)。这不是 ∞ 新引入的洞 (档 (i) 接力联盟同样能孤立整条超长尾巴, 只是更贵), 但 ∞ 把该 regime 的构建者侧成本降到”一人一块”, 必须与 $\leq \Delta_{\text{lag}}$ 的乐观标注一并说清。r41 注: §5.6 结算窗口在”有人肯落”的同一条件下把这类孤立分支变成自败 (更重的诚实尾巴候选在窗口内直接盖掉它), 但”无人兜底/兜底被 L1 审查”时窗口里可能只有恶意候选——条件性不变, 只是失败模式从”恶意链最终”变为”恶意链在无竞争窗口最终”。仍如实标注。

- 深重组残留 (评审 r20-1, 档 (i)/(ii) 下重述): 无协议级 DA $\Rightarrow$ L1 看不见未落地尾巴 $\Rightarrow$ 一个合谋落地者总能落一条排除某些未落地块的合法分支、孤立其中预确认。worst case 深度 = 未落地尾巴长度 $\approx \Delta_{\text{lag}}$ + 兜底响应 (尾巴被落即最终)。两档规则下的构建者侧成本: 档 (i) 单块 $\leq G_{\text{max}} - 1$ 、更深需接力稀疏联盟; 档 (ii) 单块即可提供一条以落地头为父、孤立至多整条未落地尾巴的分支。但两者都只在落地者合谋时生效——诚实落地者按 §5.2 fork-choice 落最重尾巴、无视这类短分支 (见 §5.2 落地者策略); worst-case 深度 ($\leq \Delta_{\text{lag}}$) 与”需落地者合谋”这一绑定约束都不变, 变的只是档 (ii) 把满孤立分支的构建者侧成本从”密集联盟”降到”一个恶意构建者”。这是 §1 信任模型下明码接受的残留 (附录 A-3 成本恒等式), 只打破第 2 档承诺 (§5.4, 本就声明可被单个恶意构建者撤销)。 G_{max} (档 i) 仍是压制稀疏联盟可行性的治理旋钮。

4.3 双签罚没 (L1 直接验证)

- 过错定义: 同一构建者对同一 slot 签出两个字节不同的块头。
- 证据与执行: 任何人把两个 (块头, 签名) 提交给 L1 罚没合约; 合约直接在链上验两个签名 + 同 slot + 不同哈希——不需要零知识证明、不需要仲裁期, 验证通过即罚没 L_{eq} , 其中 $\geq 80\%$ 销毁、余下付给提交者 (沿用 v15 的销毁主导原则, 防自罚套利)。
- 罚没的生效时点是 slot 基准的确定性规则^[36]: 罚没交易被 L1 纳入的区块时间戳对应 L2 slot s_0 , 则罚没自 slot $s_0 + \delta_{\text{slash}}$ (初始 $\delta_{\text{slash}} = 64$ slot) 生效: 生效后该构建者已排班的 slot 全部视为缺口——电路规则: 批次落地时, L1 合约把自己记录的 (被罚没者, 生效 slot) 清单作为已验证输入交给证明验证; 批次内任何”由已生效罚没者签名、slot $\geq$ 生效 slot”的块都不合法。生效以 slot 为界 (而不是以各块的锚定块为界), 保证所有分叉对”谁已被罚没”的判断一致, 不存在锚定竞态。没有这条, 被罚没的零保证金构建者仍能出合法块 (§4.1 与 §5.1 将互相矛盾)。
- 生效还须挂在【批次接受时点】上, 堵住历史 slot 回填^[37]: 上面只按 $\text{header.slot} \geq$ 生效 slot 判定, 而 header.slot 是签名者自填的——被罚没的零保证金旧密钥可以在生效之后签一个 $\text{header.slot} <$ 生效 slot 的历史块, 绕过该比较、继续出合法块 (还能被用于陈旧尖端 grief)^[38]。r41 (option C) 判据 = 该块所在【候选批次】的 L1 落地时点——天然不可伪造, 取代 v1.40 的承诺时间戳 (承诺层已随旧 §5.7 删除) 与 v1.39 的 δ_{land} 猜测^[39]: 罚没在 L1 生效后, 凡【落地 L1 时点 $\geq$ 生效时点】的候选批次一律不得包含该

signer 的块——无论 header.slot 填多少；生效前已落地 (provisional 或 window-final) 的候选中该 signer 的块祖父化有效。候选的落地时点是 L1 交易的固有属性、不可伪造：事后回填的历史块只能出现在生效后落地的候选里 → 必被拒；生效前已在某候选里落地的块 → 保留。代价^[40]：该 signer 生效前已签、生效时尚未落进任何候选的在途块被一并拒绝——且其上的诚实后继块因 parent_hash 穿过它而级联搁浅（双签者只广播一个分身时，诚实后继全建其上； δ_slash 短于首批证明时，生效后含它的候选全拒）。这仍落在“可罚双签 + $\leq$ 实际尾巴深度”的已接受残留之内，但 L_eq 定价与用户语义须承认该后继级联敞口；不需要 δ_land 宽限参数（删除）。

- 这一类过错是挑战者提交、L1 机械验证：比 v15 中依赖 L2 证据链的 preconf-vs-record 变体 (§8b) 强一级——提交后零延迟、无需瞭望塔跑证明，只需要有人肯提交（有赏金，是有偿的无许可工作）。

5 签名链：合法性、正统性、预确认语义

5.1 单块合法性 (validity, 电路规则)

一个块合法，当且仅当（全部电路可验）：

1. 签名有效，且签名者 = lookahead(slot) (排班表在电路内从 L1 锚定数据重算) ——唯一例外：满足 §7 合法性谓词 P_forced 的仅强制块 (forced-only block) 豁免本条排班检查—— P_forced 是精确、电路可验的准入测试（任意密钥 + 父为 window-final 头〔档(ii) 重根〕或前一仅强制块〔档(i) 同-lane 续接，r37/r42 本摘要补齐〕 + slot $\leq$ 墙钟
 - 非空且逐块重验成熟的强制前缀 + 最大消息消费，详见 §7)，取代 v1.31 前已删除的旧“三条件”表述^[41]；
2. parent.slot < slot 且父块满足 §4.2 的两档规则之一^[42]：(i) 缺口 slot - parent.slot $\leq G_max$ 时父块是签名链里任一真实前块（落地状态无关）；(ii) 缺口无上限但父块必须是最终 L1 已落地块。parent_hash 指向该合法父块。档(ii) 块另须 final_ref 合法^[43]：final_ref 见证父块已达 $F_l1-final$ 且 final_ref $\leq$ 本块 anchor (§4.2)；父”是当前 canonical 尖端”仍由 §7 在落地时校验（两个谓词分开：签名时 final 由 final_ref 证，落地时 canonical 由入口验）；
3. slot 未越过落地批次声明的范围；时间戳规则自动满足 (= slot)；
4. 按 §7 的逐块容量规则包含到期且链上尚未包含的强制条目——统一的“最大前缀”定义^[44]：块首强制前缀 = 队列 FIFO 中同时满足”条数与 gas 均在 C_force/C_bridge 份额内”的最长前缀 (C_force 是条数与 gas 双封顶，取先触顶者；桥接队列同理按 C_bridge)；取到该最长前缀即合法，少于它才不合法（不是”恰好某个条数”）。余量确定性溢出到后续块。消费不等于执行——对前置状态非法的条目弃置消费^[45]；
5. 块内交易执行合法（正常的状态转换验证）；锚定交易 (anchor tx) 满足新鲜度规则 (§8) 且满足 §8 的因果序不变量 anchor.L1_timestamp $\leq$ L2 时间戳 (slot)^[46]；且 L1→L2 入站消息按 §8 消费到【统一最大前缀】——FIFO 中同时满足”条数 $\leq C_anchor$ ”且”累计声明 gas $\leq$ L1→L2 消息 gas 份额”的最长前缀^[47]；取到该最长前缀即合法，少于它才不合法（杜绝”用新鲜 anchor 却零处理入站消息”的无限期审查）^[48]。

5.2 正统性 (canonicity) 与分叉选择 (fork choice)

- **L1 已落地部分**^[49]：正统链 = 各结算窗口收盘的 window-final 批次串起来的链 (§5.6)。窗口内冲突候选由 §5.2 全序取代、收盘定终身；收盘后才是“永久出局”。
- **未落地部分 (P2P 尾巴) 的协调规范**——排序判据统一由下条“最优链全序”给出^[50]：诚实构建者/节点接在按下条全序最优的链头上 (旧文的“最多块 + 高 slot + 哈希”已并入全序的第 2/3/4 级，不再单列，以免与全序的 lane 优先冲突)。效果定位^[51]：这是链下 P2P 协调规范、不是电路或 L1 入口规则——它只约束诚实 P2P 节点的收敛，不能约束恶意聚合者/兜底者/私有中继/L1 提议者的择叉；对恶意落地者的真实约束由 §5.6 结算窗口竞争给出 (更差候选被更重候选机械盖掉)，深跳过残留敞口的定界仍见 §5.4。
- **最优链的全序 (best-chain total order)**——分叉选择、落地策略、**§5.6** 窗口候选比较三处共用^[52]。本条取代 v1.35 的“恢复优先/冻结死尾让位”规则——独立审核 (第 1/2 轮) 指出那条会命令诚实落地者丢弃一条完整可见、可落地的健康长尾，这是错的：“不可 P2P 扩展”不等于“应被丢弃”。给定分叉点 F，在延伸 F 的候选链中比较，依次：比较的对象是标量四元组，不是逐对结构比较^[53]。每条延伸 F 的候选链独立求出四元组 $\text{key} = (\text{lane}, \text{count}, \text{tip_slot}, \text{tip_hash})$ ：
 1. **lane (candidate-local)**^[54] := 该候选自 F 起第一个块的类别——自由裁量块 = 内容 lane (大)，仅强制块 = 仅强制 lane (小)。首块一经决定，整条候选继承、后继块不改变它^[55]；
 2. **count**：自 F 起的合法块数；
 3. **tip_slot**：tip 的 slot；
 4. **tip_hash**：tip 的块头哈希 (取小者，末级确定 tie-break)^[56]。比较 = 四元组的字典序 (lane 降序、count 降序、tip_slot 降序、tip_hash 升序)——标量字典序天然自反、完全、传递。§5.6 的“严格更重”与收盘赢家由此良定义、与提交顺序无关。(形式化性质测试列 §12 第 18 项。) 内容 lane 优先的理由不变：仅强制只是审查底线，内容块本负强制前缀义务，优先内容不牺牲审查抵抗。“冻结长尾”不再被丢弃：链头落后墙钟 $> G_{\max}$ 、P2P 中不可再扩展 (无档-(i) 子、又非 final 无档-(ii) 子) 的长尾，仍是候选中的最优链——落地者应先落它 (兑现其预确认)，再从其 final 尖端恢复出块 (§6.4)；落地无需在 P2P 里扩展它，只需证明并张贴。原先担心的“恢复与 fork-choice 死锁”由此消失：先落最优 (含冻结) 链 $\rightarrow$ 从其 final 尖端一跳恢复。
- 最终仲裁仍是 L1：分歧在结算窗口收盘时由 §5.6 全序一次性消解^[57]。双签者被罚没 (§4.3)；因分区无辜分叉的块成为孤块 (orphan)，其中交易回到内存池重新被打包。
- **落地者的尾巴选择策略**^[58]：落地者面对多条候选尾巴时，对自己的视图应用上文最优链全序落最优者。这不再需要是“被信任的义务”——落更差链在 §5.6 窗口内被更重候选直接盖掉、白花成本，落最优链是唯一不徒劳的策略；协议不需要判断落地者“该落什么” (那正是 v1.39/v1.40 证明无法机械判定的)，只需让最重的已证明候选赢。 Δ_{prop} 保留为落地者视图完整的链下策略参考 (落传播沉淀满的前沿)^[59]，不再是任何链上判责参数。“有更好的尾巴但落地者没看到”的处置^[60]：某个落地者没看到更重尾巴、落了较差候选？——没关系：看到它的任何人在同一 §5.6 窗口内把更重尾巴带证明落进去，直接盖掉。单个落地者的视图不完整不再是安全依赖^[61]；整体仍需窗口内至少有一个视图完整且愿意落的参与者——正常 P2P 传播秒级、待落前沿早已传遍，该条件在 §6.3 兜底可行性下自然满足；全网 eclipse 级的致盲归 §12 P2P 规范。

- 可见性假设与“只经济、不强制”的定性^[62]：本设计假设正常 P2P 传播下，聚合者（以及任何落地者）多数情况下能看到当前最长的可用签名链——但协议不能、也不试图强制落地者一定落最长链。原因是结构性的：链的可见性本身可以被剥夺——最简单的例子，链尾构建者扣住自己刚签的块不发（§5.5 扣体/扣签名），聚合者拿不到就无法包含，这不是过错。因此“落最长可见链”是经济激励下的理性选择，不是可罚没的义务：落了较短链的落地者不受罚，其后果只有两条经济通道——(i) 在 §5.6 窗口内被更重候选取代、白付 gas 与证明费；(ii) 按下文（§5.6/§6.5）的基础费分成规则，落的链越短、分成基数越小。v1.39/v1.40 试图把“该落而未落”做成可裁决过错并被证伪，正是这一定性的反面教材。
- 节点本地选链规则 = 上文全序^[63]：L2 节点动态维护“本地最优签名链”用的判据就是上文四元组全序，不需要另一套规则。双签会被 §4.3 重罚，但不能假设它不发生——它发生时同一 slot 出现两个签名块、链一分为二，两条分叉仍是两条各自合法的候选链，四元组照常给出全序（块数不同按 count，块数相同按 tip_slot/tip_hash 仲裁），节点视图收敛不被双签打断；双签者的罚没（§4.3）与选链正交。为什么费用总和不进全序^[64]：曾考虑把“链上全部 base fee 总和”作为排序数据之一。结论是不能进正统性判据：费用是攻击者可以自己付给自己刷出来的量——恶意构建者在自己 slot 的块里塞满高费自转账，就能让一条更短的链在费用维度上压过诚实长链，等于用资本买断诚实多数的选链结果（且若费用部分回流落地者，合谋者的刷费成本还被部分返还）。因此全序保持以真实签名块数为主键（伪造须盗构建者私钥，不可刷）；费用总和只进入报酬（§6.5 基础费分成），不进入排序——激励对齐靠钱，正统性判定靠签名。
- 诚实定界^[65]：在落地之前，一个双签构建者 + 一个愿意为其分叉落地的合谋者（聚合者，或兜底窗口内的任何落地者，乃至被贿赂的 L1 提议者）可以让“另一条”尾巴成为正统，孤立掉其中的预确认。这个攻击 (i) 需要一次可被 L1 直接罚没的双签（代价 = 按窗口累积敞口定价的 L_{eq} ，§4.1）；(ii) 深度上限 = 未落地尾巴的长度。尾巴长度不是被 Δ_{lag} “硬性”压住的^[66]： $lag > \Delta_{lag}$ 只是把兜底落地的授权打开，不强制任何批次被接受；扣落地的聚合者拖住期间，尾巴在兜底响应时间内继续生长。如实的深度界 = Δ_{lag} + 兜底响应时间（一轮证明 + 落地 $\approx 10-15$ 分钟 $\approx 2-3$ epoch），再可被 L1 层对兜底交易的持续审查拉长——后者是 §6.3 已定价的最贵审查类别。这是软上界 + 明示的响应时间假设，不是硬帽。所以预确认在落地前的安全性是威慑 + 有界深度，不是绝对——§5.4 的用户语义按此表述。把尾巴保持短（§6.3）就是把这类攻击的收益压小的主要手段。
- 正统性仍只由 L1 决定（G2）——但落地“该落哪条”现在是可罚没的义务：链下 fork-choice 给诚实节点收敛（上文全序），而“最优链胜出”由 §5.6 结算窗口机械给出——L1 不裁决“谁该落什么”，只让窗口内最重的已证明候选最终。故本设计不是“L1 强制最长链”（那需协议级 DA 看见未落地尾巴），而是“L1 在已落地、已证明的候选之间选最重”——无 DA、无事后裁决，是所有者在 §1 约束下选定的机械中间点（option C）^[67]。

5.3 缺口 (gap)

- slot s 没有出现合法块（构建者掉线、被 DoS、块体没传开），链在 s 处留空， $s+1$ 的构建者接在更早的链头上。缺口是常态化处理，不是异常：不需要证明“为什么缺”，也没有人因缺口被罚没（出块是机会，不是义务——继承 v15 §9.3 的原则，但现在适用于每个 slot）。
- 缺口的代价由激励兜住：跳过的 slot 挣不到费；预确认过的用户找该构建者的声誉算账。滋扰性长期缺勤由注册表的活跃度规则处理（可选项，§12）。

5.4 删除 vs 缺口：签名链修复了什么、剩下什么（明说）

所有者提出的父块哈希串联修复，效果精确陈述如下：

- 修复了：聚合者的选择性删除。因为子块签名覆盖 `parent_hash`，从链中间抽掉一个块会使后继全部断链、证明必然失败。聚合者对一条签名链只剩取前缀的权力：落地 $[..m]$ ，把 $[m+1..k]$ 留在链外。（“恶意构建者免双签地从旧点另起短分支供聚合者择叉”的深重组路径——单块被 $G_{\max}$ 封顶，但接力稀疏分支可达整条未落地尾巴）^[68] 而被留下的后缀仍然接在新的 L1 链头上，下一个批次（任何人的）照样能落地它——截断是延迟，不是销毁。配合落地截止时间（§6.3），聚合者恶意截断买不到什么。
- 剩下的：相邻构建者的跳过（`skip`）——且它是可获利的^[69]。builder $s+1$ 接在 $s-1$ 上、声称没见过 s 的块——协议内无法证伪（“没收到”不可证明）。v1.4 此前把“行为者无收益”错误地外推到了跳过上：跳过者可以把被跳块 s 里的高优先费交易与 MEV 重新打包进自己的块，照常收取自己 slot 的全部收益——跳过是单个不可信构建者即可实施的、有收益、不可罚没的预确认撤销。诚实定界四点：(1) 该选择签名固定、公开可见（`parent_hash` 白纸黑字 + s 的签名块在 P2P 流传），归责有实锤；(2) 相邻跳过的单次收益上限 = 一个 slot 的费用/MEV 转移；深跳过 / 稀疏私有分支^[70]：一个恶意构建者用单块深跳过一次可波及至多 $G_{\max} - 1$ 个未落地块；但一个少数派联盟把多块沿私有分支接力串起（每跳父距 $\leq G_{\max}$ ，逐块合法、无双签）^[71] 可波及整条未落地诚实尾巴。§5.2 修订后的协调规范使诚实网络不跟随这类叉，它须被抢先落上 L1 才能生效。竞速的要害^[72]：若合谋落地者只是兜底者，这确是与在途诚实批次的竞速（窗口 $\approx$ 正常落地节奏，且兜底权在 $\text{lag} > \Delta_{\text{lag}}$ 前根本不开）；若合谋者就是在任聚合者，正常运行期它是唯一常态落地者，无竞速可言。该组合是本设计明码标价的安全残留，其深度界按攻击形态分三档^[73]：
 - 档 (i) 单块深跳过（未落地父）：深度 $\leq G_{\max} - 1$ （约 1 分钟尾巴），一个恶意 slot；
 - 档 (i) 接力稀疏分支：深度 $\leq$ 未落地尾巴 $\approx \Delta_{\text{lag}} +$ 兜底响应，代价是联盟须在尾巴的每个 $G_{\max}$ 窗口有一个排班 slot（密度要求随 $G_{\max}$ 调小而更苛刻）；
 - 档 (ii) 单块经落地头^[74]：一个恶意构建者以 `final` 落地头为父出一块，一步即可提供孤立至多整条未落地尾巴的分支，无需联盟密度。前提^[75]：该分支要能被合谋落地者落上 L1，其 `final` 父块须同时是当前 canonical 落地尖端（§7 只让延伸当前尖端的批次落地）——即需当前尖端本身已 `final`。稳态下尖端通常已 `final`，前提常成立、不弱化威胁，此处只是把残留说准（尖端未 `final` 时该单块攻击须先等尖端 `final`，与 §4.2 恢复的 `F_l1` 等待同源）。深度上界 $\leq$ 未落地尾巴长度，把满孤立分支的构建者侧成本从“密集联盟”降到“一人一块”，worst-case 深度与“须落地者合谋”绑定约束均不变——接受 $G_{\max_landed} = \infty$ 的代价，换任意时长停摆一跳恢复（§6.4/§9）。“ $\approx \Delta_{\text{lag}}$ ”是条件性上界^[76]：尾巴长度 $\approx \Delta_{\text{lag}}$ 仅当无许可兜底按期落地（与 §6.3 Byzantine 聚合者活性界同一条件）；兜底停摆或落地交易被 L1 审查时，尾巴随停摆无界增长，档 (ii) 一块即可孤立远超 Δ_{lag} 的尾巴——详见 §4.2 该条件性论证。此非 ∞ 独有（档 (i) 接力联盟同样可孤立超长尾巴，只是更贵），但 ∞ 把该 regime 成本降到一人一块，须一并如实标注。两档频率都 $\leq$ 联盟拿到的排班 slot 数、全程可归责（parent 选择签名在案，聚合者截断落地在 L1 公开）、机械威慑只有席位价值与声誉、协议不罚没构建者；但 r41 起，这些孤立分支要真正最终化，还须在 §5.6 结算窗口内胜过更重的诚实候选——只要有人把诚实尾巴带证明落进窗口，孤立分支即自败；残留收窄为“窗口内无人落诚实尾巴”（兜底可行性条件，§6.3）。完全的结构消除仍需协议级 DA（附录 A-3），按 §1 不采纳。 $G_{\max}$ 由此是治理旋钮：调大增强缺口容忍（§6.4），调小抬高接力稀疏分支

所需的联盟排班密度。直接重打包收益仍以攻击块自身容量为上限；两种跳过的频率都由不诚实构建者占比约束——§1 信任模型的”诚实多数”正是在这里承重（不是保守冗余）；(3) 对用户的伤害分层：纯”包含”类承诺通常在下一 slot 经重新打包兑现（跳过者恰有动力收下那些交易的费用），被真正打破的是”排序/精确状态”类承诺；(4) 处置 = 公开归责 + 注册表活跃度/声誉规则 (§12 第 1 项) + 用户对惯犯的流量惩罚。这是威慑级残留，本设计接受并明说；结构级消除需要协议级 DA/可用性证明（成本恒等式见附录 A-3）。

- 推论——预确认语义（写给用户文档的版本）^[77]：一个签名块的预确认，被打破的方式有四种：
 1. 签它的构建者双签且合谋方落地另一叉——代价是被 L1 直接罚没按窗口累积敞口定价的 L_{eq} (§4.1)；可孤立深度 $\leq \Delta_{lag} + \text{兜底响应时间}$ ^[78]；
 2. 某个后继构建者跳过它——对跳过者可获利（费用/MEV 转移）且不可罚没^[79]；公开可见、签名固定；相邻跳过波及单 slot，单块深跳过至多 $G_{max} - 1$ 个未落地块，少数派联盟接力稀疏分支可达整条未落地尾巴^[80]，一般须赢得落地竞速^[81]——但在任聚合者合谋时无需竞速^[82]；频率由诚实多数假设约束；纯包含类承诺通常经重打包兑现 (§5.4 上文)；
 3. 签它的构建者扣住块体不发 (§5.5) ——块永不能被落地，预确认自然作废，同样是威慑级处理；
 4. L1 深度重组——所有 L2 共有的残留。正确的表述是：没有任何单一参与者能”无代价且悄无声息”地废掉一批预确认——每条路要么付出 L1 可直验的罚没，要么留下签名在案的公开行为记录；但”付得起代价的合谋”可以在有界深度内做到。这仍强于 v15 单席位（那里一个席位的沉默就废掉整个 epoch 的服务且无人可替），但不是绝对保证，不应向用户宣传为绝对保证。产品必须区分三档承诺^[83]：(1) 包含意向 (inclusion intent) ——交易会上链，通常跨越跳过/扣体存活（重打包）；(2) 排序/状态预确认 (ordering/state preconfirmation) ——本设计中可被下一 slot 的单个恶意构建者有收益地撤销^[84]，只有频率约束与公开归责，无罚没；(3) L1 落地最终性 (landed finality) ——绝对（至 L1 重组深度）。面向用户的措辞按档位如实标注，不得把第 2 档宣传为强安全承诺。

5.5 签头不发体 (withhold-body) ——已知的非罚没不当行为^[85]

- 行为：构建者广播（块头 + 签名）但扣住块体。后果：后继构建者无法计算其状态、只能跳过；聚合者拿不到数据、永不能落地它；持有回执的用户的预确认作废。
- 结构性自愈^[86]：构建者只能在自己持有块体、能执行出状态的父块上构建——这是执行的物理事实，不必作为规范去”要求”。因此一段被扣体的后缀会在第一个不合谋的构建者轮到时被自动绕开（它别无选择，只能接在最后一个可用块上），产生一条人人可落地的可用分叉。扣体链要拖住整条链的最终性，需要其后所有排班构建者持续合谋私享块体——那是全卡特尔情形，归 §7 的底线条款管辖。
- 定性，明说：这不是可罚没过错——”它有块体却不发”在协议内不可证明（和”跳过”同源的不可判定性）。本设计不假装解决它，处置是：
 1. 落地前的预确认显式携带 DA 假设：用户文档必须写明”落地前的预确认依赖块体在 P2P 网络的尽力可用 (best-effort availability)，没有协议级 DA 保证”；
 2. 回执格式 (§4.2) 使受害用户手握（块头 + 签名 + 包含证明）——扣体行为的公开证据，

供声誉系统与链下追责使用；

3. 扣体者自己一无所得（它的块拿不到费、必然被跳过），这是纯损人行为，威慑靠声誉与注册表活跃度规则（§12 第 1 项）。
- 因此本设计的诚实结论是^[87]：可罚没的过错是 L1 机械可判的两类——双签（§4.3）与落地超时计次（§6.3）；落地者落更差链不是“可罚过错”而是“自败”（§5.6 窗口内被更重已证明候选盖掉，白花成本——不需要也无法机械裁决“该落什么”）^[88]；构建者层面的跳过、扣体仍不可罚没。§10、§11 的表述按此更新。弱执行类的如实清单^[89]：扣体通常无直接协议收益；跳过（含深跳过）可获得费用/MEV、可撤销排序/状态类预确认，且不可罚没——处置只有频率约束（诚实多数）与公开归责。

5.6 结算窗口与最重已证明批次最终性 (settlement-window finality) ——机械、无 DA 的尾巴保护^[90]

本节整体取代 v1.39 的“挑战罚没”（旧 §5.6）与 v1.40 的“链头承诺累加器”（旧 §5.7）。独立审核第 3/4 轮证明了那两层的共同死穴：要在 L1 上机械裁决“落地者跳过了一条更优链”，L1 必须能证明那条链当时存在、块体可得、执行有效——没有完整 DA + 挑战/押金/分账本状态机就做不到；做到了就是 *episode* 复杂度回流。所有者据此决定换根 (*option C*)：不再事后裁决“该落而未落”，而是把最终性从“第一个被证明的批次即最终”松弛为“结算窗口内【最重的已证明批次】胜出即最终”。于是“更优的链”不需要被 L1 事后认定——它自己带着证明落上来、在窗口内直接赢。比较只发生在都已落地、都已通过有效性证明的候选之间，故：- 无块体可得性问题（第 4 轮发现 1 消失）：拿不出块体的链根本成不了候选，谁也不因它受罚；- 无承诺账本/投毒问题（第 4 轮发现 2 消失）：没有任何跨窗口的承诺状态，每个窗口从当前最终头重新开始；- 无挑战/押金/罚没状态机：没有 *L_land/L_chal/L_commit/举证/反证/响应期*——落更差链不是“可罚的过错”，而是自败的徒劳（在窗口内被更重者直接盖掉，白花 *gas* 与证明费）。最终性仍由证明系统 + 确定性窗口给出——只有带有效性证明的批次能参赛，窗口收盘是确定性的 L1 高度；代价是最终性多等一个 *W_settle*（见 §6.2，预确认秒级体验不变）。

窗口的生命周期一图流（与下文伪代码、附录 C 可执行模型一一对应）：

- 候选(candidate)与窗口(settlement window)——含【基线冻结 + 候选版本化 + 收盘提交】状态机^[91]：设当前窗口最终头为 F。窗口开启即冻结基线元组 *base* = (F, F.stateRoot, F_consumed, m_consumed)；窗口内每个候选都对同一个冻结 *base* 验证(证明的起始游标/状态根 = *base*，消费同一队首)，验证通过只记录该候选自己的终局元组 *end_i* = (tip_i, stateRoot_i, F_consumed_i, m_consumed_i) 与 §5.2 权重 *key_i*——provisional 接受不改变任何 canonical 状态（不推游标、不换父、不执行桥接效果）。窗口收盘时，仅把当前最重候选（*key* 最大者）的 *end* 元组一笔原子提交为新 canonical，其 *tip* 成为 F'，下一窗口从 F' 的新 *base* 开始。伪代码：

```

openWindow(F):      base ← (F, F.stateRoot, F_consumed, m_consumed); best ← 1; close_at ← L1.now + W_settle
acceptCandidate(c): requires best=1 ∨ L1.now < close_at; # 收盘先于接受 (r44 规范化, W2):
                    # 同一 L1 高度先算 closeWindow 再受理;
                    # 到点后到达的候选只能开启下一窗口
                    if best=1: openWindow(当前窗口最终头); # 首个候选开窗
                    verify(c.proof, from = base);          # 全部候选同一冻结基线
                    requires best = 1 ∨ key(c) > key(best); # §5.2 四元组字典序, 严格更重
                    best ← (c.end, key(c))                  # 只记账, 不动 canonical
closeWindow():      requires L1.now ≥ close_at ∧ best ≠ 1;

```

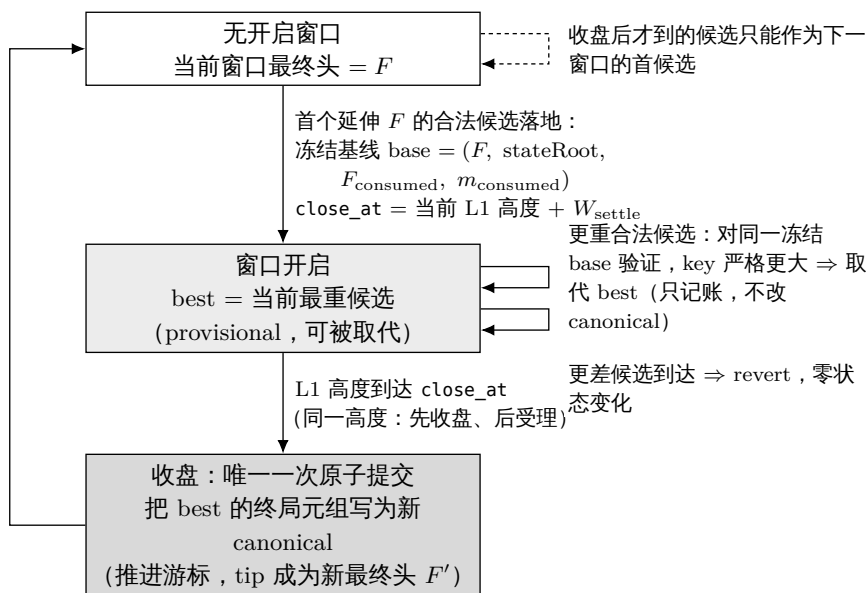


图 5: 结算窗口的生命周期状态机：窗口开启即冻结基线，窗口内所有候选对同一基线验证且只记账，直到收盘时才把最重候选的终局元组一次性原子提交为新的 canonical 状态。

```

canonical ← best.end;                                # 唯一一次原子提交
credit(best.beneficiary,
    φ_land × Σ base_fee(best))                        # 基础费分成随收盘记账 (§6.5, r47)
  
```

(基线冻结的对象是【游标与状态】，不含队列内容^[92]：强制/消息队列在 L1 上 append-only，条目按全局序号引用、内容一经入队不可变——故无须“冻结队列快照”：两个候选从同一冻结游标起步，读到的每个序号位置必然相同；窗口中途入队的新条目对“其块能覆盖到它”的后续候选可见且确定（更重候选合法地多消费），窗口态仍是 L1 历史——含入队事件——的纯函数。实现上这要求队列承诺按逐条/append-only（哈希链或 MMR）组织、证明按序号引用条目，不得用“落地时刻的单一可变队列根”做公共输入——否则窗口内先后候选的公共输入漂移。模型性质 P12。closeWindow 可由任何人调用或并入下一窗口首个交易的 lazy close；同一 L1 高度的判定序规范化为“收盘先于接受”——close_at 高度及之后到达的候选不参与本窗口、只能作为下一窗口的首候选，任何实现（含 lazy close）必须与该语义一致^[93]；窗口态与 best 都是 L1 历史的纯函数，浅层 L1 重组随 L1 回滚——并入 §12 第 9 项。）如实标注：这是一个小型窗口状态机（基线冻结/候选版本化/收盘提交三个动作）——比挑战/押金/DA 或旧 episode 小得多，但 v1.41 说“无额外状态机”不准确，r42 更正。§7 的游标原子推进规则相应改读：起始核对与推进都针对窗口基线与收盘提交，不再是“先落者即推”。

- 为什么“最重的已证明批次”就是对尾巴的机械保护：诚实落地者把完整诚实尾巴（按 §5.2 全序必然最重——诚实多数下，任何排除诚实块的分支块数更少，见 §5.4）带证明落上来，它直接赢。恶意落地者抢先落一条更差的链？——窗口内被诚实候选盖掉，链上不留痕、也不用赔付裁决，恶意者白付 L1 gas + 证明费（天然的、无需设计的 griefing 成本）。旧 v1.39/v1.40 的一切“怎么证明它当时该落”难题不再存在：该落的链自己落上来赢，不需要任何人证明“它存在过”。
- 窗口收盘的“狙击”是良性的（自审第 1/2 轮）：有人在收盘前最后一刻落一个略重的候选？按全序“更重 = 同 lane 更多真实签名块”——伪造不可能（要伪造得盗构建者私钥），所以更重的候选必然包含更多真实预确认块，它赢对用户更好（更完整的尾巴被最终化）。唯一

受损的是先落者的 gas——经济残留，非安全残留。故固定窗口即可，无须棋钟/展期（展期反而引入可玩弄的时序）。扣块体晚放（builder 扣体 → 兜底落了可见尾巴 → 收盘前攻击者放体落全尾巴盖掉）同理：结果是更完整的尾巴最终化，被”盖掉”的兜底者损失 gas——按 §6.3 兜底补偿路径处理，列为经济残留。

- **W_settle** 的设置器不变量： $W_settle \geq P_prove,max + T_include,max + \text{余量}$ ——保证从窗口开启起，任何看见更重尾巴的人来得及证明并落地它。诚实落地者无须等窗口：看见尾巴就开证，常态下窗口开启时最重候选已在路上。常态成本为零：无竞争时窗口里只有聚合者自己的一个候选，无人多花一分钱——竞争机制只在被攻击时才被使用。
- 窗口状态是 **L1** 历史的纯函数：候选都是 **L1** 交易，窗口开/收盘是 **L1** 高度，当前最优是逐笔比较的确定结果——任何节点重放 **L1** 即得同一窗口状态；浅层 **L1** 重组下随 **L1** 一起回滚（并入 §12 第 9 项的原子回滚清单，无额外状态机）。
- 活性与反刷（自审第 5/7 轮）：取代要求严格更重 + 合法证明，刷候选者每次都要真实证明费且被真实尾巴长度封顶——无 livelock；窗口收盘后到达的候选良性搁浅（针对旧 **F**，重建到新 **F'** 即可，同 §6.4 搁浅语义）。候选的受益人（报酬/退款地址）做进证明公共输入，mempool 抄袭改不了指向（v15 I8，同 §7 仅强制块）。
- 明示残留（诚实标注）：(a) 窗口期的临时排序摆动：窗口内”当前最优候选”可能被更重者取代，**L2** 节点跟随的临时头在窗口内可变——这精确对应既有的第 2 档预确认语义（可撤销、以窗口收盘为准），不新增敞口，只是把它显式化；提款/桥接只从 window-final 状态执行（§6.1/§6.2）。(b) 被排除块的兜底：若最重候选仍排除了某些真实块（如攻击者用足够多自己排班 slot 的稀疏分支在某窗口胜出——诚实多数下须密集排班联盟，同 §5.4 定界），被排除交易照旧回内存池/强制队列，下窗口重打包——最坏深度仍 $\leq$ 未落地尾巴，与 §5.4 既有残留同界，**C** 不扩大它。(c) 最终性 + **W_settle**：这是 option **C** 的真实成本，所有者明码接受（§6.2 核算）。

6 落地 (landing)

6.1 批次 (batch)

- 批次 = 一段连续签名链 $[m+1..k]$ （从当前窗口最终头 m 延伸，§5.6）+ 块体数据 (blob) + 一个有效性证明，验证 §5.1 的全部规则与逐块状态转换。
- 原子落地 (atomic landing) + 窗口最终性^[94]：数据、证明、状态根仍一笔交易提交、**L1** 当场验证——仍然没有”提议了还没证明”的中间态、没有封存截止/取消视界/取消级联/计时豁免网（v15 复杂度削减保持）。但”验证通过”现在得到的是该窗口的当前最优候选 (provisional)，而非立即最终；最终性在结算窗口收盘时授予窗口内最重的已证明候选（§5.6）。常态无竞争时，聚合者的候选就是唯一候选，收盘即最终——语义差异只在被攻击时显现。提款/**L2**→**L1** 桥接效果只从 window-final 状态执行，不从 provisional 状态执行。
- 批次大小以 blob 容量与证明成本为上限；批次之间无需对齐 epoch 边界。
- 不得落地未来的 slot^[95]：批次尾块的 slot 必须 $\leq$ 落地所在 **L1** 块时间戳对应的 **L2_slot**（允许时钟偏差初始为 0）——**L1** 入口检查、电路同证。排班表提前 768 个 slot 公布，若无此约束，构建者可预签未来 slot 的块、批次可把已落地链头推到墙钟之前——破坏时间戳/锚定/强制到期语义，并把 §6.3 的 lag 人为压成负数。相应地，lag 的算术定义下限截断为 0 (safe arithmetic)。

6.2 节奏

- 目标落地节奏：聚合者应把 §6.3 定义的滞后量 lag （最优 provisional 候选的 tip 相对 L1 墙钟的落后 slot 数——L1 可观测，不引用 P2P 状态）^[96] 保持在 Δ_{lag} 以内。证明时延约 10–15 分钟意味着稳态下 provisional 落地落后实时链头约 15–20 分钟；window-final 最终性再加一个 W_{settle} （ $\approx$ 一轮证明 + 落地，§5.6/§12）^[97]——预确认体验（秒级）与最终性节奏（分钟级）解耦不变，用户感知的是前者。四级确定性的阶梯与两个滞后量：

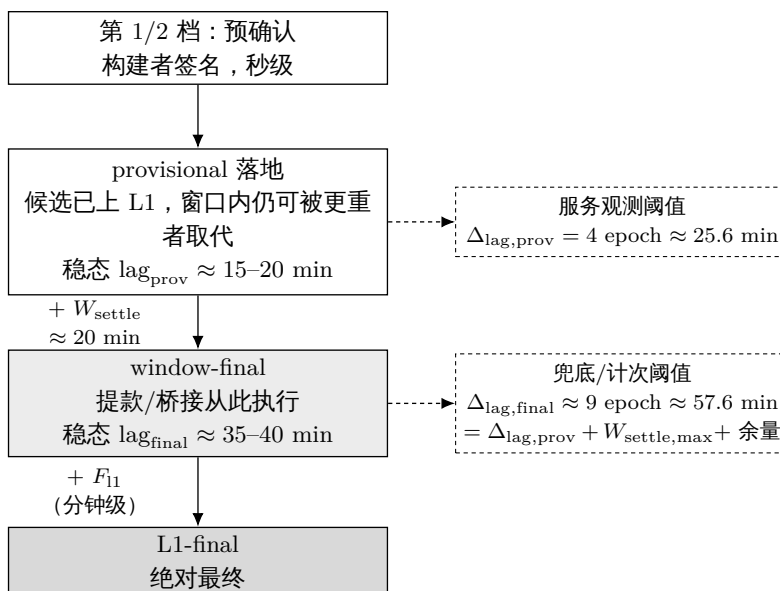


图 6：四级确定性阶梯与两个滞后量：预确认为秒级，provisional 落地与 window-final 之间相差一个结算窗口； lag_{prov} 只用于服务节奏观测， $\text{lag}_{\text{final}}$ 才进入兜底与罚则会计。

- L2→L1 桥（提款）延迟 = 落地节奏 + W_{settle} （提款只从 window-final 状态执行，§6.1）。
- 不设快速收盘（fast path）：即便窗口内只有一个候选也等满 W_{settle} 再最终——“提前收盘”的判定会重新引入可玩弄的时序，v1 不做；作为优化列 §12（如“候选覆盖全部已知签名链头即可提前收盘”须证明不可被扣块操纵后才可引入）。
- 两个 lag，各司其职^[98]： lag_{prov} = 最优 provisional 候选 tip 的落后量——只用于服务节奏观测（聚合者该不该被换等治理信号）； $\text{lag}_{\text{final}}$ = window-final 头的落后量——用于安全/可撤销敞口核算与兜底会计（下条与 §6.3）。两个阈值，一一对应^[99]： $\Delta_{\text{lag},\text{prov}}$ （= 旧 Δ_{lag} ，初始 4 epoch $\approx$ 25.6 min，设置器不变量 $\geq$ 正常 provisional 滞后带上界）^[100] 作用于 lag_{prov} ； $\Delta_{\text{lag},\text{final}} = \Delta_{\text{lag},\text{prov}} + W_{\text{settle_max}}$ （墙钟换算）+ 余量（初始 $\approx$ 9 epoch $\approx$ 57.6 min ——r46 重校：r44 曾写 8 epoch $\approx$ 51.2 min，但 $\Delta_{\text{lag},\text{prov}} + W_{\text{settle_max}} = 128 + 150$ L1 slot = 55.6 min 已超过它，8 epoch 违反本行自己的公式；模型 P9a 非空化后（部署值独立声明、不等式互检）当即抓出，改为 9 epoch = 288 L1 slot，余量 2 min）作用于 $\text{lag}_{\text{final}}$ 。理由：窗口最终性下稳态 $\text{lag}_{\text{final}} \approx \text{lag}_{\text{prov}} + W_{\text{settle}} \approx 35\text{--}40$ min，本身就高于旧阈值 25.6 min——若兜底/计次以 $\text{lag}_{\text{final}}$ 判定却沿用旧阈值，兜底窗口在完全健康的系统里永久开启，“成本 + 加成”报酬承诺永续有效、尽责聚合者被持续计次直至终止；r42 只换判定量未抬阈值，正是此 bug。可撤销敞口的如实界（中危 1）：provisional 候选虽已落 L1 但收盘前仍可被取代，故第 2 档预确认的最坏可撤销集合 = 未落地尾巴 + 窗口内 provisional 增量，真实界 = $\Delta_{\text{lag},\text{prov}} + W_{\text{settle}}$

(墙钟) + 兜底响应 $\approx \Delta_{\text{lag}, \text{final}} + \text{兜底响应}$ ——不能再写成“ $\approx \Delta_{\text{lag}}$ ”单独；全文其他“深度 $\approx \Delta_{\text{lag}} + \text{兜底响应}$ ”式深度界中的 Δ_{lag} 按 r44 读作兜底开窗阈值 $\Delta_{\text{lag}, \text{final}}$ (同一个量的新名，界不变、只是命名如实)。为把该界钉住， W_{settle} 除下界外另设共识上界 $W_{\text{settle}_{\text{max}}}$ (§12；含 L1 缺 slot 时的墙钟换算余量)，并入用户语义与 §5.4 表述。

6.3 落地义务与无许可兜底（这是聚合者唯一的义务）

本节在评审 r1-2 后重写。v1 初稿把义务写成“覆盖到 P2P 链头”——但 P2P 链头是 L1 不可观测的量，该义务无法机械执行；恶意聚合者每到截止前落一个只含 1 个块的微批次，就能既“从不超时”又让最终性无界落后。修复：把一切判定改到 L1 可观测的滞后量 (*lag*) 上，并让兜底落地本身成为聚合者失职的罪证。

- 滞后量 (L1 可观测)： $\text{lag}_* = \text{当前 L1 时间戳对应的 L2 slot 号} - \text{参照头的 slot 号}$ ；参照头取最优 provisional 候选 tip 得 lag_{prov} 、取 window-final 头得 $\text{lag}_{\text{final}}$ [101]。术语约定 [102]：本节以下 (开窗/计次/迟到罚金/ 骑线判定) 出现的裸 $\text{lag}/\Delta_{\text{lag}}$ 一律读作 $\text{lag}_{\text{final}}/\Delta_{\text{lag}, \text{final}}$ ； $\text{lag}_{\text{prov}}/\Delta_{\text{lag}, \text{prov}}$ 只承担 §6.2 的服务观测，不进入任何罚则；历史校准注记 [103] 中的 Δ_{lag} 指当时的单一阈值，其值今由 $\Delta_{\text{lag}, \text{prov}}$ 继承。规范定义 [104]： $\text{L2_slot}(t) = \text{floor}((t - \text{GENESIS_L2}) / 1 \text{ 秒})$ ，其中 t 取判定所在 L1 执行块的时间戳 (受 L1 共识约束，单个提议者只有秒级抖动的影响力)；不引用任何 L2 或 P2P 数据。两个量都是 L1 状态的纯函数，任何合约调用都能算。
- 兜底窗口与计次的因果链 (各环节的精确定义见本节后续各条)：
 - 兜底窗口是状态条件，不是计时器：只要 $\text{lag}_{\text{final}} > \Delta_{\text{lag}, \text{final}}$ (初始 $\approx 9 \text{ epoch} \approx 57.6 \text{ 分钟} = \Delta_{\text{lag}, \text{prov}} + W_{\text{settle}_{\text{max}}} + \text{余量}$) [105]，兜底窗口自动处于开启状态——任何人可落地任何合法批次，按指数成本 (L1 费用 + 证明成本 + 加成) 从聚合者保证金获得报酬 (v15 §6.7 过错付费模式的移植)。窗口不因“聚合者落了个批次”而关闭——只因 $\text{lag}_{\text{final}}$ 降回 $\Delta_{\text{lag}, \text{final}}$ 以内而关闭。微批次因此无效：落 1 个块降不了 lag ，窗口照样开着。兜底资格与报酬以 $\text{lag}_{\text{final}}$ 判定并【快照保持到结算窗口收盘】 [106]：兜底窗口一旦按 $\text{lag}_{\text{final}} > \Delta_{\text{lag}, \text{final}}$ 开启，资格与“成本 + 加成”报酬承诺对本结算窗口内的全部候选保持有效直至收盘；报酬付给收盘赢家及每个曾严格改进最优的候选 (覆盖完整证明 + 纳入成本) ——被更重者盖掉的诚实改进者不白干，provisional 短候选抢不走资格。
 - 过错与罪证 (L1 机械判定)：一次“计次” (strike) = 兜底窗口开启期间有一个非聚合者的批次被接受。这个定义的妙处在于零误伤：如果 P2P 上根本没有块可落 (构建者全缺勤)，那么谁也落不了地，聚合者不会因“没东西可落”被计次；而一旦有人落成了，就在 L1 上证明了“存在可落的内容而聚合者没落”——失职由事实自证，不需要观测 P2P。每次计次罚没聚合者活性保证金一档。计次按持续开窗时间累积、按 G_{strike} 限速 [107]：计次单位改为持续开窗区间——开窗期 ($\text{lag} > \Delta_{\text{lag}}$ 的连续区间) 每持续满一个 G_{strike} (初始 1 epoch)，若该区间内有 ≥ 1 个非聚合者批次被接受，计一次。两个性质同时保住：(i) 限速防刷：同一 G_{strike} 区间内拆分再多微批次也计一次，攻击者无法加速时间 [108]；(ii) 持续失职必然累积：窗口一直开着、兜底一直在落，计次就每 G_{strike} 涨一次， m_{agg} (2 次) 在 $\approx 2 \text{ epoch}$ 内凑满、换人——r13-1 的滴灌拖延恰好落进这一支。首次计次仍要求窗口已连续开启 $\geq G_{\text{strike}}$ (瞬时故障宽限不变)。对称的反向武器化 (恶意兜底者滴灌微批次作废聚合者追赶证明、替它刷计次)：聚合者自己的批次永不计次、可无限加价抢先，且

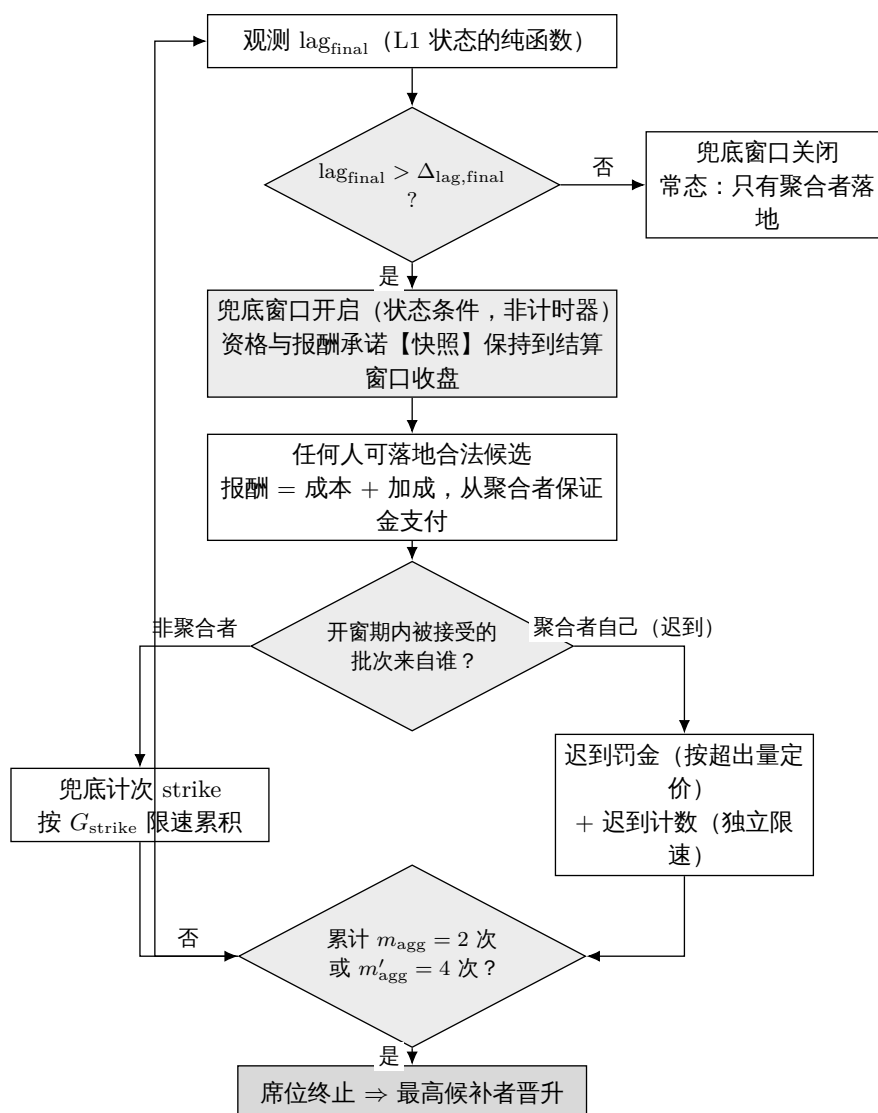


图 7: 无许可兜底窗口与席位计次的因果链：兜底资格由 L1 可观测的 $\text{lag}_{\text{final}}$ 越阈这一状态条件开启，开窗期内谁落地决定计入兜底计次还是迟到罚金，计次累积到阈值即触发席位终止。

批次证明须支持从已证前缀增量续证/聚合（工程要求，并入 §12 第 6 项）——微批次推进链头即不再迫使任何人从头重证，滴灌的牙齿被拔掉；残余仍是”对聚合者落地交易的 L1 层持续审查”（v15 §11.5 定价过的最贵审查类别），明码接受。保证金耗尽即终止：迟到罚金与计次把活性保证金扣穿（低于最低维持额，校准项）视同终止触发，罚金由此也是独立的终止路径，不再只是流血。

- 迟到落地本身也计费——堵住”骑线”策略^[109]：只按”兜底批次被接受”计次有一个漏洞：在线的恶意聚合者可以蹲在阈值上——等 lag 刚超过 Δ_{lag} 、兜底窗口一开，就抢跑（front-run）兜底者落自己的追赶批次，把窗口关掉，永不吃计次，让最终性长期贴着 Δ_{lag} 骑线，还白白烧掉兜底者的证明成本。修复（同样 L1 机械可判）：任何在 $lag > \Delta_{lag}$ 状态下被接受的批次——包括聚合者自己的——都对聚合者收取按超出量定价的迟到罚金（从活性保证金扣），且聚合者自己的迟到批次被接受这一事件计入一个较轻的违约计数，累计 $m_{agg'}$ 次（初始 4 次）同样触发终止。计数挂在”迟到批次被接受”上而不是”窗口被打开”上^[110]：窗口可能因构建者全缺勤、证明系统故障等与聚合者无关的原因打开，此时谁也落不了地、任何计数都是冤枉；而”迟到批次被接受”（无论是聚合者自己的还是兜底者的）本身就在 L1 上自证了”存在可落的内容”，零误伤成立。骑线抢跑的循环仍被堵死：抢跑者每关一次窗口就必然落一个自己的迟到批次，每次都计数。诚实降级^[111]：”外部性故障期间分文不罚”目前只对兜底 strike 完全成立；迟到罚金/迟到计数仍会命中”证明系统全网故障恢复后落第一个恢复批次的诚实聚合者”（恢复批次必然迟到）。修复其一：迟到计数与兜底 strike 同样按 G_{strike} 限速累积（每持续开窗一个 G_{strike} 且区间内有自身迟到批次被接受，至多计一次）^[112]。其二：系统性证明中断的豁免机制升格为阻塞级待定项（§12 第 8 项）——在它定义完成并可机械验证之前，”零误伤”声明仅限兜底 strike，不覆盖迟到罚金。被抢跑的兜底者的落地 gas 损失从迟到罚金中按预注册意向补偿（细则并入 §12 第 3 项）。
- 堵住”自擦 lag + 落更差链”的零罚重置^[113]：仅按”接受时 lag 是否 $> \Delta_{lag}$ ”计费，给了聚合者一个卡阈值的循环：它可以掐在 $lag = \Delta_{lag}$ （非 $>$ ）时落一条自己的仅强制/短链盖过健康内容尾巴，接受即零迟到罚金，落地后 lag 掉回，每证明周期重复——最终性长期被压制却零罚、席位永不更换。两处堵死：(a) 落更差链自败^[114]：它每次落的更差仅强制/短链只是窗口里的一个候选——那条健康内容尾巴由任何人带证明落进同一窗口，按全序直接盖掉它；恶意候选白花 gas + 证明费、什么也压制不了。”零罚可反复”变成”零收益必亏本”，不需要举证/罚没裁决。(b) lag 债按接受前状态结算，不容整数骑线^[115]：迟到罚金/迟到计数的判定改为 $lag \geq \Delta_{lag}$ ，罚金按 $late_units = \max(0, preLag - (\Delta_{lag} - 1))$ 定价——即在 $lag = \Delta_{lag}$ 处 $late_units = 1$ （最低非零罚金，不再是”超出量 = 0 的免罚点”）； $m_{agg'}$ 计数改为独立维护”自身迟到批次被接受”的按上次计次 L1 slot 限速状态（不再依赖”连续开窗满 G_{strike} ”这个在等号点无定义旧条件——发现 8 指出旧限速定义在 $= \Delta_{lag}$ 处既未开窗也无持续时间）。由此掐点到 $= \Delta_{lag}$ 不再免罚、也不再无计次。限速单位统一^[116]： G_{strike} 的规范单位是 L2 秒（1 epoch = 384 L2 秒）； $m_{agg'}$ 限速状态存”上次计次的 L1 slot”，比较时按（当前 L1 slot - 上次 L1 slot） $\times 12 \geq G_{strike_L2}$ 秒换算——消除”L1 slot 数直接当 L2 epoch 用”导致 12 倍偏差的歧义。两条合力：自擦 lag 的循环要么落更优真尾巴（无害）、要么被窗口盖掉白亏 + 迟到罚金，§6.3/§9 的”Byzantine 聚合者有界”恢复成立（在窗口内有人愿落诚实尾巴的同一兜底可行性条件下）。
- 终止与换人：累计 m_{agg} 次兜底计次（初始 2 次）或 $m_{agg'}$ 次自身迟到批次（初始 4 次）→ 席位终止，最高候补者（standby）晋升，晋升延迟 1 个 epoch（本设计里聚合者不做预

确认，换人没有服务感知，延迟可以激进)。

- 报酬防磨：聚合者的服务报酬按落地推进的 slot 数计费，不按批次数计费——微批次不但压不住兜底窗口，也挣不到钱。
- 聚合者活性是”经济假设下的界”，不是纯协议界^[117]：计次与 m_agg' 只在兜底窗口内有批次被接受时累积。若聚合者静默停落、且无人兜底——证明成本高于报酬、L1 费用尖峰、或根本没有可用证明者——则不产生计次、席位不易主、最终性可无限期停滞。因此 §9 表”Byzantine 聚合者有界”这一行的界是条件性的：它成立当且仅当无许可兜底在经济上可行（有人愿意且能负担兜底的证明与 gas 成本）。这是一条经济/工程假设，不是协议强制的活性；把它明说，与 §1”诚实多数是经济假设”同款诚实条款。兜底报酬的指数成本上限、极端拥堵下的兜底激励，列为 §12 第 3/7 项的校准项，兜底证明者的可得性/成本正式并入 §1 威胁模型与 §12。
- 兜底的可用性前提^[118]：兜底落地需要块体在手，而落地前的块体可用性只有 P2P 尽力保证 (§5.5)。所以”Byzantine 聚合者有界”的确切条件是：未落地尾巴或其某条可用分叉，对至少一个愿意兜底的落地者可用。由 §5.5 的结构自愈，这归结为”排班中存在 ≥ 1 个不合谋的构建者”（它自动绕开被扣体的段落，产出人人可落地的分叉）——该前提被 §1 信任模型（构建者诚实多数）直接蕴含且有充分余量，所以在模型内，Byzantine 聚合者的有界性是无条件的。若构建者与聚合者整体合谋（块体私享、任何人都无从落地），则计次机制确实不触发——这是零误伤原则的正确行为（此时的停滞归因于卡特尔，不归因于聚合者个体）；该情形按 §1 信任模型属模型外，协议不为其提供最终性保证，仅保留 §7 强制路径作纵深防御：用户提交强制条目 $\rightarrow$ 逾期 + lag 超限 $\rightarrow$ 任何人以仅强制块推进最终性。
- 关键性质：因为落地无需权威 (§0 第 3 点)，兜底不需要任何新信任。聚合者掉线：排序服务完全无感（构建者照常出块、预确认照常发），最终性由兜底者维持在 Δ_lag 附近。聚合者在线但恶意拖延（Byzantine）：微批次救不了它——lag 超限即兜底开启，兜底落地即计次， m_agg 次即换人；在上一轮的可用性前提下，最终性的最坏滞后被钉在 $\Delta_lag +$ 兜底响应时间，有界。这两种情形与全卡特尔情形在 §9 的表里分开列出。

6.4 停摆与缺口恢复 (recovery) ——普通出块经落地头一跳接管，无 episode^[119]

本节整体替换 *v1.8-v1.30* 的”重锚 episode”（公告 *announce* / 存在挑战 *challenge* / 押金 B_ra, B_ch / 新鲜度窗口 / s_base, s_ra / Δ_cont 续接 / 重钉 / 状态机）。那套机制的地基是”把一个块钉在墙钟 slot 的新鲜度窗口内”，而证明时延 (10-15 分钟) 与 L1 落地延迟 $\gg slot / G_max$ (64 秒) 时序窗口，导致任何这类时序规则天生脆弱，经约八轮评审^[120] 反复产生同根二阶漏洞、逐点补丁只搬移不收敛。所有者据此 (2026-08-25, 方向 B) 决定整体重设计：恢复不再是一个独立子协议，而是 §4.2 档 (ii) 父块规则的直接推论。本重设计经设计者六轮自我对抗复核^[121]。

停摆数小时乃至数天后，恢复只是”一次落地 + 一个档 (ii) 块”：

- 机制（就一条）：当缺口 $> G_max$ （连续缺勤超上限）或聚合者扣落地导致正常出块无法在未落地尾巴上继续时——任一当轮排班构建者以【当前最终 L1 落地头】为父，按 §4.2 档 (ii) 出一个普通块（内容块；全卡特尔下由任意密钥出仅强制块，§7）。该块 slot = 当轮墙钟 slot，父距任意（档 (ii) 无上限）。L2 排序由此恢复：落地头接回墙钟附近，后续构建者在其上正常出块（档 (i)）。恢复块的出块/落地本身没有公告、押金、新鲜度窗口、状态机——它只是 §5.6 结算窗口里的一个普通候选，与其他候选按全序竞争。先落最优链的顺序见下条：若停摆前存在冻结长尾，先落它、再从其尖端恢复，不丢其预确认。

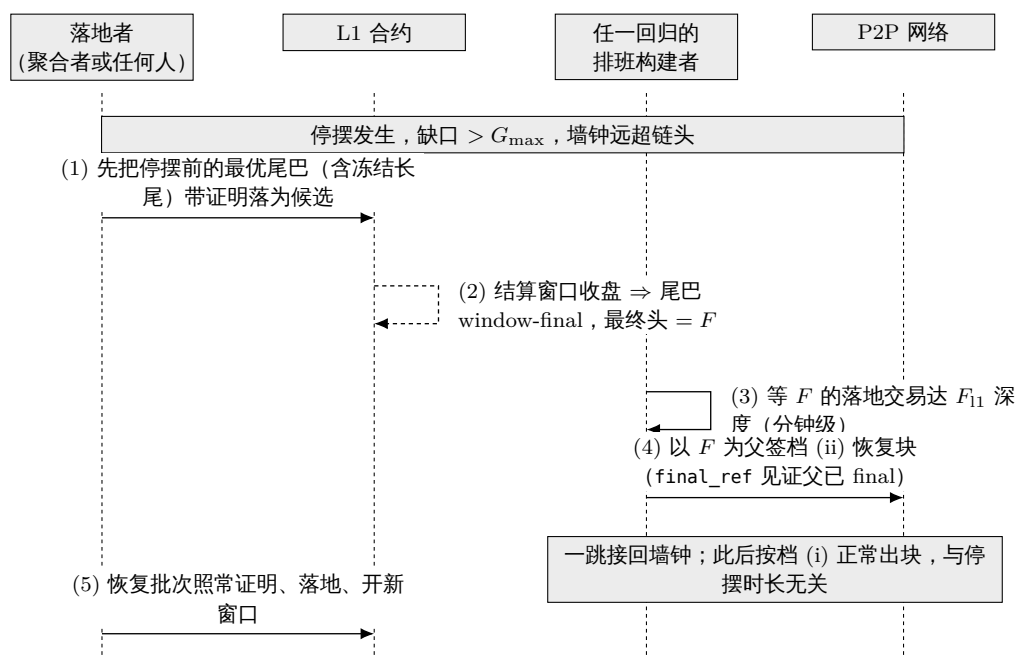


图 8: 数小时乃至数天停摆后的恢复时序: 先把停摆前的最优尾巴落地并收盘为最终头 F , 再由任一回归构建者以 F 为父出一个档 (ii) 恢复块, 一跳把排序接回墙钟, 恢复过程不含独立子协议。

- 先落最优 (含冻结) 链、再从其 final 尖端恢复——不丢尾巴^[122]: v1.35 曾写”停摆前的冻结长尾让位于一块恢复块”, 这是错的——它会命令诚实落地者丢弃可落地的健康长尾。正确顺序:(1) 停摆前的长尾按 §5.2 全序仍是最优链 (“不可 P2P 扩展”≠“应丢弃”), 落地者先把它落地 (兑现其全部预确认, $G_{\max}$ 缺口只是让 P2P 无法再接、不妨碍证明并张贴它); (2) 它落地并达 F_{l1} 后成为新的 final 尖端, 恢复块以该新尖端为父按 §4.2 档 (ii) 出块, 从这里一跳接回墙钟。于是”恢复 vs fork-choice 死锁”消失 (不靠丢尾巴, 靠先落尾巴), 健康长尾的预确认不被牺牲。只有当确实没有更长/更优链 (真全停, 连冻结尾巴都没有) 时, 恢复块本身才是最优链、直接落。跳过更优尾巴去落恢复块自败: 更优尾巴由任何人落进同一 §5.6 窗口即胜出^[123]。
- ”一跳”指重新生根, 持续推进仍需常态活性底线^[124]: 恢复块 R (档 (ii)) 一跳把排序重新生根到落地头; 但 R 本身是未落地 P2P 块, 要真正接回墙钟, 须有后续构建者在 $G_{\max}$ 内接着扩展它 (档 (i), 与每个常态块一模一样)——R 出证明落地要 10–15 分钟, 这段时间排序靠 P2P 在 R 上继续 (正常的”预确认秒级、最终性分钟级”节奏, 非新问题)。”恢复块永不过期”精确指 R 的落地合法性 (档 (ii) 无缺口上限, R 何时被证明都能落), 不指 R 可被免维护地扩展。因此恢复活性的确切条件不是”一个构建者出一次块”, 而是”排班构建者以 ≥ 1 个/ $G_{\max}$ 窗口的密度持续回归”——即常态出块本就需要的 $G_{\max}$ 活性底线。若只有单个、排班稀于每 $G_{\max}$ 一次的构建者回归: R 在 $G_{\max}$ 内无人接续 $\rightarrow$ R 自身冻结 (§5.2) $\rightarrow$ 下一构建者在同一旧落地头上另出 R' (R 未落地、未 final, 不能被档 (ii) 引用)。这个重试循环良性、不丢地 (落地头没动, 每次都是从同一点重新生根), 一旦参与度回到 $G_{\max}$ 底线以上, 某个 R 被接续、落地、恢复完成。此底线与常态出块底线同一, 不是恢复独有的新要求; 真正的全员长期宕机属社会恢复、模型外。仅强制续接同理: 全卡特尔下任意密钥的仅强制块也须以 $\geq 1/G_{\max}$ 密度持续出块才能排空积压, 单次一块不足以持续推进。
- 为什么任意时长停摆都能一跳恢复 (这是相对旧设计的根本改进): 档 (ii) 无【缺口】上限,

所以恢复块不因父距过大而过期（缺口维度不存在“过期”）。停摆 2 小时还是 2 天，恢复都是“以落地头为父出一块 → 一轮证明 → 落地”。旧设计 r28/第 7 轮 P1 那一族“证明时延顶穿缺口新鲜度窗口”的问题因此消失。注意区分维度：这里说的是缺口不过期；anchor 新鲜度这个另一个维度仍适用（见下条 + §8）——“永不过期”精确指前者，不指后者，二者不矛盾。

- “一跳”的精确边界——**F_l1** 瞬态等待^[125]：档 (ii) 的父须是 final 落地头，而 §7 只让延伸当前 canonical 落地尖端的批次落地。二者只在一种瞬态下不重合：停摆恰在一次落地后不久开始、当前尖端尚未达 F_l1。此时恢复块只能等当前尖端 final（额外 $\leq F_{l1}$ ，分钟级）——建在更旧的 final 头上会 fork 掉较新尖端、§7 拒落。故精确恢复界 = $\max(0, F_{l1} + D_{\text{anchor}} - \text{当前尖端已存活时长}) + \text{一轮证明落地}$ ^[126]：稳态灾难停摆（上次落地早已远超之）退化为纯一跳；紧接落地后起停的最坏瞬态叠加 $\leq F_{l1} + D_{\text{anchor}}$ 的等待（ ≈ 19.2 分钟，分钟级）。无论哪种，与停摆总时长无关（不随 2 小时/2 天增长）。
- **anchor** 新鲜度对恢复块的约束——“永不过期”须加限定^[127]：档 (ii) 消除的是缺口过期（无缺口上限，R 无论父距多远都合法），但 anchor 新鲜度 (§8 $D_{\text{anchor_max}}$) 仍适用：R 在签名时即钉定 anchor，须在落地 L1 块高 - anchor $\leq D_{\text{anchor_max}}$ 内落地，否则 R 的 anchor 过陈、通不过 §8、须换新鲜 anchor 重签重证。故精确表述：“恢复块永不因缺口过期”成立；但它须在其 anchor 的 $D_{\text{anchor_max}}$ 新鲜度窗口内被证明并落地。这与旧重锚 s_ra 的关键区别：旧 s_ra 新鲜度窗口被结构性钉死在 G_{max} （64 秒墙钟窗口），证明时延（10–15 分钟）必然顶穿；而 $D_{\text{anchor_max}}$ 是自由可设的参数，设置器不变量（完整公式见 §12）^[128] 保证恢复块在正常 L1 活性下不会过期——旧设计输在窗口结构性太小、非机制本身。r41 起，陈旧-anchor 尾巴死锁由 §8 的窗口几何解除（ $D_{\text{anchor_max}} \geq D_{\text{anchor}} + \Delta_{\text{lag,final}} + P_{\text{prove,max}} + T_{\text{include,max}} + \text{余量}$ ）^[129]。若 L1 对 R 的落地交易持续审查超过 $D_{\text{anchor_max}}$ 且 R 未被承诺，R 以新鲜 anchor 重建重证（不丢排序）——该情形属 L1 活性退化，§1 假设 L1 安全且活时不发生。
- 确定性与良性搁浅（设计者自审第 1 轮）：档 (ii) 的父 = final 落地块（达 F_{l1} 最终性深度，谓词在浅层 L1 重组下稳定，挂钩 §12 L1 重组）；落地时的确定性由 §7 落地规则兜底——批次只能延伸当前 canonical 落地尖端。一个建在旧尖端上的恢复块若尖端已被推进 → 良性搁浅，重建即可（无 episode/押金可乱）。而尖端被推进 $\Leftrightarrow$ 有新落地发生 $\Leftrightarrow$ 链没真停；真停摆时尖端不动，恢复块必落。
- **L1-sync** 与强制积压按逐块上限追平（设计者自审第 4/5 轮）^[130]：“一跳恢复”精确指 L2 排序一跳恢复；而长停摆积累的 L1→L2 消息与强制条目积压，按逐块上限追平：强制条目每块 C_{force} （桥接 $C_{\text{bridge},§7}$ ），L1→L2 消息处理游标 m_{consumed} 每块 $\leq C_{\text{anchor}}$ 条（§8—— C_{anchor} 绑在消息游标上，anchor 引用本身无逐块推进上限，可一跳跳到新鲜高度；此前本行“anchor 的 L1 推进量每块 $\leq C_{\text{anchor}}$ ”是与 §8 冲突的残文，已改正）。故完整核算：排序 = 1 跳；L1-sync 与强制积压 = 积压 / 逐块上限个块（恢复后 1 块/秒，通常秒级 ~ 分钟级清完）。
- 恢复活性的确切条件（设计者自审第 2 轮）^[131]：档 (ii) 的内容恢复块仍需 $\text{signer} = \text{lookahead}(\text{slot})$ ，故自由裁量恢复的确切前提 = 排班构建者以 ≥ 1 个/ G_{max} 窗口的密度持续回归（= 常态出块的 G_{max} 活性底线，非恢复独有；单个稀疏回归者只能重新生根、不能持续推进，见下“一跳：指重新生根”条）；审查底线由仅强制块（任意密钥）兜底，构建者全宕也能推强制内容。全员宕机（如全网客户端 bug）属社会恢复、模型外，任何链皆然。针对性 DoS 近未来排班构建者只是把恢复推迟 $\approx 1/p \text{ slot}$ ，与 §3.2 常态残留同级，非新洞。

- 删除清单^[132]：整套 §6.4 episode——公告/B_ra、存在挑战/B_ch/押金结算、钉定父块/重钉、新鲜度地板/s_base/s_ra、 Δ_{cont} 续接授权、episode 状态机——全部删除。随之消失的评审待办:r28（新鲜度赶不上落地）、第 7 轮 P1（ Δ_{cont} 短于证明）、r29-P2（押金结算三处矛盾/多 H_ch 无唯一赢家/Executed 后二次结算）——这些机制不存在了，问题不再适用。§12 相应移除。
- 恢复核算见 §9（一跳 + 一轮证明，含长停摆一跳；L1-sync 按积压追平）。残留见 §5.4（经落地头单块可孤立 $\leq$ 未落地尾巴 $\leq \Delta_{\text{lag}}$ ，仅合谋落地者下生效，与既有 worst case 同界）。落地者的尾巴选择策略见 §5.2。

6.5 聚合者席位与经济

- 席位由常驻拍卖产生（继承 v15 §4 骨架：绑定竞价、候补队列、q 延迟过渡、保证金、T_max 到期重拍——全部沿用，其中 q 与 T_max 语义不变）。
- 费源不得与活性构成循环依赖^[133]：服务费若来自协议费流，链空闲时费流枯竭会造成“欠付 → 理性掉线 → 依赖兜底”的循环；因此兜底报酬只从聚合者保证金支付（已如此，§6.3），且服务费源的设计（§12 第 4 项）必须保证费流不足时聚合者仍盈亏平衡（如金库兜底的最低服务费）。低价夺席 + 摆烂的组合已被 §6.3 的计次机制封死（摆烂 = 计次 = 换人），此处只需管住“诚实但被欠付”的情形。
- 费用方向反转，明说：v15 的席位因为独占排序权而向金库付费；本设计的席位是服务岗（打包 + 证明 + L1 gas），竞拍标的是最低服务费率（reverse auction，服务费从协议费收入中支付），保证金另计。竞价维度：（服务费率，保证金）二元组，费率低者胜，同费率保证金高者胜。细化是 §12 的校准项。
- MEV 与优先费归各 slot 的构建者（coinbase）。聚合者不接触排序，从结构上没有 MEV 位置——这是有意的。
- 基础费分成——落更长链的正向激励^[134]：候选批次内全部块的 base fee 总和的一个固定比例 ϕ_{land} （§12 校准项）作为落地报酬，在 §5.6 窗口收盘的原子提交中记入收盘赢家的受益人地址（受益人在证明公共输入中，§5.6 反抄袭条款，mempool 抄袭改不了指向）；其余 base fee 沿用现行协议去向。三个直接推论：(i) 分成基数随链长单调增长——落更长的链、分成更多，与 §5.2“落最长可见链是理性选择”闭环；(ii) 被更重候选取代的 provisional 落地者分文不得（赢家通吃）——落短链的机会成本 = 全部分成 + 已付的 gas 与证明费；(iii) 分成在收盘时随赢家终局元组一并原子记账，不需要额外交易或事后领取。支付时点是收盘、不是 provisional 落地^[135]：provisional 候选在窗口内可被取代，落地即付就要对被取代者追回（clawback），而收盘支付等价于“落地流程的最后一步自动分账”、无需追回——用户与落地者的现金流只差一个 w_{settle} 。§6.3 既有的“报酬按落地推进的 slot 数计费”（报酬防磨）继续适用于服务费部分；基础费分成是叠加在其上的内容量激励，两者防磨方向一致（微批次既压不住兜底窗口、也几乎没有分成基数）。

7 强制包含（forced inclusion）：保留，规则简化为逐块义务

- 队列语义沿用，数据载体改为 calldata^[136]：用户在 L1 合约 saveForcedInclusion 提交强制条目（交易或桥接消息），付基础费；费用、大小上限、逐源隔离等语义沿用现行部署。但数据载体必须改：现行队列只存 blob 引用（LibBlobs.BlobReference），blob 约 18 天过期——已部署合约的 init3 正是因为 blob 过期而不得不作废旧条目。本设计的 §7 逃生路

径恰恰要在长期停摆（可能超过 blob 保留期）时兜底，所以强制条目的载荷改为随提交交易以 calldata 携带，合约在入队时存储其内容哈希（content hash）：载荷永久可从 L1 历史重建（calldata 属于链历史，不过期），仅强制块的证明对照存储的哈希打开载荷即可， $\leq F_delay + H_force$ 的界不再依赖 blob 保留期。强制条目通常很小（一笔交易或一条桥接消息），calldata 成本可由基础费覆盖；对确需大载荷的条目，可另设“blob 载体 + 过期前须被包含否则作废退款”的次级通道（是否需要属 §12 校准项）。这与 v15 §6.4 对桥接条目采用“L1 权威的 calldata + 链上哈希”是同一手法，且顺带消除了 v15 里因 blob 过期而生的整套作废/退款/召回机制的主要触发面。

- 包含规则（电路强制，逐块，带容量上限）：条目在提交后 F_delay 到期。任何 slot 为 s 的块，必须按队列顺序、作为块首前缀，包含“到期 $slot \leq s$ 、且此前链上未包含”的条目——直至该块的强制容量上限 C_force （按 gas 与条目数双重封顶的共识常量（consensus constant））；未装下的余量保持到期状态，由后续块（含仅强制块）继续按序消化。块首前缀不足上限却遗漏了到期条目的块不合法（§5.1 规则 4 的精确形式）——审查强制条目的构建者出的块直接作废，等价于自己弃权变缺口。为什么必须有上限^[137]：v1.3 写的是“必须包含全部”，没有上限——攻击者只要在某个 slot 前攒够超过单块容量的到期条目，就能让所有块（包括仅强制块，§5.1 规则 4 对它同样适用）永远无法“全部包含”而集体不合法，链在 F_delay 后永久停摆。上限 + 确定性溢出正是 v15 §6.5/§6.8 的每快照容量上限与溢出规则（“超额确定性溢出到下一 epoch”）在逐块粒度上的移植——v1.3 移植规则时漏掉了这一半。积压的清空速度下界 = 每块 C_force ，攻击者堆积压只是花钱买延迟，买不到停摆。时间界是积压感知的，不是常数^[138]：条目 i 的执行时间界 = 到期时点 + $\text{ceil}(\text{其前方到期未消条目数} / C_force) \times \text{落地节奏}$ ——积压受限时退化回常数界；持续高速付费入队的攻击者能推长后来者的等待（经济型抗 DoS：花费随条目数线性增长，换不来停摆，只有按序延迟）。安全消息保留配额——双队列实现^[139]：强制队列拆为两条独立 FIFO——普通队列（游标 f_cur_ord ）与桥接/退出队列（游标 f_cur_br ），入队时按条目类型确定归属（桥接消息经 L1 权威入口识别）。每块的强制前缀规范顺序：先取桥接队列到期条目至多 C_bridge （初始 = $25\% \times C_force$ ），再取普通队列到期条目补足 C_force ；两游标各自按 r7-3 的原子规则推进，确定性与电路可验性不变。由此桥接条目的时间界只依赖桥接队列自身的积压，普通条目洪泛与之无关；配合现行的逐源隔离与基础费。§9 表涉及强制路径的时间界一律按（各自队列的）积压感知公式理解。队列游标语义^[140]：“此前链上未包含”用队列游标 f_cursor （该链已消费的条目总数）精确表述：游标是被证明链状态的一部分；每个块的强制前缀恰好消费队列位置 $[f_cursor, f_cursor + k)$ 的到期条目（ k 取“到期且容量允许”的最大值， $k \leq C_force$ ）并推进游标——同一批次内的后续块读到的是前面块推进后的游标，不是批次起点的 L1 快照，因此不存在同批次内重复要求或漏判。原子状态转换^[141]：L1 合约维护 canonical 的已消费游标对 $F_consumed = (f_cur_ord, f_cur_br)$ （与两条队列各自的入队尾游标分开）。落地时：(1) 批次声明的起始游标对必须逐分量等于合约当前 $F_consumed$ ；(2) 证明的公共输入是队列快照 =（两队列根、两尾游标）。净规则^[142]：每个块的强制义务对照【该块签名头已承诺的 anchor】逐块计算与逐块证明；这里的“批次级快照”专指 $F_consumed$ 这一对已消费游标（L1 落地时逐分量匹配起始、原子推进至末值），不是独立于逐块 anchor 的第二套到期判定——到期永远按各块自己的 anchor 判，游标对只记“消费到哪”。快照绑定改为逐块、写进签名块头^[143]：此前 r18-2 把快照定为“批次首块的 anchor 高度 H_batch ”（批次级唯一），但 Codex 第 18 轮指出这让一个已签名块的强制义务取决于落地者最终选的批次边界——签名的 P2P 尾巴可在任意前缀处被拆落地，若 B2 的 forced_root/状态转换是按快照 H1 签的、落地者却只落

B1 使 B2 成为下一批次首块 (快照变 H2)，则 B2 或漏掉 H1-H2 间到期的条目、或无法在以 B1 为起点的批次里被证明；而 forced_root 已签不可改，尾巴无法修复，普通前缀落地就会把本来合法的块搁浅。修复：每个块的强制义务对照它自己签名块头里已承诺的 anchor (§4.2 块头本就含 anchor 引用) 计算——义务在签名时即固定，与批次边界无关；批次证明逐块按各自 anchor 验强制前缀。因 anchor 沿链不减，F_consumed 仍单调推进 (每块消费”对照本块 anchor 到期、且游标之后”的前缀)。L1 入口的新鲜度检查必须逐块 (评审 r38, DeepSeek Critical——删去”或批次末块，其 anchor 最高”的旧捷径: anchor 只单调不减，批次可以末块 anchor 新鲜、而靠前的块 anchor 陈旧；只查末块会漏放这些靠前块，而它们的强制义务/m_consumed 是对照陈旧 L1 视图算的，强制包含这条无条件安全路径不容此漏) 对照落地 L1 块高 - 块 anchor $\leq D_anchor_max$ ，仍封死”用陈旧 anchor 藏到期条目”，且现在同一签名块无论落在哪个批次边界，义务不变、不可被搁浅。退出时限仍为 §7 积压公式 + D_anchor_max (有界)。(3) 验证通过后合约把 F_consumed 整对原子推进到批次末游标对——r42 改读：核对与推进都按 §5.6 窗口状态机进行：候选验证对照窗口冻结基线的游标对 (同窗口全部候选同一起点)，验证只记录候选自己的末游标对，收盘时仅把赢家的末游标对原子提交；(4) 并发候选不再”先落者成功”，而是同窗口按 §5.2 全序取代、收盘定赢家。前文 r5-6 的单游标表述按分量意义理解 (每条队列各自”消费到期前缀、推进游标”)。

- 每个块块首的强制消费，一图流 (”统一最大前缀”规则，与 §5.1 规则 4、附录 C 模型一致)：

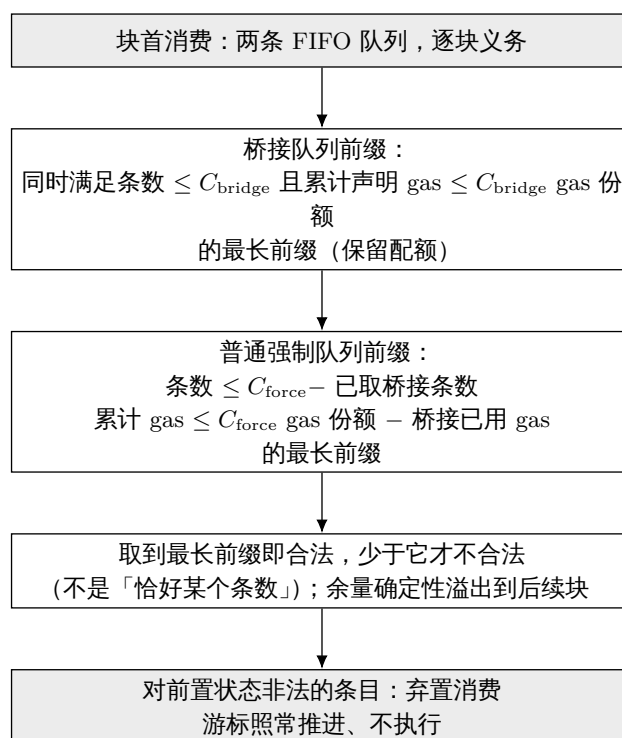


图 9: 每个 L2 块块首的强制条目消费规则：桥接队列先按保留配额取最长前缀，普通强制队列再在剩余条数与 gas 配额内取最长前缀，状态非法条目按弃置消费处理以避免队头死锁。

- 非法条目的准入与弃置消费^[144]：v1.8 的规则对一条畸形或状态非法的条目构成两难——包含它违反 §5.1 规则 5 (执行合法性)，不包含违反规则 4 (强制前缀完整性)——一笔付

费提交即可让普通块与仅强制块全体永久不合法、链停摆。修复为两道防线，判定全部确定性：

1. 入队验证 (L1 合约执行)：载荷已改为 calldata 随交易提交 (本节首条)，合约因此能够在 saveForcedInclusion 时直接验证：可解码为格式良好的签名交易、签名可恢复、chainId 正确、条目 gas 上限不超过其所属队列的单条目份额——普通条目按 $C_{\text{force}} - C_{\text{bridge}}$ 、桥接条目按 C_{bridge} 计^[145]。不过关即拒绝入队；桥接条目的上限由桥合约在构造消息时同步强制 (设置器不变量：桥接消息的最大 $\text{gas} \leq C_{\text{bridge}}$ 单条目份额)。容量参数下调的设置器不变量^[146]： $C_{\text{force}}/C_{\text{bridge}}$ 的下调拒绝生效，除非新的单条目份额 $\geq$ 相应队列未消费条目的最大 gas。实现用水位线 (watermark)：各队列在入队时维护未消费条目 gas 的最大值 (队列排空时重置)，设置器对照水位线检查——保守但 $O(1)$ 、L1 直查。由此任何可达状态下都不存在“已入队却装不进配额”的条目，无需任何借额特例。桥接队列条目由桥合约构造，格式良好由构造保证。
 2. 消费时弃置 (consume-and-discard，电路规则)：入队验证挡不住状态依赖的非法性 (nonce 不匹配、余额不足以覆盖 gas 上限 $\times$ 费率上限——入队后状态会变)。规则：块的强制前缀消费到某条目时，若该条目对消费点前置状态不满足确定性可执行判定，则该条目照常消费 (游标照常推进、计入 f_{consumed}) 但不执行 (除消费记录外不改状态)。判定是前置状态的纯函数，电路可验、所有节点一致；块的合法性由此与条目内容解耦——规则 4 要求的是“按序消费”，不是“成功执行”。这是 v15 默认派生规则“逐源默认处置”职能中唯一必须保留的部分：v2 删掉了默认派生整套机制 (§0 表)，但“非法输入的确定性弃置”不是派生策略、是完备性要求，以最小形式回收于此。用户侧语义：强制费入队时收取、弃置不退——格式与签名在入队时已验过，剩余的弃置只可能源于提交者在入队后自己动了 nonce 或余额。nonce 抢占弱化强制路径——阻塞级待定项^[147]：强制条目用普通 L2 账户 nonce，弃置不退。若用户此后 (例如审查期间为维持运转) 用同一 nonce 发另一笔交易先被打包，强制条目到消费点即因 nonce 冲突被弃置——审查方可主动接受一笔冲突交易来废掉受害者的强制条目，削弱本应无条件的强制路径 (§5 桥接/退出消息尤其敏感)。这不是纯用户失误，是对审查抵抗底线的真实攻击面。候选修法 (择一，需设计)：(i) 强制条目占用每源独立的强制 nonce/计数器，与普通 nonce 空间分离；(ii) 入队即冻结该 nonce，普通交易不得抢占直至强制条目被消费；(iii) 桥接/退出消息本就由桥合约构造、不走用户 nonce (§7 桥接队列)，把该保护延伸到更广的强制类。升为阻塞级待定项 (§12 第 12 项)：在选定并形式化前，强制路径对 nonce 抢占的无条件性不成立，面向用户的强制承诺按此打折。
- 仅强制块 (forced-only block) ——全卡特尔下的审查底线，现为 §4.2 档 (ii) 的直接应用^[148]：
 - 场景：全部构建者串谋、不出任何自由裁量块 (§9 模型外情形)。强制内容仍需一条进入路径。
 - 定义：仅强制块 = anchor (系统级隐式交易，§8) + 到期强制条目 (按队列顺序、至逐块上限 $C_{\text{force}}/C_{\text{bridge}}$ ，§7) 之外别无自由裁量内容；任意密钥签名 (电路验有效性、豁免排班检查)；父块规则见谓词 2 (重新生根用档 (ii) final 落地头，后续用档 (i) 串联前一个仅强制块)^[149]。coinbase = 出块者自选受益地址 (收强制费)，作为批次证明公共输入，mempool 复制无法改指向 (v15 I8 预承诺受益人)。
 - 合法性谓词 **P_forced** (精确、电路可验，取代 §5.1 旧“三条件”引用)^[150]：一个块作为仅强制块被电路接受，当且仅当同时满足：

1. 签名有效（任意密钥，豁免 $\text{signer} = \text{lookahead}(\text{slot})$ 排班检查——这是它相对 §5.1 规则 1 的唯一豁免；签名仍须覆盖块头、可 L1 直验）；
 2. 父块满足以下之一^[151]：
 - * (重新生根) 档 (ii)：父是 final L1 落地块 (F_{l1} 深度)，落地时须是当前 canonical 落地尖端 (§7) ——用于仅强制链的起点，或前一条仅强制链冻结 ($> G_{\max}$) 后的重新生根；
 - * (同 lane 续接) 档 (i)：父是一个 $\text{slot} - \text{parent.slot} \leq G_{\max}$ 的前一个仅强制块（父块本身满足 P_{forced} ）——使仅强制块能在全卡特尔下以每块 C_{force} 的密度串联、无需等每块 F_{l1} 最终化即可续下一块，积压/ C_{force} 逐块排空由此成立。父限定为仅强制块（不是任意档 (i) 父）：防任意密钥经档 (i) 注入诚实排班尾巴——仅强制块只在仅强制 lane 内串联，诚实内容尾巴仍只由排班构建者按 §5.1 规则 1 扩展；两条 lane 由“父是否 P_{forced} ”分开，而按 §5.2 全序内容 lane 全局优先于仅强制 lane，故更重的仅强制链也不能击败健康内容尾巴^[152]。
 3. **slot** 合法： $\text{parent.slot} < \text{slot}$ 且 $\text{slot} \leq L2_{\text{slot}}$ (落地所在 L1 块时间戳) (§6.1 不得落地未来 slot, L1 入口 + 电路同证) ——这即是“当轮墙钟 slot”的可机械验证形式，不依赖任何在册身份；
 4. 内容仅强制 + 逾期 H_{force} 才开放^[153]：块体 = anchor (§8) + 按 §7 队列顺序、至 $C_{\text{force}}/C_{\text{bridge}}$ 份额的强制前缀，别无自由裁量交易；该前缀非空，且其队头条目已逾期 $\geq H_{\text{force}}$ (即 $\text{slot} \geq \text{due_slot}(\text{队头}) + H_{\text{force}}$) ——给排班构建者 H_{force} 先纳入的窗口，任意密钥仅强制块只在此后才合法，与 §9/§12 的“逾期兜底”时间界一致。逐块重验，lane 续接不继承成熟资格^[154]：每个仅强制块（含同-lane 续接块）都须对它自己消费的队头满足 $\text{slot} \geq \text{due_slot}(\text{该队头}) + H_{\text{force}}$ ；lane 续接只免父块 final 检查，不免各块的条目成熟检查；
 5. 消息消费 + 因果序：按 §8 统一最大前缀（条数 $\leq C_{\text{anchor}}$ 且累计 $\text{gas} \leq \text{份额}$ ）^[155] 消费 $L1 \rightarrow L2$ 消息；且满足 §8 因果序不变量 $\text{anchor.L1_timestamp} \leq L2$ 时间戳 (slot)。slot 唯一性怎么保证^[156]：仅强制块因任意密钥签名，不靠双签罚没保证“每 slot 至多一块”（无在册身份可罚）。同一 slot 可能出现多个合法仅强制块 = 竞争分支，由 §5.2 fork-choice + 原子落地择一（先落者正统，§5.2）——与普通分叉同一消解路径，不需要 slot 唯一性作为电路不变量。谓词 4 的非空前缀 + 谓词 3 的 $\text{slot} \leq \text{墙钟}$ 共同确保它只在“有到期强制内容且不越未来”时合法。
- 合法性与竞争，统一归入 §5.4 残留（消除旧设计的双侧条件与 finding-6 抢跑担忧）：仅强制块和任何档 (ii) 块一样——正常运行时它是 final 落地头上的短分支，诚实落地者按 §5.2 fork-choice 不跟随、不落它，故不与在途正常内容块抢位（这正是旧设计用 $\text{lag} > \Delta_{\text{stall}}$ 双侧条件苦苦维持的互斥，现在由 fork-choice 天然给出，不需要额外开关）；只有合谋落地者才会落它孤立尾巴 = §5.4 明码残留（同界）。全卡特尔时它是唯一在推进的链，由 permissionless 落地推进最终性。
- 单方 Byzantine 聚合者可自源孤立分支——必须明说的残留收紧^[157]：任意密钥豁免意味着恶意聚合者自己就能造出这条档-(ii) 仅强制分支（先自入队一条强制条目、等 F_{delay} 到期，即有合法非空前缀），无需策反任何排班构建者、不产生可罚没双签。于是 §5.4“须落地者合谋”的残留，其“合谋”可以塌缩为单个不受信任的聚合者一方（它既是分支作者又是落地者），而非“恶意构建者 + 另一个合谋落地者”两方。r41 起，这条攻击是“自败”而非“残留”：(a) 它打不动健康尾巴——按 §5.2 全序内容 lane 全局

优先于仅强制 lane (且 lane 在分叉首块固定, 普通子块洗不白); (b) 它抢落的仅强制候选只是 §5.6 窗口里的一个候选——健康内容尾巴由任何人带证明落进同一窗口即按全序直接胜出, 恶意候选白花 gas + 证明费。”单方零罚可反复”变成”反复必亏本”。残留收窄为一条: 窗口内没人落诚实尾巴 (兜底可行性, §6.3 同一条件)。被孤立尾巴的自由裁量交易照旧回 mempool、不销毁。

- 审查时间界：一条强制条目在”有人出一个含它的块”后 backlog / C_force 个块内被包含；含它的块可以是普通块 (强制前缀义务, §7) 或仅强制块 (全卡特尔下)。当该含它的块是经落地头的档 (ii) 块 (全卡特尔恢复路径) 时, 其出块继承 §6.4 恢复界, 含紧接落地后起停最坏叠加 $\leq F_{l1}$ 的尖端 final 等待^[158]——稳态无此项。仅强制路径 permissionless——全卡特尔下任何持任意密钥者都能出仅强制块推进, 故审查底线不依赖任何在册身份或聚合者。这继承现行部署 permissionlessInclusionMultiplier 的直觉与 v15 I6 (强制内容总会流动), 用两档父块规则 + 强制前缀义务替代了 v15 的整个无政府模式。
- 为什么保留队列 (评估文档 §6 的结论在本设计下依然成立, 且更便宜): 轮换在” ≥ 1 个诚实构建者”时给出有界等待, 但对全员卡特尔给零保证, 且链上无法检测卡特尔; 桥接消息需要不依赖任何运营方的进入路径。本设计里队列的日常成本几乎为零 (就是 §5.1 的一条规则), 灾难尾部 (v15 §6.4 的桥接召回握手等) 因为没有”作废退款”路径而大幅缩水: 条目要么被包含, 要么等 H_force 后被任何人包含——没有作废分支, 也就不需要终止取消握手。(blob 过期问题不存在: 强制条目数据本来就在 L1 上。)

8 锚定与桥 (L1 $\rightarrow$ L2)

- 沿用现行的逐块锚定模式: 每个块的第一笔交易是锚定交易 (anchor tx), 引用一个深度 $\geq D_{\text{anchor}}$ (32 L1 slot) 的 L1 区块, 且相对上一个锚定单调不减、新鲜度不超过上限 (数值沿用 v15 §6.6 的约束形态)。
- **anchor** 不得晚于本块 slot 时间——L1 $\rightarrow$ L2 因果序不变量^[159]: 除单调不减 + 新鲜度外, 还须 anchor 的 L1 时间戳 $\leq$ 本块 slot 对应的 L2 时间戳 (等价: $L2_slot(anchor.L1_timestamp) \leq slot$), 对普通块与仅强制块一律适用。为什么必须有: §6.1 规则 3 只挡”未来 slot”, §8 只挡”anchor 太旧”, 两者都不挡”旧 slot 配新 anchor”——一个构建者可扣住一个旧 slot, 事后挑一个刚到、已含某条新 L1 $\rightarrow$ L2 消息的新鲜 anchor, 签一个档 (i) 续接把该消息消费进一个 slot 时间戳早于该消息源 L1 块的 L2 块, Byzantine 聚合者抢在诚实恢复前落它 $\rightarrow$ L1 $\rightarrow$ L2 因果序被破坏、基于 block.timestamp 的桥接/ 消息时限语义失真。加此不变量后: 一个块永不能引用自己 slot 时间之后的 L1 状态。不影响恢复: 恢复块引用的是深度 $\geq D_{\text{anchor}}$ (≈ 6.4 分钟前) 的 L1 块, 其时间戳恒 $\leq$ 当轮墙钟 slot 时间, 诚实块天然满足; 本不变量只拒”旧 slot + 新 anchor”这一畸形组合。L1 入口与电路同证 (anchor.L1_timestamp 是被引用 L1 块头的纯函数)。
- 最坏路径下 anchor 年龄的时间几何 ($D_{\text{anchor_max}}$ 设置器不变量的由来, 附录 C P9a):
- 陈旧-anchor 尾巴死锁的解—— **$D_{\text{anchor_max}}$** 覆盖兜底授权期^[160]: 恶意聚合者长期扣落地时, 一条诚实尾巴的早期块 anchor 会在”落地时”超过 $D_{\text{anchor_max}} \rightarrow$ 尾巴既不能被落 (首块超期)、又不能被跳过 (parent_hash 穿过它)。解法是参数几何, 不是新机制: 设置器不变量 $D_{\text{anchor_max}} \geq D_{\text{anchor}} + \Delta_{\text{lag,final}} + P_{\text{prove,max}} + T_{\text{include,max}} + \text{余量}$ ^[161]——即新鲜度窗口必须罩住”从块产生, 到 lag_final 越阈开启兜底, 到兜底证明

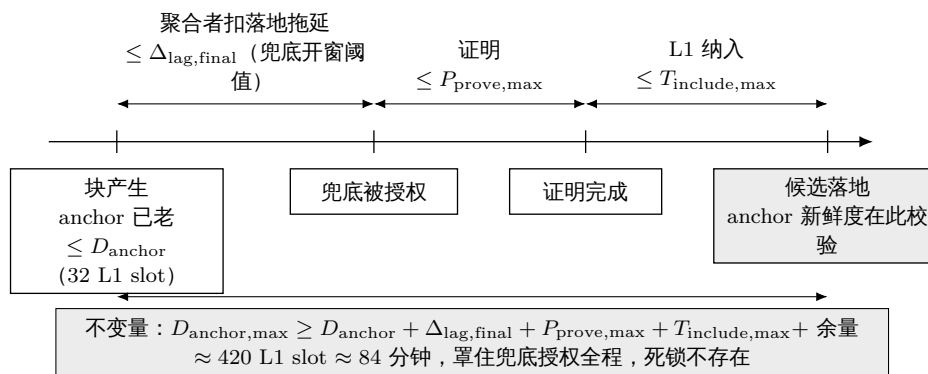


图 10: 最坏路径下 anchor 年龄的时间几何：新鲜度窗口 $D_{\text{anchor,max}}$ 必须罩住从块产生、经聚合者拖延至越阈授权兜底、再到证明与 L1 纳入的全程，这一参数关系即为陈旧-anchor 尾巴死锁的解。

并落地”的全程。于是凡兜底被授权落的尾巴 ($\text{lag_final} \leq \Delta_{\text{lag,final}}$ 时产生、越阈后被兜底落) 其 anchor 必然仍新鲜，死锁不存在。按初始参数粗算： $32 + 288 + 75 + 10 + \text{余量} \approx 420 \text{ L1 slot}$ ($\approx 84 \text{ 分钟}$)^[162]，仍是可部署的值。超出该窗口的情形 (兜底也持续缺位 $> D_{\text{anchor_max}}$)，陈旧前缀不可再落，链从窗口最终头重建——与 §6.3 兜底可行性是同一条件性残留，如实标注。因果序不变量 (上条) 不受影响。

- 因为每秒都有块 (正常情况下)，L1→L2 消息的摄入节奏是秒级的——优于 v15 中强制专用节奏的兜底路径。缺口与卡特情形下，桥接消息走 §7 的强制路径。
- L1→L2 消息处理上限 **C_anchor**——绑在【处理游标】上，不绑在 anchor 引用上^[163]：区分两个独立量：
 - **anchor** 引用 (块引用哪个 L1 区块做新鲜度基准)：仍单调不减 + 必须新鲜 (落地 L1 块高 - anchor $\leq D_{\text{anchor_max}}$)。恢复块引用一个当前的新鲜 anchor——即使它父块 (数天前的落地头) 的 anchor 很旧，这一次大跨度的单调跳增是允许的 (只是引用一个近期 L1 块，满足新鲜度)，anchor 本身不设逐块推进上限。
 - L1→L2 消息处理游标 **m_consumed** (全局消息序号，不是 L1 高度)^[164]：唯一规范算法 = 【统一最大前缀】^[165]：L1→L2 消息构成一条公开 FIFO 队列 (合约维护队列根 + 尾游标，与 m_consumed 分开)；每块必须消费 FIFO 中同时满足“条数 $\leq C_{\text{anchor}}$ ”且“累计声明 gas $\leq$ L1→L2 消息 gas 份额”的最长前缀 (限定在本块 anchor 引用高度内已到达且未消费者；取到该最长前缀即合法，少于它才不合法——同 §5.1 规则 5，再无第二套“恰好某条数”判据)。恢复块引用新鲜 anchor (覆盖数天 L1)，每块按此前缀消化，余量由后续块继续 (m_consumed 单调推进)。积压追平界相应双约束：积压清空时间 = $\max(\text{ceil}(\text{条数}/C_{\text{anchor}}), \text{ceil}(\text{累计 gas}/G_{\text{llmsg 份额}}))$ 个块 (§6.4/§9 的界按此理解)^[166]。没有这条下限，m_consumed 单调却可停滞：构建者持续处理 0 条、仍用新鲜 anchor，即可无限期审查 L1→L2 消息，使 §6.4/§9 的积压 / C_{anchor} 追平界失效^[167]。加下限后，追平界是电路强制的、可兑现的。消费不等于执行——对前置状态非法的进站消息按 §7 的 consume-and-discard 同规则弃置消费。真正的反审查界是 **D_anchor_max**，不是 **C_anchor** 单独^[168]：最大前缀只强制“消费到本块 anchor 引用高度内已到达者”，故构建者若把 anchor 钉在新鲜度允许的最旧高度 (落地头 - $D_{\text{anchor_max}}$)，可以合法地不消费任何更新的消息。但这不是无限期审查：anchor 须满足新鲜度下限 anchor $\geq$ 落地 L1 块高 - $D_{\text{anchor_max}}$ 且单调不减，随着落地头推进，该下限抬升，一条在高

度 h 到达的消息在 落地头 - $D_anchor_max > h$ 时被强制落入“anchor 引用高度内”→必被消费。故一条 $L1 \rightarrow L2$ 消息的最坏包含延迟 $\approx D_anchor_max$ （新鲜度逼迫 anchor 前沿前进）+ 积压 / C_anchor （进入待消集后的处理速率），两个参数共同定界，不是 C_anchor 单独。^[169] 为什么这样拆^[170]：§4.2 档 (ii) 恢复块要从数天前落地头一跳接回墙钟。若把 C_anchor 加在 anchor 引用上 (v1.31 原文的错误)，则恢复块的 anchor 只能从陈旧的父 anchor 前进 C_anchor 、仍然陈旧、通不过新鲜度检查——恰好在它要支持的长停摆上失效 (Codex r34-1)。改绑到消息处理游标后：anchor 引用新鲜 (满足 §8)、 $L2$ 排序一跳恢复；而海量 $L1 \rightarrow L2$ 消息按每块 C_anchor 逐块处理，完整同步用 消息积压 / C_anchor 个块 (恢复后 1 块/秒，很快)。这与 §7 强制条目每块 C_force 的积压分摊是同一手法、同一形状 (fresh 引用 + 限速游标)。 C_anchor 为 §12 校准项。

- 共享块级 gas 预算 + 单消息 gas 上限——堵住组合 gas 停链^[171]：§7 的强制前缀 (C_force/C_bridge) 与本节的 $L1 \rightarrow L2$ 消息消费 ($\min(C_anchor, \dots)$) 是两个各自有量的强制义务，但此前没有共享块 gas 预算，也没有单条消息 gas 上限。反例：块 gas 30M，强制到期前缀恰占 20M, $m_consumed$ 队头是一条前置合法、执行需 11M 的入站消息——规则 4 要它取满强制前缀、规则 5 又要它消费该消息，合计 31M + anchor 固定开销 > 块上限，而少消费任一侧又不合法 → 普通/恢复/仅强制块全都不存在合法状态转换，全诚实也永久停链，桥接/退出游标一并停死。修复（全确定性、电路 + $L1$ 可验）：
 1. 单条准入 gas 上限：入站 $L1 \rightarrow L2$ 消息与强制条目在入队时各设单条 gas 上限，超限拒入队（消息侧新增）^[172]；
 2. 共享块级预算不变量：anchor 固定开销 $G_anchor + C_force$ (含 C_bridge) gas 份额 + $L1 \rightarrow L2$ 消息 gas 份额 $\leq block_gas_limit$, 作为共识常量的设置器不变量；三方各自的“扣除他方保留配额后的保证容量”都能在任意对方流量下装进本块（沿用 §7 双队列保留配额的同一手法，扩到三方）；
 3. 确定性优先级与溢出：块内强制执行顺序 = anchor → 强制前缀（桥接优先，§7）→ $L1 \rightarrow L2$ 消息前缀；任一方按各自游标”消费到本块保证容量，余量确定性溢出到后续块”——所有游标只由合法块推进，积压只买延迟、买不到停链；
 4. 参数下调水位线： C_force/C_anchor /各 gas 份额下调时，沿用 §7 水位线不变量（新份额 $\geq$ 相应队列未消费条目最大 gas），保证任何可达状态下都不存在”已入队却装不进保证容量”者。由此三个各自限速的义务被一个共享预算协调，不再能组合出”无合法块”的死锁。gas 份额数值并入 §12。

9 各角色掉线时会发生什么（活性核算，一张表说清）

谁掉线	用户看到什么	恢复机制	恢复时间
一个构建者	它的 slot 变缺口，别的 slot 照常	无需恢复，下个 slot 自然继续	~1 秒
连续缺勤 $\leq G_{\max} - 1$ (63 slot, 下一块父距 $\leq G_{\max}$)	若干秒无新块	下一构建者按 §4.2 档 (i) 接上；缺勤者事后补签亦可桥接	~ 掉线人数 $\times$ 1 秒
连续缺勤 $\geq G_{\max}$ 个 slot, 乃至数小时/数天 (灾难级停摆)	停摆期无新块	先落最优链、再从其 final 尖端恢复 (§5.2/§6.4) ^[173] ：停摆前若有冻结长尾，落地者先落它 (兑现其预确认)，达 F_{l1} 后恢复块以其尖端为父按 §4.2 档 (ii) 出块；确无更优链时恢复块本身即最优、直接落。落更差候选在 §5.6 窗口内被更重者盖掉 ^[174]	排序 $\approx$ 一轮证明 + 落地 ($\approx 10\text{--}15$ 分钟，与停摆时长无关) + W_{settle} 窗口收盘 ^[175] + 紧接落地后起停最坏叠加 $\leq F_{l1} + D_{\text{anchor}}$ 等待 ^[176] ；有冻结长尾时含”先落尾巴”的一轮证明；L1-sync/强制积压按每块 $C_{\text{anchor}}/C_{\text{force}}$ 追平；持续推进需排班构建者以 $\geq 1/G_{\max}$ 窗口密度持续回归 (常态 $G_{\max}$ 活性底线) ^[177] 或仅强制底线 (同密度)
聚合者掉线 (诚实但离线)	无感 (预确认照常)；最终性暂缓	$\text{lag} > \Delta_{\text{lag}} \rightarrow$ 兜底窗口开启，任何人落地并计次； m_{agg} 次 $\rightarrow$ 候补晋升	最终性滞后钉在 $\Delta_{\text{lag}} +$ 兜底响应；排序服务 0 中断
聚合者在线但恶意拖延 (Byzantine, 落微批次)	同上	微批次压不住 $\text{lag} \rightarrow$ 兜底窗口照开、照计次、照换人 (§6.3)	同上 (有界——但条件于无许可兜底经济可行) ^[178] ；席位在 m_{agg} 次计次内易主
聚合者 + 无候补	同上	兜底落地持续可用 (无需席位)；拍卖出清后恢复常态节奏	排序服务 0 中断
全部构建者 (灾难/卡特尔) —— 模型外 (§1 信任模型)	无新预确认	强制条目逾期 H_{force} 后任何人以任意密钥出 ³⁷ 仅强制块 (§4.2 档 (ii) 建在落地头 + §7, 纵	强制路径 $\approx$ 各队列积压感知界 (§7)；自由裁量服务待新构建者加入

对照 v15：那里”一个席位掉线”= 20–26 分钟服务中断（有候补时）；这里同类事件是 1 秒（构建者）或 0 秒（聚合者，只影响最终性节奏）。这就是本设计存在的理由。

10 罚没与保证金总表

过错	判定方式	后果
构建者双签（同 slot 两个块头）	挑战者提交两个签名块头，L1 合约直接验签	罚没 L_{eq} ($\geq 80\%$ 销毁，余付提交者) + 移出注册表
构建者跳过前块 / 缺勤	不构成过错（出块是机会不是义务）	无罚没；跳过对跳过者可获利（费用/MEV 转移） ^[181] 。孤立深度按 §5.4 三档：相邻跳过波及单 slot；档 (i) 单块深跳过至多 $G_{max} - 1$ 个未落地块，少数派联盟接力稀疏分支可达整条未落地尾巴 $\approx \Delta_{lag}$ ^[182] ；档 (ii) 单个恶意构建者经 final 落地头一块即可孤立至多整条未落地尾巴 $\approx \Delta_{lag}$ ^[183] 。三档 worst-case 深度同为 $\leq$ 实际未落地尾巴长度（兜底按期落地时 $\approx \Delta_{lag}$ ，条件性） ^[184] 、均须在 §5.6 窗口内胜过更重诚实候选方可最终、签名在案公开归责，频率受诚实多数假设约束；缺勤者损失自己 slot 的费收入
构建者签头不发体（withhold-body，§5.5）	不可判定（”有体不发”无法证明）	无罚没；块必被跳过、自身一无所得；回执即公开证据，声誉处置
聚合者失职（兜底窗口开启期间他人落地成功 = 计次，§6.3）	L1 机械判定（lag 状态 + 兜底批次被接受的事实）	每次计次罚活性档；累计 m_{agg} 次 $\rightarrow$ 终止 + 候补晋升
聚合者取前缀/落更差链（截断后缀）	不构成可罚过错 ^[185]	无罚没；落更差链自败：§5.6 结算窗口内任何人落更重的已证明候选即将其盖掉，恶意者白花 gas + 证明费；只取前缀则后缀仍可入后续窗口
任何人提交非法批次	证明验不过，交易回滚	白花 gas，无协议后果

本表的结构性特点，按评审 r1-11 修正后的诚实表述：所有可罚没的过错，其判定要么是 L1 直接验签（双签）、要么是 L1 时钟与 L1 事实的机械判定（落地计次）——不存在需要 L2 证据链 + 仲裁期的罚没类（v15 §8b 那类瞭望塔依赖的 * 罚没 * 在这里不存在，因为”预确认”与”块”是同一个签名对象，equivocation 就是双签，直接可验）。但存在两类不可罚没、只有威慑的不当行为（跳过、扣体，§5.4/§5.5）——本设计不消除它们；精确表述^[186]：扣体对行为者无收益；跳过可获利（单 slot 的费用/MEV 转移），其频率由 §1 的诚实多数假设约束——两者都留下公开的签名证据供归责。

11 与 v15 的详细对照（给读过 v15 的读者）

- v15 的九条不变量怎么样了：I1（派生是纯函数）继承并加强（时间戳/排班全部纯函数）；I2（单一判定/计算态证书）的精神继承于落地超时判定，但不再有 epoch 判定对象；I3（epoch 总会推进）被”L1 链头总可被任何人推进（兜底落地 + 仅强制块）”替代；I6（强制内容总会流动）继承（§7，且执行更强：逐块电路义务）；I7（封存无发送者性）被”落地无权威性”推广；I8（预承诺受益人）继承（兜底报酬与罚没赏金的领取人做进证明公共输入）；I9（安全罚没用 ETH 计价）继承于 L_{eq} 。
- v15 的评审遗产不作废：拍卖骨架（§4）、过错付费模式（§6.7）、锚定新鲜度约束（§6.6）、销毁主导的罚没分成（§8）、参数设置器的不变量检查纪律，全部直接搬用。v15 中围绕”每

epoch 一次不可逆判定”的部分 (EBC、AC、默认派生、无政府模式、取消级联) 在本设计中无对应物，相应的历史评审发现不适用。

- 评估文档四点批评的回应：单点搬家 → 已解 (落地无权威 + 基于滞后量的兜底与计次，对掉线和 Byzantine 两种聚合者都有界——§6.3)；builder 过错全是瞭望塔依赖类 → 可罚没类已解 (双签 L1 直接验签)，但两类不可罚没的不当行为 (跳过、扣体) 以威慑级残留存在并明说 (§5.4/§5.5) ——是”降级并明码标价”，不是”消除”；公平交换成为承重件 → 部分分解 (中间删除被签名串联根除；残留同前)；epoch 机制需重写 → 承认，本文就是那次重写，且结果比 v15 短。

12 待定项 (open items)

1. 构建者集规模与准入细则：N_max、保证金加权抽签的权重上限 w_max、超额竞争规则、活跃度要求 (长期缺勤是否降权)。
2. L_eq 定价：单 slot 最坏可提取价值的估计方法；是否随市场状况由治理调整。
3. 落地参数： $\Delta_{lag,prov}/\Delta_{lag,final}$ ^[187]、m_agg、 δ_{slash} 、H_force、兜底报酬的指数成本上限；恢复参数^[188]：F_l1 (档 (ii) final 落地块的 L1 最终性深度)、C_anchor (§8 消息处理游标 m_consumed 的逐块条数上限——不是 anchor 推进上限，anchor 引用无逐块上限)^[189]、 Δ_{prop} (§5.2 落地深度的传播沉淀量，通常 $\ll \Delta_{lag}$)、D_anchor_max (anchor 新鲜度上限)^[190]。D_anchor_max 设置器不变量^[191]： $D_{anchor_max} \geq D_{anchor} + \Delta_{lag,final}$ (换算 L1 slot) + P_prove,max + T_include,max + 队列/时钟余量^[192] (r42 补 Δ_{lag} 项、全部加数先换算 L1 slot——独立审核第 5 轮高危 3: 本行此前漏 Δ_{lag} ，合法参数组如 130 会让被扣落地至兜底开启的诚实尾巴必然过期；合约 setter 只实现这一条最强不变量。D_anchor=32 L1 slot; P_prove,max= 最坏证明时延；T_include,max=§1 有界纳入界。另一设置器关系^[193]： $T_{include,max} < \min(W_{settle} - P_{prove,max} - 余量, D_{anchor_max} - D_{anchor} - \Delta_{lag} - P_{prove,max})$ ——v1.38 曾漏掉 anchor 在签名时已至少老 D_anchor 这一项，若只写” $\geq$ 证明 + 落地”会得出一个无攻击者也永久失败的参数组 (reviewer 反例：设 70、实需 102)。否则恢复块 (及任何块) 会在证明 + 落地窗口内 anchor 过期、须重签重证 (§6.4 anchor 新鲜度条款)；这是恢复”不因缺口过期”的前提，把旧 s_ra 的 64 秒结构性窗口换成可设足够大的自由参数。如实标注：有限 D_anchor_max 仍是”证明 + 纳入延迟 vs 新鲜度窗口”，只是窗口可配置到足够大，故 C1 明确条件于”临时 L1 拥堵/审查 $\leq D_{anchor_max}$ ”这一有限纳入假设，不宣称对任意长审查无条件。结算窗口参数^[194]：W_settle (结算窗口长度，L1 块计；设置器不变量：下界 $W_{settle} \geq P_{prove,max} + T_{include,max} + 余量$ ，共识上界 W_settle_max (含 L1 缺 slot 墙钟换算余量——上界钉住 §6.2 可撤销敞口 $\Delta_{lag} + W_{settle}$ (墙钟) + 兜底响应)^[195]，初始建议 ≈ 100 L1 slot ≈ 20 分钟)；窗口状态的 L1 重组回滚并入第 9 项。final_ref 语义 (§4.2 档-(ii) 块头字段，证签名时父已 F_l1-final，已入块头元组与 §5.1)。 Δ_{prop} 降为落地者视图完整的链下策略参考 (§5.2, 非链上判责参数)。^[196] 共享 gas 份额 (§8)：G_anchor、C_force/C_bridge gas 份额、L1→L2 消息 gas 份额，满足三者之和 $\leq block_gas_limit + 单条准入上限 + 水位线$ ^[197]。(旧重锚 episode 的 $\Delta_{ra}/\Delta_{ra_ext}/B_{ra}/B_{ch}/\Delta_{cont}$ 随 episode 删除，不再是参数。) 计次机制在极端拥堵 (L1 费用尖峰导致无人愿兜底) 下的行为。§8/§9 中围绕这些参数给出的具体界 (积压/C_anchor、恢复时间、D_anchor_max 反审查界等) 在参数标定前均为【暂定值】^[198]——形态确定、数值待定。
4. 聚合者反向拍卖细则：服务费率资金来源 (协议费/金库)、费率与保证金的评标函数、候

补队列的绑定性。

5. **P2P 层规范**：签名块的传播协议、块体可用性的工程保障（预确认回执格式、构建者间的中继义务、纠删码传播是否值得）、跳过与扣体行为的公开度量（做成公共看板，威慑的抓手——§5.4/§5.5 的威慑级残留全靠它落地）。
6. **证明电路成本**：逐块验签（最多 384 签/epoch）+ 排班重算在电路内的成本评估；BLS 聚合签名是否值得引入（把逐块 ECDSA 换成可聚合签名，降低电路与潜在的 L1 直验成本——涉及构建者密钥体系选型）；批次证明的增量续证/聚合能力^[199]：链头被小批次推进后，针对旧链头在证的批次须能复用已证前缀续证，不得从头重证——这是滴灌拖延战术无牙化的前提。
7. **多批次并发与重组**：兜底窗口内多方落地竞争的 gas 竞争处理（预计沿用 v15 §11.3 的“竞争者自担成本、链照常推进”结论，需复核）。
8. **证明系统中断的豁免机制**^[200]：形态沿用 v15 §10.4 第 3 级（独立证实 + 有界豁免 + H_{toll_max} 型上限），在没有取消级联的前提下大幅简化，需要一次专门推导；在它完成前，§6.3 的“零误伤”声明仅限兜底 strike。
9. **L1 重组处理**：落地批次遇 L1 重组的回滚语义（预计显著简单于 v15 §3.1/§13-S.1，因为没有跨交易的证书状态机，但需写明）。
10. **迁移路径**：从现行部署 / 从 v15 到本设计的升级顺序。
11. **交互时序图**^[201]：批次落地、等高分叉消解、兜底窗口、仅强制块逃生阀四者的交互目前分散在 §5–§7，值得一张规范化时序图。
12. **强制条目的 nonce 抢占防护**^[202]：见 §7；在每源强制 nonce / nonce 冻结 / 桥合约构造三方案中选定并形式化前，强制路径对 nonce 抢占非无条件。
13. **兜底证明者的可得性与成本**^[203]：并入威胁模型——§9 的 Byzantine 聚合者活性界条件于此；极端拥堵/证明成本 > 报酬下的兜底激励设计。
14. **saveForcedInclusion 入队验证的 L1 成本上界**^[204]：L1 上解码 L2 签名交易 + 验签 + chainId + 份额检查有非平凡 gas 成本、且面向畸形 calldata 有 griefing 面。需验证自身的 gas 上限与不可解析提交的廉价拒绝路径（提交者付费），或把验证下放到更便宜的域。
15. **lag 阈值余量的最坏情形校准**^[205]： $\Delta_{lag,prov}$ (4 epoch ≈ 25.6 min) 对正常 provisional 滞后带上界 (20 min) 只余 ≈ 5.6 min； $\Delta_{lag,final}$ (≈ 9 epoch ≈ 57.6 min)^[206] 对稳态 lag_final 带 (≈ 35 – 40 min) 的余量由 w_{settle_max} 的余量项承担。证明尖峰 + L1 最终性抖动叠加下可能把尽责聚合者推过阈值、误开兜底窗口。需评估是否加宽。
16. **强制包含数据载体迁移**^[207]：§7 把强制条目载荷从 blob 引用改为 calldata（永久可重建），但现行 MainnetInbox 存的是 LibBlobs.BlobReference。需要一条迁移/兼容路径：已入队的 blob-ref 条目在切换时如何处理（重放为 calldata、宽限期内双载体受理、或旧队列排空后再切）——否则升级瞬间的在队条目可能被搁浅。
17. **创世/引导语义**^[208]：初始 lag（无落地头时 lag 可能任意大、兜底窗口从创世即“开启”）、首个聚合者选取、首批次落地前的计次会计，均未定义。需定创世锚点（genesis landed head）+ 引导期兜底/计次的抑制规则。
18. **形式化规范补强——实现前的门**^[209]：（已交付，r43）§5.2 全序与 §5.6 窗口状态机的可执行参考模型 + 性质测试（P1–P7，含第 5 轮两项阻塞的全部不变量）——见附录 C 与 settlement-window-model.py/settlement-window- RESULTS.md，改规则必须同步改模型重跑。（已交付，r44）后半：模型补 P8–P11（桥接预留、anchor 几何/因果序、罚没生效时点、兜底

会计快照，19 项断言全过)+ 实现前复核文档 `settlement-window-implementation-review.md` (Solidity 级 `acceptCandidate` 存储/gas、Inbox 集成路径、下列 (a)/(b)/(c) 的逐项处置与仍开放项清单——其中 (b) lookahead 抽样算子的精确定义仍开放，electing 实现前必须闭合)^[210]。原清单:(a) 一份块合法性 / 父块选择 / 最优链全序 (§5.2 四元组，含反自反/完全/传递性质测试)^[211] / 结算窗口 `openWindow/acceptCandidate/replaceCandidate/closeWindow` 状态机 (§5.6，含双候选取代、强制/消息非空时的基线冻结、L1 重组与 lazy-close 测试)^[212] / 落地 / 强制包含 / 窗口 challenge (§5.6) / 共享 gas 预算与溢出 (§8) / L1 重组下 landed head+ 状态根 + `F_consumed/m_consumed+lag` 的原子回滚 (§12 第 9 项) 的伪代码或状态机^[213]；(b) lookahead 的确定性加权抽样算法的精确定义——r47 起 §3.2 给出完整可执行候选算子 (capped 权重前缀和 + 种子模总权重定位的 Python 代码，lookahead-model.py 可运行验证)，待所有者确认该算子后本项闭合；(c) 术语上把 L2 接受最终性与 L1 最终性 `F_l1` 用不同词区分 (“final” 混用使两档父块规则更难推理)。

参数总表^[214]

参数	初始值	单位	关键不变量 / 出处
slot	1	秒	§3.1
epoch	384	slot	§3.1
H_look 前瞻视界	768	slot (≈ 2 L1 epoch)	§3.2
D_snap 快照延迟	5	L1 epoch	$\geq H_look + F_final + \text{余量}$, §3.2 r8-1
w_max 权重上限	20%	—	§3.2
N_max 注册容量	64	地址	§4.1
G_max 缺口上限 (档 (i) : 缺口 $\leq G_max$, 父块落地状态无关) ^[215]	64	slot	§4.2 ; 深度旋钮非硬帽 r20-1
G_max_landed (档 (ii) : 缺口无上限, 父须 L1-final 落地块)	∞ (无上限)	—	§4.2 方向 B ; 论证见 v1.31 记录
F_l1 final 深度	待定	L1 slot	档 (ii) 父块最终性深度, §4.2
C_anchor L1→L2 消息处理上限	待定	消息/块	§8 绑 m_consumed 游标, r34
Δ_prop 落地传播沉淀	待 定 ($\ll \Delta_lag, prov$)	slot	§5.2 落地深度, r33
δ_slash 罚没生效延迟	64	slot	§4.3
$\Delta_lag, prov$ 服务观测阈值	4	epoch (≈ 25.6 min)	$\geq$ 正常 provisional 滞后带上界, §6.2 r10-2/r44 ; 不进入罚则
$\Delta_lag, final$ 兜底/计次阈值	≈ 9 ($= \Delta_lag, prov + 128 + W_settle_max + 150 + \text{余量}$ 10, L1 slot 计 = 288) ^[216]	epoch (≈ 57.6 min)	稳态 $lag_final \approx lag_prov + W_settle$ (35–40 min) > 旧阈值, 不抬则兜底永开 ^[217] ; §6.2/§6.3
W_settle_max 窗口共识上界	待 定 ($\approx 1.5 \times W_settle_max$)	L1 块 (含墙中换算余量)	钉住可撤销敞口与 $\Delta_lag, final$, §6.2 r42 中危 1
G_strike 计次限速	1	epoch	§6.3 r13-1
m_agg / m_agg'	2 / 4	次	终止阈值, §6.3
F_delay 强制到期延迟	待定	slot	唯一到期谓词 $due_slot = submission_slot + F_delay$, $submission_slot := L2_slot$ (入队交易所所在 canonical L1 块 timestamp) (不取 L2 头——巨大停摆下两解分叉) ^[218]
H_force 逾期兜底阈值	1	epoch	§9 事件用 : 到期后再逾 H_force 触发任意密钥仅强制块, 非第二套到期判据, r39
C_force / C_bridge	待 定 / $25\% \times C_force$	gas+ 条目	单条目份额 = 扣对方保留后, §7 r19-1
D_anchor / D_anchor_max	32 / 待定 ^[219]	L1 slot	锚定深度/新鲜度上限 ; $D_anchor_max \geq D_anchor + \Delta_lag, final(L1\ slot) + P_prove, max + T_include, max + \text{余量}$ ^[220]
W_settle 结算窗口	待定 (≈ 100 L1 slot ≈ 20 min)	L1 块	§5.6 : $\geq P_prove, max + T_include, max + \text{余量}$, r41 option C
ϕ_land 基础费分成比例	待定	—	候选链 base fee 总和 $\times \phi_land$ 于收盘记入赢家受益人, §5.6/§6.5 ^[221]
gas 份 额	待定	gas	§8 : 三者和 $\leq block_gas_limit + \text{单条准入上限} + \text{水位线}$, r39
G_anchor/C_forcegas/C_llmsgas			

(数值为初始建议，标”待定”者属 §12 校准项；单位换算：1 L1 slot = 12 s = 12 L2 slot。)

附录 A：与所有者建议的分歧点^[222]

1. 排班随机数不混入证明哈希。所有者建议“L1 共识层数据 + 最近证明的哈希”；本文只取前者。理由 (§3.2)：证明哈希是出证明者可零成本重磨的熵，混入它把排班部分控制权交给聚合者，方向与去信任目标相反。纯 L1 信标随机数的 1-bit 级偏置残留对排班用途可接受。
2. 父块链接用块头哈希，不用父块签名。所有者两案并提；本文选哈希为共识规则（结构必需且传递承诺整条历史），父块签名进子块降级为可选的证据打包习惯 (§4.2)。
3. 跳过 (skip) 残留定性为威慑级并接受。签名串联消除了聚合者的中间删除，但相邻构建者跳过在协议内不可证伪 (§5.4)。本文选择明说并以公开性 + 声誉 + 单 slot 损失上限处置，不引入不可判定的“我发布过”仲裁。如所有者希望结构级消除，已知代价是每 slot 向 L1 发布承诺（费用回到聚合省费之前）——不建议。（这句就是 §1 所引的成本恒等式^[223]：为预落地阶段购买协议级承诺/DA 的费用 $\approx$ 本设计靠聚合批量落地省下的费用——买了前者，后者的节省就不存在了。）
4. 基础费分成的支付时点：收盘，而非 **provisional** 落地。所有者建议“base fee 分配应该在 landing 时直接分给聚合者”；本文选择在结算窗口收盘的原子提交中记账 (§6.5)。理由：provisional 落地在窗口内可被更重候选取代 (§5.6)，落地即付就必须为被取代者设计追回机制 (clawback) ——那是一个新的状态机与攻击面；收盘支付则是收盘原子提交的一部分，零额外机制，且恰好实现“赢家通吃”的激励（被取代者分文不得，落短链的机会成本最大化）。代价只是落地者的现金流推迟一个 w_settle (≈ 20 分钟)。若所有者坚持落地即付，需要先接受 clawback 状态机的复杂度回流。

附录 B：术语对照表

中文	英文	说明
槽位	slot	L2 的 1 秒时间单位
排班表	lookahead	slot $\rightarrow$ 构建者地址的确定性映射
构建者	builder	按排班出块并签名的预确认者
聚合者	aggregator	打包 + 证明 + 落地的服务席位
签名链	signature chain	由 parent_hash 串联、逐块签名的未落地链
落地	landing	批次 + 证明上 L1 成为候选；窗口收盘最重者最终 ^[224]
批次	batch	一段连续签名链 + 数据 + 证明
缺口	gap	无合法块的 slot
跳过	skip	构建者签名选择不接前一个已存在的块
兜底落地	fallback landing	聚合者超时后任何人落地并领偿
仅强制块	forced-only block	排班豁免、只含到期强制条目的块
双签	equivocation	同 slot 签两个不同块头，L1 直验罚没
两档父块规则	two-tier parent rule	判据是缺口：(i) 缺口 $\leq G_{\max}$ （父落地状态无关）；(ii) 缺口无上限但父须 L1-final 落地头 ^[225]
恢复块	recovery block	停摆时按档 (ii) 建在 L1-final 落地头上、接回墙钟的普通/仅强制块 (§6.4)
良性搁浅	benign stranding	恢复块的父尖端被推进时该块作废、重建即可，不损坏状态 (§6.4)
弃置消费	consume-and-discard	对前置状态非法的强制/入站条目照常推进游标但不执行 ^[226]
最优链全序	best-chain total order	候选自身四元组 (lane, count, tip_slot, tip_hash) 字典序；lane = 候选自冻结基线起首块类别（内容 1 > 仅强制 0）、整条继承不可洗白 ^[227]
落最优链策略	best-chain landing strategy	落地者按 §5.2 全序落最优链——非义务：落更差链在 §5.6 窗口内被更重候选盖掉、自败 ^[228]
结算窗口	settlement window	首个延伸窗口最终头的候选落地后开启、W_settle 个 L1 块后确定性收盘 ^[229]
候选批次	candidate batch	延伸窗口最终头、带有效性证明的已落地批次；窗口内可被按 §5.2 全序严格更重者取代 (§5.6)
窗口最终	window-final	结算窗口收盘时的最重已证明候选；提款/桥接只从此状态执行 (§5.6/§6.1)
(已删) 挑战罚没 / 链头承诺	challenge / head-commitment	v1.39/v1.40 的问责与承诺层，独立审核第 3/4 轮证明无 DA 不可机械化，r41 由结算窗口取代
(已删) 重锚 episode	re-anchor episode	v1.8–v1.30 的恢复子协议，v1.31 方向 B 整体删除，由两档父块规则取代 (§5.6 现为结算窗口最终性) ^[230]

附录 C：结算窗口可执行参考模型（normative pseudocode + 性质验证）

§12 第 18 项”实现前的门”的交付^[231]。规范性伪代码以本附录与 §5.6 正文为准；可执行版本在 *settlement-window-model.py* (零依赖 *Python*, 直接运行), 验证结果在 *settlement-window-RESULTS.md*。纪律: 凡修改 §5.2 全序、§5.6 窗口状态机、§7/§8 游标与 *gas* 规则, 必须同步改模型、重跑、更新 *RESULTS*——三者不同步视为规范缺陷 (沿 *v15 model_checker* 的同一做法)。

已验证的性质^[232]：

性质	内容	对应发现
P1	全序 key (lane, count, tip_slot, tip_hash) 反自反/完全/传递；第 5 轮 A>B>C>A 环消解为 B>C>A	严重 1
P2	收盘赢家与候选提交顺序无关（全排列验证）	严重 1 推论
P3	强制/消息队列非空时:provisional 不改 canonical；更重候选对同一冻结基线可验证并取代；收盘恰好提交赢家终局一次；游标单调无重复消费	严重 2
P4	lane 洗白抵抗：仅强制首块的 13 块链 < 3 块内容链	第 3 轮发现 3
P5	lazy close 不改赢家；收盘后候选只能开启下一窗口	收盘边界
P6	窗口态是 L1 历史纯函数；浅重组截断后重放一致、被重组候选原子消失	L1 重组
P7	共享 gas 预算 + 单条准入下，300 组随机可达队列状态（含非创世游标）均存在合法块；双约束最大前缀守上限	第 4/5 轮 gas 死锁
P8	桥接队列满载洪泛下，普通强制队列每块仍消费 $\geq$ 保证容量 (C_bridge 预留生效，不被饿死)	r19-1;DeepSeek-on-v1.43 C2 ^[233]
P9	设置器不变量在独立声明的部署值间互检 ($\Delta_{lag,final} \geq prov+W_{settle_max}$ 、 $D_{anchor_max} \geq$ 最坏路径、 $W_{settle} \geq$ 证明 + 纳入) ^[234] ；因果序 $anchor.L1_time \leq L2_time(slot)$ （显式时基，含等号边界）	§8/§12；r44 引入，r46 非空化 ^[235]
P10	罚没生效按候选 L1 落地时点判：生效后落地拒含该 signer，生效前已落地祖父化	§4.3 r41；门后半 ^[236]
P11	兜底资格按 $lag_final > \Delta_{lag,final}$ 开窗即快照、保持到窗口收盘；provisional 短候选清零 lag_prov 不撤销资格	§6.3 r42/r44 高危 2
P12	窗口中途入队：队列 append-only 按序号引用，基线只冻结游标/状态——先前候选终局不变、更重候选可确定地消费新条目、入队时点不影响验证结果	§5.6 r46 澄清 ^[237] 47

模型边界（如实）：签名/证明/执行合法性为占位（模型只精确建模本设计新增的共识对象：全序 key、窗口状态机、游标算术、gas 份额、时序几何）。r44 起 anchor 几何/因果序、罚没生效时点、兜底会计快照已入模型（P9–P11）；Solidity 级 acceptCandidate 入口的存储布局与 gas 成本是分析性（非可执行）复核，见 settlement-window-implementation-review.md。基础费分成（§6.5 ϕ_{land} ）^[238] 是收盘记账的经济动作、不改变窗口态与游标语义，暂不入模型。最终判定 = 所有者 + 人类安全评审——模型与复核文档是门，不是签字。

附录 D：版本变更历史（设计过程记录）

逐版本记录每一轮对抗评审的发现与修复（新在前、旧在后）。这是设计的完整论证轨迹，非规范性；正文与之冲突时以正文为准。

草案 v1.48, 2026-08-27——排版线改为 *LaTeX*：单栏学术论文式 *PDF*，评审注记外置为附录 E。所有者指示”转成 *LaTeX*、单栏、直接给 *PDF*；图要黑白灰、学术论文风格；所有引用自他人所有者的评论作为附录放到最后”。三项落地：(1) 新增 *build-pdf.py* (*Markdown* → *XeLaTeX* + *ctex* → *slot-chain-spec.pdf*, 68 页, *A4* 单栏)，含标题页/摘要/目录/页眉页脚，§ 与附录引用为可点击交叉引用；正文章节编号与文档自身的 §0–§12 对齐，附录 A–E 不参与编号。(2) 10 幅 *mermaid* 图全部重画为 *TikZ* 灰阶图 (*tex/figures.tex*, 仅黑/白/灰，含一幅真正的时序图)，随正文浮动排版并带图题。(3) 评审注记外置：正文中约 310 处形如 “[239]” “[240]” 的出处括注被抽为尾注 [n]，集中到新增的附录 E；实质内容在前、出处在后的括注只搬出处部分，参数值等规范内容仍留在正文。删除 *HTML* 线 (*slot-chain-spec.html*、*build-html.py*)；*tex/* 下只入库手写的 *figures.tex*, *main.tex* 与编译中间件由 *.gitignore* 排除。规范语义、参数、两个模型 (21 + 6 项断言) 均未改动。草案 v1.47, 2026-08-27——所有者三项指示：最长链的可见性定性 + 基础费分成激励、排班表代码化、可读性再修。(1) §5.2 新增两条：可见性假设与“只经济、不强制”的定性——协议不能强制落地者落最长签名链（链尾构建者扣签名即可剥夺可见性，非过错），落短链只承受经济后果；节点本地选链规则明确为既有四元组全序（含双签分叉情形——双签被罚但两条链仍可比较，视图收敛不断）；并记录“费用总和和进全序”的取舍（费用可自付刷量，进正统性判据等于让资本买断诚实多数——费用只进报酬）。§6.5 新增基础费分成：候选链全部 *base fee* 之和 × ϕ_{land} 在窗口收盘的原子提交中记入赢家受益人，分成基数随链长单调、被取代者分文不得；支付时点取收盘而非 *provisional* 落地（免 *clawback* 状态机），与所有者建议的偏差记入附录 A-4, ϕ_{land} 入参数表。(2) §3.2 嵌入排班表完整可执行 *Python*（逐步注释），新增 *lookahead-model.py* (6 项性质：纯函数/窗口对齐/快照最终性几何/ w_{max} 封顶与权重成比例)，抽样算子 (*capped* 前缀和 + 种子取模定位) 为 §12-18(b) 的定形候选，待所有者确认后该项闭合。(3) 可读性：全文去除约 670 处句中加粗（保留行首标签式加粗约 300 处）；*HTML* 生成器修复嵌套列表解析 (*tab_length* = 2 + 段落后续列表的空行补全)、§/附录引用自动变锚点链接 (700+ 处)、粗体减重为 600。草案 v1.46, 2026-08-26——*DeepSeek-on-v1.45* 批次：模型检验力修复 + 一处真实数值矛盾重校。(W1) P9a 非空化——并当即抓出 *r44* 的数值松弛：模型此前把 *D_ANCHOR_MAX* 定义为“最坏路径 + 5”再断言“最坏路径 ≤ *D_ANCHOR_MAX*”，恒真、无检验力。修：时序参数全部改为独立声明的部署值字面量（照抄参数表），P9a 用不等式互检。非空化后立即失败——*r44* 的 $\Delta_{\text{lag}, \text{final}}$ 初值 8 epoch (51.2 min) 低于其自身公式 $\Delta_{\text{lag}, \text{prov}} + W_{\text{settle}_{\text{max}}} = 128 + 150 \text{ L1 slot}$ (55.6 min)。重校： $\Delta_{\text{lag}, \text{final}}$ 初值 9 epoch = 288 L1 slot ≈ 57.6 min；连带 *D_anchor_max* 初值估算 380 → ≈420 L1 slot (≈84 分钟) (§6.2/§6.3/§8/§12/参数表/两幅图同步)。(W2) 基线冻结语义澄清 + P12：冻结对象是【游标与状态】，不含队列内容——队列 *append-only*、按序号引用、内容不可变，故窗口中途入队对能覆盖到它的后续候选可见且确定；实现上队列承诺须逐条/*append-only*（哈希链或 *MMR*），不得用落地时刻的单一可变队列根做公共输入 (§5.6 新条 + 实现前复核文档 *baseCommit* 一节同步)。模型新增 P12a/P12b（中途入队不改先前候选结局；入队时点不影响验证结果），21 项断言全过。(W3) *HTML* 再生成纪律：*build-html.py* 加 *--check* 模式（再生成并 *diff*，脏则非零退出），*README* 写明“改 *md* → 重新生成 + *check*”约定；*monorepo* *CI* 工作流不在本 *docs* 分支范围，列为合入时的仓库级跟进。（建议）P9b 强化为按显式时基函数检验 $\text{anchor.L1_timestamp} \leq \text{L2_timestamp}(\text{slot})$ （含等号边界）；*build-html.py* 用 *with* 管理文件、注明依赖版本。草案 v1.45, 2026-08-26——可读性修订^[241]。第 1 轮（结构）：约 400 行版本变更历史从文档顶部移入本附录，正文前新增“如何读这份文档”导读^[242]与目录。第 2 轮（排版）：中文语境内的半角逗号/分号/冒号统一为全角（约 630 处，代码块与行内代码不动）。第 3/4 轮（图示）：新增 10 幅 *mermaid* 图嵌入正文——§0 全链路通路与缺口时间线、§3.2 排班快照几何、§4.2 两档父块规则判定流程、§5.6 结算窗口生命周期、§6.2 四级确定性阶梯与双阈值、§6.3 兜底与计次因果链、§6.4 停摆恢复时序、§7 双队列最大前缀消费、§8 *anchor* 新鲜度几何——

全部以 *diagram-as-code* 形式 (*GitHub* 原生渲染, 非贴图), 并经 *mermaid-cli* 逐幅渲染验证。§0 摘要句拆分润色, 正文对旧“变更记录”的引用改指本附录。第 5 轮 (终读): 全文复读、残留修补, 并新增 *HTML* 版 *slot-chain-spec.html* (由本 *markdown* 生成, *mermaid* 客户端渲染, 便于浏览器阅读)。规范语义、参数、模型 (19 项断言) 均未改动。草案 **v1.44, 2026-08-26**——*DeepSeek-on-v1.43* 批次修复 + §12 第 18 项“实现前的门”后半交付。(C1, 高危) 拆阈值: $\Delta_{lag, prov} / \Delta_{lag, final}$ ——*r42* 把兜底/计次判定量换成 lag_{final} 却沿用旧阈值 Δ_{lag} ($4\text{ epoch} \approx 25.6\text{ min}$); 窗口最终性下稳态 $lag_{final} \approx lag_{prov} + W_{settle} \approx 35\text{--}40\text{ min}$ 本身就高于旧阈值 → 健康系统里兜底窗口永久开启、尽责聚合者被持续计次终止。修: 两个 lag 各配阈值, $\Delta_{lag, final} = \Delta_{lag, prov} + W_{settle_max} + \text{余量}$ (初始 $\approx 8\text{ epoch} \approx 51.2\text{ min}$), §6.2/§6.3 全部罚则判定改用之; §6.3 加术语约定 (裸 lag/Δ_{lag} 读作 *final*, $W3$ 清理); 涟漪 D_{anchor_max} 公式的 lag 项同步升为 $\Delta_{lag, final}$ (兜底以 lag_{final} 开窗, 新鲜度窗口须罩住更晚的授权时点; 初值估算 $250\text{--}380\text{ L1 slot}$)。 (C2) 模型桥接预留漏建——*settlement-window-model.py* 曾把 C_{force} 全额给桥接队列, 漏 *r19-1* 的 C_{bridge} 预留; 修 + 新增 $P8$ (桥接满载洪泛下普通队列不被饿死)。 (W2) 收盘先于接受规范化——§5.6 伪代码显式 $L1.now < close_at$ 才可受理, 同高度先收盘再受理 (与模型 *replay* 语义一致)。 (W1) 附录 *B*“最优链全序”词条更新为 *r42* 四元组。门后半交付: 模型补 $P9$ (anchor 几何/因果序)、 $P10$ (罚没按候选落地时点)、 $P11$ (兜底资格快照), 19 项断言全过; 新增 *settlement-window-implementation-review.md* (Solidity 级 *acceptCandidate* 存储/gas 分析、*Inbox* 集成路径、§12 第 18 项 (a)/(b)/(c) 逐项处置; 最终判定 = 所有者 + 人类安全评审)。改动落点: §5.6、§6.2、§6.3、§8、§12 (第 3/15/18 项、参数表)、附录 B/C、模型与新文档。草案 **v1.43, 2026-08-26**——§12 第 18 项“实现前的门”前半交付: 结算窗口可执行参考模型 + 性质验证 (附录 C)。新增 *settlement-window-model.py* (零依赖可运行) 与 *settlement-window-RESULTS.md*: 把 §5.2 全序 *key* 与 §5.6 窗口状态机 (基线冻结/候选版本化/收盘提交/lazy close/L1 重放) 及 §7/§8 双约束游标算术写成可执行模型, 验证 $P1\text{--}P7$ 共 14 项断言全过——含第 5 轮两项阻塞的全部不变量 ($A/B/C$ 环消解、提交顺序无关、*provisional* 不改 *canonical*、无重复消费)、*lane* 洗白抵抗、*lazy close*、 $L1$ 重组纯函数、300 组随机队列状态无 *gas* 死锁。纪律: 改规则必须同步改模型重跑 (沿 *v15 model_checker* 做法)。§12 第 18 项后半 (实现前复核: 模型未覆盖项 + Solidity 级入口) 仍开放, 动手实现前必须完成。改动落点: §12 第 18 项、附录 C (新)、新增两个文件。草案 **v1.42, 2026-08-26**——独立审核第 5/5 轮 (终轮) 修复: 结算窗口的数学与状态机闭合。终轮结论: *option C* 方向正确 (承诺头/挑战两层的死穴确认消除), 但 *v1.41* 执行有两处阻塞: (严重 1) 全序不传递——旧“看两链首个相异块定 *lane*”是 *pairwise* 判据, 可构造 $A > B > C > A$ 环, 收盘赢家随提交顺序漂移。修: *lane* 改为候选自身的不变标量 (= 候选自 F 起第一个块的类别, 整条继承), 比较 = (*lane*, *count*, *tip_slot*, *tip_hash*) 四元组字典序——天然全序, 仍防洗白 (§5.2)。 (严重 2) *provisional* 候选推进 *canonical* 游标——首候选一落就推 $F_{consumed}$ /状态, 第二个更重候选起始游标对不上、无法参赛, “先落者永久赢”复辟。修: §5.6 补 基线冻结 + 候选版本化 + 收盘提交状态机 (窗口开启冻结 $base = (F, stateRoot, F_{consumed}, m_{consumed})$, 全部候选对同一 *base* 验证、各存自己的 *end* 元组, 收盘只把赢家 *end* 一笔原子提交; *openWindow/acceptCandidate/closeWindow* 伪代码入正文; 如实承认这是小型窗口状态机) [243], §7 游标规则改读、§5.2“先落者正统”残文删除。 (高危 1) 有界纳入假设明示——固定 $W_{settle}/D_{anchor_max}$ 截止推不出对临时审查的无条件稳健; §1 显式新增 $T_{include, max}$ 假设 + 设置器关系, 相关表述改条件性。 (高危 2) 兜底会计以 lag_{final} 判定并快照保持到收盘——防短候选清 *provisional lag* 掐断纠错报酬; 报酬付赢家与每个严格改进者。 (高危 3) D_{anchor_max} 公式三处统一补 Δ_{lag} (漏则合法参数组必然搁浅诚实尾巴)。 (中危 1) 拆 lag_{prov} (服务) / lag_{final} (安全), 可撤销敞口如实改 $\Delta_{lag} + W_{settle}$ (墙钟) + 兜底响应, W_{settle} 加共识上界。 (中危 2) $submission_slot := L2_slot$ (入队 $L1$ 块 *timestamp*) 唯一化 (巨大停摆下“取 $L2$ 头”另解会分叉)。一致性: 档 (ii) 父须 *window-final* 且 $L1-final$ (等待界 $\max(W_{settle}, F_{l1}) + D_{anchor}$)、§5.1 P_{forced} 摘要补档 (i) 续接、§4.3“不误伤”措辞改为承认双签后继级联敞口、 δ_{land} 表行/挑战残文/术语表清理。改动落点: §1、§4.2、§4.3、§5.1、§5.2、§5.6、§6.2、§6.3、§7、§9、§12、附录 B。草案 **v1.41, 2026-08-26**——所有者决定 *option C*: 结算窗口最终性, 整体取代挑战/承诺层 + 独立审核第 4 轮修复。第 4 轮证明 *v1.40* 的“轻量承诺”仍不成立: 链头承诺证不了块体可得 (承诺头后扣体 → 诚实落地者落唯一可落的链 → 攻击者事后揭体挑

战 → 诚实者被冤罚/全体停服)，单一承诺账本会被一次择差落地永久投毒、有限审查仍穿透承诺窗口——修好它 = 完整 DA + 分代承诺历史 + 挑战/押金状态机 = *episode* 复杂度全量回流。所有者据此换根 (*option C*)：删除整套挑战/承诺层^[244]，新 §5.6 = 结算窗口最终性——落地不再”接受即最终”，而是窗口 W_{settle} 内最重的已证明候选收盘即最终；更优的链自己带证明落进窗口直接赢，不需要 $L1$ 事后裁决”该落而未落”。比较只在都已落地、都已证明的候选间进行，故：块体可得性问题不存在（拿不出体成不了候选）；无跨窗口状态，无投毒；收盘狙击是良性（更重 = 更多真实签名块 = 更完整的尾巴最终化）；固定窗口即可，无棋钟。落更差链从”可罚过错”改为自败（被盖掉白花成本）——§5.5/§6.3/§10 相应撤回”落错链可罚没”。成本（明码）：最终性 + 一个 W_{settle} （≈20 分钟，提款从 *window-final* 执行）；设置器 $W_{settle} \geq P_{prove,max} + T_{include,max} + \text{余量}$ 。连带解决：陈旧-*anchor* 尾巴死锁改由参数几何解（ $D_{anchor,max} \geq D_{anchor} + \Delta_{lag} + P_{prove,max} + T_{include,max} + \text{余量} \approx 250 L1 \text{ slot}, §8$ ）；罚没祖父化判据 = 候选批次的 $L1$ 落地时点（不可伪造，§4.3，删 δ_{land} ）；*lag* 以最优 *provisional* 候选计（活性不受窗口拖累，§6.2/§6.3）。第 4 轮独立 *bug* 同修：§8 删净”恰好 $\min(\text{条数})$ ”残文、统一双约束最长前缀 + 积压界改 $\max(\text{条数}/C_{anchor}, \text{gas}/\text{份额})$ （严重 3）； G_{strike} 单位统一 $L2$ 秒 + $L1 \text{ slot}$ 换算（中危 3）； H_{force} 逐块重验、*lane* 续接不继承成熟资格（中危 1）；恢复界补 + D_{anchor} 项（中危 2）；*final_ref* 入块头元组与 §5.1（一致性 1）；§10 三档深度改条件性；§12 C_{anchor} 措辞修正（一致性 7）。自审六轮已并入 §5.6 正文（狙击良性/固定窗口/参数不变量/ $L1$ 纯函数/反刷/残留三条）。改动落点：§4.2、§4.3、§5.1、§5.2、§5.4、§5.5、§5.6（整体重写）、§6.1、§6.2、§6.3、§6.4、§7、§8、§9、§10、§12、附录 B（§5.7 删除）。草案 v1.40，2026-08-25——所有者决定 *option B*：加轻量【链头承诺层】使挑战可机械执行 + 独立审核第 3 轮修复。第 3 轮证明 v1.39 的挑战（纯验签）无法机械证明竞争链”落地前已存在”（块自报 *slot* 可事后补签造假）也无法证”执行有效”——要么误罚诚实落地者，要么需 DA /承诺/证明才能成立。所有者据此选 *option B*：新增 §5.7 链头承诺累加器——*permissionless*、*bonded* (L_{commit})，任何人把更优合法签名链的链头 + 权重（几十字节、无块体、无 ZK ）提交上 $L1$ ， $L1$ 盖不可伪造的时间戳；这是部分 DA （仅链头），明码接受其成本换”挑战可机械判定”。§5.6 挑战改锚定在承诺时间戳上：竞争链”落地前已存在”由 $L1$ 承诺时间证明（关掉事后补签，发现 2a），分歧点须在 B 已落区间内（发现 2a 的高 *slot* 延伸挑战，一致性 3），无效链由落地者 *validity* 反证 → 罚承诺者 L_{commit} （关掉无效块冒充，发现 2b）；只罚不改最终性不变。承诺层同时修好：发现 1 陈旧-*anchor* 尾巴死锁——已承诺块的 *anchor* 新鲜度按【承诺时间】判、非落地时间，扣落地拖久也可落（§8）；发现 5 F_{l1} 边界预签循环——档-(ii) 块头加 *final_ref* 证签名时父已 *final*、拒预签（§4.2）；发现 7 罚没回填——祖父化改用承诺时间戳、精确到”生效前是否真存在”（§4.3）。独立修的 *bug*：发现 3 *lane* 洗白——*lane* 在分叉后首块定、由整支继承，一个普通块洗不白仅强制脚手架（§5.2）；发现 4 *gas* 前缀矛盾——统一为”条数 ≤ 上限且累计 *gas* ≤ 份额的最长 *FIFO* 前缀”，删”恰好 $\min(\text{条数})$ ”（§5.1/§8）；发现 8 整数骑线——*late_units* = $\max(0, preLag - (\Delta_{lag} - 1))$ 、等号处最低非零罚（§6.3）；发现 9 H_{force} ——并入 P_{forced} 谓词 4（任意密钥块须逾期 $\geq H_{force}, §7$ ）。一致性：删 §5.2 旧”块数优先”独立表述并入全序、§5.5/§10 结论纳入 §5.6 第三类可罚过错、§6.4 D_{anchor} 公式修正、”永不过期”限定为缺口维度。明示残留（§5.7）： HC 真实性依赖 ≥ 1 诚实承诺者”；承诺仅证头存在不证块体可得（扣体链仍可抗辩）；这层是当初为省成本删掉的链头承诺/ DA 的最小回流，*option B* 明码接受。改动落点：§4.2、§4.3、§5.1、§5.2、§5.5、§5.6、§5.7（新）、§6.3、§6.4、§7、§8、§10、§12、附录 B。草案 v1.39，2026-08-25——所有者决定引入”挑战罚没”层 + 独立审核第 1/2 轮严重发现修复。独立 AI 深审^[245] 证明：在”落地者完全不受信任 + §1 无协议级 DA ”下，纯链下 *fork-choice* 无法阻止恶意落地者跳过健康尾巴、落更差链，也无法阻止单聚合者零罚反复重置；若要对恶意落地者机械保证，无 DA 即不可判定。所有者据此决定加一个”只罚不改链”的问责层：新增 §5.6 落最优链义务 + 挑战罚没——落地者应落”最优链”（§5.2 新全序：内容 *lane* > 仅强制 *lane*；再块数、再高 *slot*、再首个相异子块哈希）且已传播沉淀者；跳过一条更优沉淀链，任何人可在问责窗口 W_{chal} 内向 $L1$ 举证（纯验签、举证的是早已签好存在的链、无时钟赛跑）→ 没收落地保证金 L_{land} 、赏举报人。关键：挑战只罚没、绝不回滚、不改最终性（落地即最终仍由证明系统定）——与 v1.30 前删掉的”重锚 *episode*”（改正统/需回滚/与证明时延赛跑）本质不同，故避开老死穴。据此修复独立审核的严重发现：(1) 诚实落地者被旧”恢复优先”命令丢弃可落地健康长尾——§5.2 改”最优链含冻结长尾、先

落再从其 *final* 尖端恢复，不丢尾巴”，§6.4 同步^[246]；(2) 单聚合者零罚反复重置——每次落更差链可 §5.6 罚，§6.3 加 *lag* 债按接受前结算、堵整数骑线；(3) 任意密钥更重强制链盖内容尾巴——§5.2 *lane* 优先直接判负 + 可罚。同修独立审核的高危/一致性：组合 *gas* 停链 (§8 加共享块级 *gas* 预算 + 单消息 *gas* 上限 + 确定性优先/溢出/水位线)；*D_anchor_max* 公式漏算 *D_anchor* (§12 补 $\geq D_{\text{anchor}} + P_{\text{prove,max}} + T_{\text{include,max}} + \text{余量}$)；被罚没旧密钥回填历史 *slot* (§4.3 加“按批次接受时点 + δ_{land} 宽限”第二道闸)；*C_anchor* 残文、*m_consumed* 改全局消息序号、“档 (i) 未落地父”措辞、*H_force*/*F_delay* 单一到期谓词、参数表/术语表更新。改动落点：§4.3、§5.2、§5.6 (新)、§6.3、§6.4、§8、§9、§10、§12、附录 B。明示残留 (§5.6)：挑战是经济威慑非安全保证 (需举证经济可行)；诚实但被 *eclipse* 的落地者可能被误罚 (Δ_{prop} 压低)；不改最终性 $\Rightarrow$ 被跳过预确认是“延迟非找回”。均为 §1 无 *DA* 模型下明码接受项。草案 v1.38, 2026-08-25: *Codex* 1×P1 + *DeepSeek* 1 *Critical* + 4 警告，聚焦 *anchor* 新鲜度/反审查/仅强制块交互：(*Codex* P1) *anchor* 新鲜度可使恢复块过期——档 (ii) 消除“缺口过期”，但 §8 *D_anchor_max* 新鲜度仍适用，恢复块须在其 *anchor* 的新鲜度窗口内证明 + 落地。修：§6.4 把“永不过期”限定为“不因缺口过期”，新增设置器不变量 $D_{\text{anchor_max}} \geq \text{证明时延} + \text{最坏落地延迟}$ (§12)；关键区别是 *D_anchor_max* 是自由可设参数，不是旧 *s_ra* 结构性钉死的 64 秒窗口。(*DeepSeek Critical*) 批次末块新鲜度捷径不成立——*anchor* 只单调不减，末块新鲜不代表靠前块新鲜，只查末块会漏放强制义务对照陈旧 *L1* 视图的靠前块。修：§7 改必须逐块查新鲜度，删“或批次末块”捷径。(*DeepSeek W1*) 反审查界是 *D_anchor_max* 非 *C_anchor* 单独——修正“少消费即块不合法”过强：新鲜度下限逼迫 *anchor* 前沿随落地推进，消息最坏延迟 $\approx D_{\text{anchor_max}} + \text{积压}/C_{\text{anchor}}$ (§8)。(*DeepSeek W2*) 仅强制块 *vs* 排班块同 *slot*/父平局——分叉点相同使哈希 *tie-break* 失效。修：§5.2 补“同 *slot*/父下排班块优先于仅强制块，除非普通尾巴已冻结”。(*DeepSeek W3*) “*final*”过载——§4.2 标注 *L1-final vs §6.1 landed-final* (全文统一列 18(c))。(*DeepSeek W4*) 界依赖未定参数——§8/§9 具体界标注为暂定 (§12)。另修 *P_forced* 谓词 3 的嵌套反引号 *Markdown*。改动落点：§4.2、§5.2、§6.4、§7、§8、§12。草案 v1.37, 2026-08-25: *Codex* (14:43) 一项 P1——仅强制块落地后无法续接，*F_l1* 限速与 §7 积压/*C_force* 逐块排空矛盾。原 *P_forced* 谓词 2 要求每个仅强制块都用档 (ii) (*final* 落地父)：全卡特尔下第一个仅强制块落地后成为 *canonical* 头但未 *final*，第二个既不能建在它上 (未达 *F_l1*)、也不能建在旧 *final* 头上 (落地须延伸当前头) $\rightarrow$ 每块须等 *F_l1* 最终化，逐块排空落空。这恰是 v1.35 *finding-5* 注记“仅强制续接同理”隐含假设、却被谓词 2 禁止的串联。修：*P_forced* 谓词 2 放宽为两选一——(重新生根) 档 (ii) *final* 落地头 [链起点/冻结后重根]，或 (同 *lane* 续接) 档 (i) 串前一个仅强制块 (父须满足 $P_{\text{forced,gap}} \leq G_{\text{max}}$)。仅强制块由此在卡特尔下以每块 *C_force* 密度串联排空；父限定为仅强制块 $\rightarrow$ 任意密钥不能经档 (i) 注入诚实排班尾巴 (两 *lane* 以“父是否 *P_forced*”分开，恶意仅强制块对健康内容尾巴仍受 §5.2 恢复优先约束)。改动落点：§7 *P_forced*。草案 v1.36, 2026-08-25: *Codex* 第三批 (14:38) 一项 P1 (line 598) ——*anchor* 不得晚于本块 *slot* 时间的 *L1* $\rightarrow$ *L2* 因果序不变量。§6.1 规则 3 只挡未来 *slot*、§8 只挡 *anchor* 太旧，二者都不挡“旧 *slot* 配新 *anchor*”：构建者可扣一个旧 *slot*、事后挑一个已含新 *L1* $\rightarrow$ *L2* 消息的新鲜 *anchor*，签档 (i) 续接把该消息消费进一个 *slot* 时间戳早于消息源 *L1* 块的 *L2* 块，*Byzantine* 聚合者抢落 $\rightarrow$ 破坏 *L1* $\rightarrow$ *L2* 因果序、失真基于 *block.timestamp* 的桥接时限语义。修：§8 加不变量 $\text{anchor.L1_timestamp} \leq \text{L2 时间戳 (slot)}$ (普通块 + 仅强制块一律适用，§5.1 规则 5 与 §7 *P_forced* 谓词 5 同步)。不影响恢复 (恢复块的 *anchor* 深 $\geq D_{\text{anchor}} \approx 6.4$ 分钟前、时间戳恒 $\leq$ 墙钟 *slot* 时间，天然满足)，只拒“旧 *slot* + 新 *anchor*”畸形组合。改动落点：§8、§5.1、§7。^[247]草案 v1.35, 2026-08-25: *Codex* 第二批 (14:31) 两项 P1，均为对 v1.34 声明的精确化收紧 (非推翻)：(P1 line 916) “一跳”指重新生根，持续推进仍需常态活性底线——恢复块 *R* 是未落地 *P2P* 块，出证明落地要 10–15 分钟，期间排序靠 *P2P* 在 *R* 上继续；“恢复块永不过期”精确指 *R* 的落地合法性 (档 (ii) 无缺口上限)，不指 *R* 可免维护扩展。故恢复活性的确切条件是“排班构建者以 $\geq 1/G_{\text{max}}$ 密度持续回归” (= 常态 G_{max} 活性底线，非恢复独有)；单个稀疏构建者会让 *R* 冻结 $\rightarrow$ 在同一旧落地头重试 *R'*，良性不丢地，参与度回底线即恢复。§6.4/§9 措辞收紧。(P1 line 1102) 单方 *Byzantine* 聚合者可自源孤立分支——任意密钥豁免让恶意聚合者自己 (自入队强制条目 + 等 *F_delay*) 造出档-(ii) 仅强制分支并亲自落地，§5.4“须落地者合谋”的“合谋”可塌缩为单个不受信任聚合者一方，无需策反排班构建者、无可罚没双签。边界不变：被 §5.2 恢复优先挡在健康尾巴

外 (只在尾巴冻结时才被跟随)、深度仍 $\leq$ 未落地尾巴、只及第 2 档可撤销预确认、内容仅强制 (自由裁量交易回 mempool)。修正措辞：仅强制路径频率不受“恶意排班 slot 数”约束，危害受上述三条夹住；显式并入 §1/§5.4 的 *untrusted*-聚合者残留。改动落点：§5.4、§6.4、§7、§9。草案 v1.34，2026-08-25：Codex 与 DeepSeek 对方向 B 新设计的评审^[248]——四项真·新洞 (非旧机制回潮) 已修：(P1) 恢复 *fork-choice* 死锁^[249]：真停摆后停摆前的长尾块数更多、却已冻结不可扩展，§5.2“最多块”规则会让诚实节点死守它、饿死一块的恢复块，恢复与 *fork-choice* 死锁。修：§5.2 新增恢复优先——分支链头 $> G_{\max}$ 落后墙钟即为【冻结死分支】，让位于建在 *final* 落地头的档-(ii) 恢复块；让位判据与恢复触发判据同一，只在真停摆触发、绝不孤立健康尾巴 (其头恒在墙钟 $G_{\max}$ 内)。§6.4 同步引用。(P1) m_{consumed} 只有上限无下限^[250]：纯上限允许构建者持续处理 0 条 $L1 \rightarrow L2$ 消息却用新鲜 *anchor*，无限期审查入站消息、使 积压/ C_{anchor} 追平界失效。修：§8 加强制最大前缀消费 $\min(C_{\text{anchor}}, \text{待处理})$ (公开 FIFO 队列 + 队列根 + §5.1 规则 5 新增合法性义务，形状完全同 §7 强制队列)。(P1) $\leq \Delta_{\text{lag}}$ 是条件性的^[251]：尾巴 $\leq \Delta_{\text{lag}}$ 仅当兜底按期落地；兜底停摆/落地被 $L1$ 审查时尾巴无界增长，档-(ii) 一块即可孤立远超 Δ_{lag} 的尾巴。修：§4.2/§5.4 如实标注该上界与 §6.3 *Byzantine* 活性界同一条件，不加不可兑现的界。(P2) 仅强制块准入谓词^[252]：把 §5.1 对已删“三条件”的引用换成 §7 的精确谓词 P_{forced} (任意密钥 + 档-(ii) *final* 父 + slot $\leq$ 墙钟 + 非空强制前缀至容量 + 最大消息消费)，并说明任意密钥下 slot 唯一性由 *fork-choice* + 原子落地消解、不靠双签罚没。其余：§4.2 块头补 *anchor* 字段^[253]；§12 新增强制载体迁移 (16)、创世引导 (17)、形式化伪代码/lookahead 抽样算法 /术语区分 (18)。v1.32 上 DeepSeek 的“落地未 *final*”*Critical* 与“§7 三条件”警告在 v1.33 已修^[254]。改动落点：§4.2、§5.1、§5.2、§5.4、§6.4、§7、§8、§12。草案 v1.33，2026-08-25：DeepSeek 对 v1.32 的评审 (2 *Critical* + 4 警告 + 3 建议，全部针对方向 B 新设计，非旧机制回潮)——采纳收敛。关键 1^[255]：两档父块规则的判据被误读为“父块落地状态”，使“已落地但未 *final*”的父块看似无档可归、每批次首块疑似非法。修正：明确判据是缺口 $s - \text{parent.slot}$ ，在签名链结构上求值、三处 (签名/证明/落地) 一致，不依赖漂移的落地状态——档 (i) 是有界缺口 (父块落地状态无关，正常出块含落地后首块永远走此档)，档 (ii) 是无界缺口但父须 *final* 落地。“已落地未 *final*”因此不构成任何空档 (§4.2/§5.1 重述)。关键 2^[256]：一跳恢复的“一跳”省略了 F_{L1} 瞬态——停摆恰在一次落地后不久开始、当前落地尖端未达 F_{L1} 时，恢复块须等尖端 *final* ($\leq F_{L1}$) 方能落 (否则 *fork* 掉较新尖端、§7 拒)。修正：恢复界写全为 $\max(0, F_{L1} - \text{尖端存活时长}) + \text{一轮证明}$ ，稳态灾难停摆退化为纯一跳 (§4.2/§6.4/§9/§7 更新)。警告：§5.1 规则 1 的“§7 三条件”陈旧引用改为“§7 定义的仅强制块” ($W1$)；§10 罚没表补齐档 (ii) 经落地头单块孤立路径、三档同界 ($W2$)；§7 游标/快照散文加“净规则”一句厘清逐块 *anchor* 为唯一到期判据、 F_{consumed} 只记消费进度 ($W3$)；§5.4 档 (ii) 残留补前提“当前尖端已 *final*” ($W4$)。建议 3 (标记已删参数为历史) 部分保留：保留 $\Delta_{\text{stall}}/s_{\text{base}}$ 等的变更史注记以存评审溯源，均已明标“已删”。改动落点：§4.2、§5.1、§5.4、§6.4、§7、§9、§10。草案 v1.32，2026-08-25：Codex 对 v1.31 首评一项 $P1$ ^[257]—— C_{anchor} 绑错对象：v1.31 把它加在 *anchor* 引用的逐块推进上，与 §8 *anchor* 新鲜度冲突，长停摆 ($> C_{\text{anchor}} + D_{\text{anchor_max}}$) 恢复块 *anchor* 只能从陈旧父 *anchor* 前进 C_{anchor} 、仍陈旧、通不过新鲜度——恰在它要支持的长停摆上失效。修复： C_{anchor} 改绑到独立的 $L1 \rightarrow L2$ 消息处理游标 m_{consumed} (每块 $\leq C_{\text{anchor}}$ 条)，*anchor* 引用本身仍单调不减 + 必须新鲜、不设逐块上限。于是恢复块引用新鲜 *anchor* (一跳恢复排序， $C1$ 保持)，海量入站消息按 m_{consumed} 逐块处理 (形状同 §7 f_{consumed})。§8/§12 更新。草案 v1.31，2026-08-25——恢复子系统整体重设计 (方向 B)，冻结解除。用两档父块规则取代整套重锚 *episode*：(i) 未落地父块 $\leq G_{\max}$ (防深重组不变)；(ii) *final* $L1$ 落地头为父，无缺口上限 ($G_{\max_landed} = \infty$)——因落地头最终、建其上不 *reorg* 任何已落地块。恢复 = §4.2 档 (ii) 的直接推论：停摆时任一排班构建者 (或仅强制任意密钥) 以落地头为父出一块， $L2$ 排序一跳恢复，任意时长停摆 (数小时 ~ 数天) 皆一跳——因档 (ii) 无上限，恢复块永不过期，旧设计“证明时延顶穿新鲜度窗口”一族 ($r28$ 、第 7 轮 $P1$) 从根上消失。净删：公告/ B_{ra} 、挑战/ B_{ch} /押金结算、钉定父块/重钉、新鲜度地板/ $s_{\text{base}}/s_{\text{ra}}$ 、 Δ_{cont} 续接、*episode* 状态机、仅强制块的双侧 $\text{lag} > \Delta_{\text{stall}}$ 条件——随之 $r29$ - $P2$ (押金确定性) 等待办不再适用。 $G_{\max_landed} = \infty$ 论证：经落地头的 *reorg* 敞口天然被未落地尾巴长度封住、尾巴又被 Δ_{lag} 封住 (超出已落地最终)，故“无限”不增大 *worst-case* 深度 (仍 $\leq \Delta_{\text{lag}}$ ，§5.4)，却换来任意停摆一跳恢复；有限值两头输 (小于证明时延永远追不上长停摆，等于

Δ_{lag} 则收紧是幻觉)。设计者六轮自我对抗复核已并入：①“当前落地头”在 $L1$ 重组下的定义 $\rightarrow$ 绑 $final$ 深度 F_{L1} + 良性搁浅；②恢复活性 $= \geq 1$ 排班构建者回归 $\vee$ 仅强制底线；③ *rule (ii)* 不门控（门控会重开 $r9-1$ 且多余——诚实落地者经 *fork-choice* 天然无害）；④/⑤ 长停摆的 $L1-sync$ /强制积压按逐块 C_{anchor}/C_{force} 追平（排序一跳、同步分摊）；⑥^[258] 落地者尾巴选择策略 $= \$5.2$ *fork-choice* + 落地深度 $\geq \Delta_{prop}$ 传播沉淀，并把“落地者视图完整”补进 $\$1$ 诚实条款。改动落点： $\$4.2$ 两档规则、 $\$5.1$ 规则 2、 $\$5.2$ 落地者策略、 $\$5.4$ 残留三档、 $\$6.4$ 全替换、 $\$7$ 仅强制块简化、 $\$8$ C_{anchor} 、 $\$9$ 恢复行、 $\$12$ 参数、附录 B。草案 v1.30，2026-08-25：DeepSeek 第 8 轮—— $r31-1$ (*Critical 1*，非冻结)： $\$4.2$ 仍写“接最高 $slot$ 合法链头”，与 $\$5.2$ $r11-2$ 的“块数最多”分叉选择矛盾，改为直接引用 $\$5.2$ (否则实现会重开深跳过被自动跟随漏洞)。其余：*Critical 2* (强制包含“无条件”*vs nonce* 抢占) 属待决策 ①($r27-1$ 已加限定)；*Warning 1/2/3* 全部落在已冻结的 $\$6.4/\7 (证明前置期 *vs lookahead* 的“恰好覆盖”措辞、仅强制首块唯一性、*episode* 状态表 *Expired* 转移)——将由恢复子系统的重设计一并消解，不逐点补。草案 v1.29，2026-08-25：Codex 第 18 轮一项 $P1^{[259]}$ ： $r18-2$ 的“批次级唯一快照 H_{batch} = 首块 $anchor$ ”让一个已签名块的强制义务取决于落地者选的批次边界，前缀落地可搁浅本来合法的签名块。改为逐块快照，对照块头已承诺的 $anchor$ ——义务签名时即固定、与批次边界无关。($\$6.4/\7 重锚相关部分仍冻结，本条属普通落地路径，不在冻结范围。)草案 v1.28，2026-08-25：所有者决定—— $\$6.4/\7 恢复子系统冻结待形式化 (方向 B)。不再对该子系统逐点打补丁：近八轮评审 ($r18/r20/r21/r23/r24/r25/r28$ 及第 7 轮 $P1/P2$) 暴露的是同一族“证明时延 *vs slot* 时序窗口”与押金结算确定性问题，根因交点容不下简单规则，应由人工设计复盘 + 状态机模型检查一次性解决。 $\$6.4$ 顶部加 *DRAFT* 冻结注 (列出复盘开放项)，其机制描述与数值界降级为暂定。落在其他部分 (激励、强制包含、参数、信任模型措辞等) 的评审照常处理。草案 v1.27，2026-08-25：DeepSeek 第 6 轮^[260] 2 *Critical* + 5 警告：*Critical 2* ($\$6.4$ 与 $\$7$ 首个仅强制块 $slot$ 矛盾) 已由 $v1.26/r25-2$ 修复； $r27-1$ 把 $\$0/\$1/\$7$ 的强制包含“无条件”声明按 $\$12-12$ *nonce* 抢占缺口打上限定 (*Critical 1* 升级为对声明语气的修正)； $r27-2$ 明确 B_{ra} 在取消 (a)/(b)/执行的不同去向 ((b) 真实故障自愈时全额退还尽责公告者， $W3$)； $r27-3$ 补 $state$ 表“续接态遇正常批次”的转移 ($W2$)； $r27-4$ s_{ra} 须留足证明前置期、期望等待含一轮证明 ($W1$)； $W4/W5$ (入队验证 $L1$ 成本、 Δ_{lag} 余量校准) 并入 $\$12$ 第 14/15 项。草案 v1.26，2026-08-25：Codex 第 16 轮两项 $P1$ (一致性传播)^[261]： $r25-2$ $\$7$ 仅强制块格式规则仍写“首块 = 窗口期满执行基准 $slot$ ”，与 $\$6.4/r24-1$ 的“接受时刻 G_{max} 窗口”矛盾，已对齐 (含 $\$6.4$ 四分法第 3 项)； $state$ 表 *Cancelled* 行补上 $r24-2$ 的取消 (b) 合取项“无未兑现 H_{ch} ”。纯一致性、无新机制。草案 v1.25，2026-08-25：Codex 第 15 轮 $P1^{[262]}$ ： $r24-1$ 让首个仅强制块把头追到墙钟、 lag 掉到 $\leq G_{max}$ ，但 $\$7$ 仍要求仅强制块 $lag > \Delta_{stall}$ ，而 Δ_{cont} 只几个 $L1$ $slot$ ——续接批次永远等不到 lag 再超 Δ_{stall} ， $r18-3$ 的“一个窗口清空积压”失效、卡特尔可对每批施加停摆。修复：续接批次豁免 $\$7$ 三个开阀条件，*riding* 本 *episode* 已建立的、被证明链状态承载的仅强制授权 (严格绑定“延伸本 *episode* 仅强制头 + Δ_{cont} 未空过 + 队列仍有到期条目”，队列排空或超时即失效)；仅首批仍复核三条件。草案 v1.24，2026-08-25：Codex 第 14 轮 $P1^{[263]}$ + DeepSeek 第 5 轮 5 警告： $r24-1$ 重锚新鲜度地板参照点从“窗口期满 s_{base} ”改为“接受时刻墙钟” (证明约 10–15 分钟 $\gg G_{max}=64s$ ，窗口期满参照点到接受时已陈旧数百 $slot$ ，单块重锚仍追不平；绑定接受时刻按构造保证落地瞬间头距墙钟 $\leq G_{max}$ ，落地者为近未来 s_{ra} 预签预证实现)^[264]； $r24-2$ 取消条件 (b) 加合取项“无未兑现 H_{ch} ” (堵排除 H_{ch} 的冲突叉借 $lag \leq \Delta_{lag}$ 绕过分支承诺)^[265]； $r24-3$ 多 H_{ch} 登记与独立结算^[266]； $r24-4$ 明说聚合者活性界条件于无许可兜底经济可行^[267]； $r24-5$ 强制条目 *nonce* 抢占升为阻塞级待定项^[268]。另按建议加参数总表、*episode* 增 *Expired* 态。草案 v1.23，2026-08-25：Codex 第 13 轮一项 $P1^{[269]}$ ： $r17-1$ 的“跳到执行基准 $slot$ s_{base} ”只加在了仅强制重锚块上，普通重锚块漏了——通用落地规则只拒未来 $slot$ ，合谋方可落一个 $slot$ 仅超父块 $G_{max} + 1$ 、却远落后墙钟的陈旧重锚块，头仍不追平、当轮构建者接不上，攻击者用预签稀疏链反复赢公告拖长恢复。修复：两条重锚路径统一加新鲜度地板 $slot \geq s_{base}$ ($\$5.1$ 规则 2 + $\$6.4$ 执行检查)，使一次重锚必把头追到墙钟； s_{base} 由窗口期满墙钟钉定、跨重钉不变。草案 v1.22，2026-08-25：DeepSeek 第 4 轮 7 警告 + 1 建议 (一致性/措辞修正为主，一处概念澄清)： $r22-1$ 澄清“冲突叉”不 *reorg* 已落地的钉定父块 ($\$5.2$ 最终性)，而是延伸它、孤立未落地尾巴，“重钉”只前移指针； $r22-2$ $\$5.1$ 规则 2 的重锚父块定义与 $\$6.4$ 钉定父块对齐；

r22-3 §5.1 规则 4 从“全部强制条目”改为引用 §7 的 C_{force} 容量规则；*r22-4* 删 §5.4 与 *r20-1* 自相矛盾的“封顶 G_{max} ”旧句；*r22-6 §4.1* 退出延迟措辞澄清（快照后退出的地址其排班 *slot* 仍有效）；*r22-7 §6.4* 明说重钉后须对新钉定父块重证；并按建议加入 §6.4 *episode* 状态转移表。^[270] 草案 *v1.21, 2026-08-25*：Codex 第 12 轮一项 $P1^{[271]}$ ：*r18-1* 的重锚取消判定“仅同分支延伸取消”塌缩——§6.1 要求每个被接受批次都延伸钉定的落地头，故任何批次都以钉定父块为祖先、都通过“同分支”测试，恶意 *deep-skip* 照样取消重锚。根因是 *episode* 只钉了父块、没钉要保护的分支。改为用存在挑战 H_{ch} 作分支承诺：取消当且仅当 (a) 被接受批次的落地链包含某个已挑战 H_{ch} （被保护尾巴真落地），或 (b) $lag \leq \Delta_{lag}$ （全量恢复）；其他推进落地头的批次（含孤立 H_{ch} 的冲突叉）只重钉、不取消、保留窗口计时，被孤立挑战退还 B_{ch} 不奖不罚。草案 *v1.20, 2026-08-25*：Codex 第 11 轮一项 $P1^{[272]}$ ： G_{max} 的逐块缺口上限只封顶单块回退深度、不封顶分支深度——此前 §4.2 误称“不可接力叠加”。一个少数派联盟把多块沿私有分支接力串起（每跳父距 $\leq G_{max}$ 、逐块合法、无双签），合谋聚合者整条落地即孤立整条未落地诚实尾巴，深度远不止 $G_{max} - 1$ 。全文 (§4.2/§5.2/§5.4/§10/§1) 改为如实两档深度界：单块深跳过 $\leq G_{max} - 1$ 、接力稀疏分支 $\leq$ 未落地尾巴长度 ($\approx \Delta_{lag} +$ 兜底响应，与双签路径同界但不可罚没)； G_{max} 重新定位为压制稀疏分支可行性（提高所需联盟排班密度）的旋钮，深度残留归入无协议级 DA 的明码取舍 (§1/A-3)。草案 *v1.19, 2026-08-25*：Codex 第 10 轮一项 $P1^{[273]}$ ：普通队头若按满 C_{force} 定份额，每块都有到期桥接条目占去至多 C_{bridge} 时它永远只剩 $C_{force} - C_{bridge}$ 可用、塞不下，持续桥接流可无限期饿死普通强制包含——两条队列的单条目份额都改取“扣除对方保留配额后的保证容量”（普通 $= C_{force} - C_{bridge}$ ，桥接 $= C_{bridge}$ ），各队头在任意对方流量下都能装进本块。草案 *v1.18, 2026-08-25*：所有者转贴的第 10 轮评审^[274]：*r18-1*——*r12-1* 的“任何批次被接受即取消”过宽，恶意排班构建者的冲突短叉（无双签、只需赢落地）同样取消重锚并作废诚实首证优势，墙钟下滴灌即可绕过 *r13-1* 且不落入审查残留；取消收窄为仅同分支延伸取消，冲突叉重组只重钉不重置窗口，冲突叉频率被攻击者排班占比硬性约束（同 §5.4 深跳过残留，证明成本 *griefing* 明码补入）。*r18-2*——“批次锚定 $L1$ 高度”在 §8 逐块 *anchor* 下不唯一，改为落地交易显式声明的批次级唯一快照高度 H_{batch} （= 首块 *anchor*）+ 新鲜度上限，退出时限加 D_{anchor_max} 陈旧项。*r18-3*——仅强制积压若每批重开窗口则代价 = 批次数 $\times$ 窗口；引入续接 *episode*，仅首批付窗口，符合 §7 复合界。草案 *v1.17, 2026-08-25*：Codex 第 9 轮两项 $P1$ (*r17-1*：空积压停摆后新提交条目的到期 *slot* 在墙钟位置， $parent.slot + 1$ 的仅强制链逐 *slot* 爬行永远追不到到期点，全卡特尔逃生阀无法自举——每个重锚 *episode* 的首个仅强制块改为跳到 *episode* 钉定的执行基准 *slot*，后续块仍 $+1$ ，自举一块完成、无自由裁量时间跳跃；*r17-2*： C_{force}/C_{bridge} 下调可使已入队条目超出新份额、*v1.16* 的借额规则覆盖不了普通队头——改为设置器不变量：下调拒绝生效除非新份额 $\geq$ 该队列未消费条目最大 *gas*（水位线实现），借额特例删除）。草案 *v1.16, 2026-08-25*：Codex 第 8 轮两项 $P1^{[275]}$ 。草案 *v1.15, 2026-08-25*：Codex 第 7 轮一项 $P2^{[276]}$ ：§9 快速恢复行的边界差——缺 k 个连续 *slot* 时下一块父距为 $k + 1$ ，快速路径覆盖 $k \leq G_{max} - 1$ （= 63），恰缺 64 个即须走重锚流程；表行按 缺勤 $\leq G_{max} - 1 / \geq G_{max}$ 重新划界。草案 *v1.14, 2026-08-25*：Codex 第 6 轮两项 $P2$ 一致性修正 (*r14-1*：§9 证明故障行曾承诺“落地义务顺延”的豁免——该豁免是 §12 第 8 项阻塞级待定项、尚不存在，行文改为如实标注恢复批次的误罚敞口；*r14-2*：附录 B 重锚流程词条仍写“*lag* 追平即取消”的 *v1.11* 旧定义，与 §6.4 的“任何批次被接受即取消”^[277] 矛盾，已改)。草案 *v1.13, 2026-08-25*：Codex 第 5 轮一项 $P1^{[278]}$ ：兜底计次的“每开窗期至多一次”绝对去重可被反向武器化——合谋兜底账户开窗后落一个微批次吃掉该期唯一计次，再持续滴灌微批次让开窗期永不关闭、 m_{agg} 永远凑不满、聚合者永不换。计次（与 m_{agg} 迟到计数）改为按持续开窗时间限速累积：每持续开窗一个 G_{strike} 且区间内有相应批次被接受即计一次——限速防刷与持续失职必然累积两个性质同时成立；批次证明的增量续证能力列为工程要求 (§12 第 6 项)，保证金耗尽视同终止。草案 *v1.12, 2026-08-25*：所有者转贴的第 12 轮深度评审^[279]：重锚流程改为钉定父块制——公告钉定 (*nonce*, 父块哈希/*slot*)，证明只对钉定父块预生成，任何批次被接受即取消公告（消解“预生成证明”与“执行时取最新父块”的协议矛盾，同时封死“部分落地追不平阈值、剩余尾巴仍被窗口斩尾”的路径）^[280]；存在挑战改为附保证金、按结局结算（虚假挑战不再是免费延期按钮）^[281]；明说重锚签名者是信任模型内的 概率界、全员卡特尔下唯一确定出口是仅强制块^[282]；深跳过敞口补上要害——在任聚合者合谋时无竞速可言，定界为明码安全残留并把

分叉选择的效果降级为 *P2P* 收敛假设、非 *L1* 保证^[283]；恢复时间四分法（安全消息/队列清空/*lag* 追平/自由裁量服务）^[284]；§5.5 残留的“弱执行类不含能获利攻击”旧句更正^[285]。草案 *v1.11*，*2026-08-25*：所有者转贴的第 11 轮深度评审^[286]。对当前版本仍有效的两项：深跳过敞口^[287]——*G_max* 允许父块指到 $s - G_{max}$ 后，“接最高 *slot* 链头”的旧协调规范会让诚实网络自动跟随深跳过叉、免费孤立至多 $G_{max} - 1$ 个诚实块；§5.2 改为“分叉点起块数最多”优先（真实缺口后的正常出块不受影响，深跳过叉降级为必须赢得落地竞速），§5.4/§10 的敞口表述同步从“单 *slot*”修正；停摆情形强制路径的复合时间界^[288]明写为 $\max(\text{逾期时点}, \text{停摆起点} + \Delta_{stall}) + \text{重锚窗口} + \text{证明落地时延}$ 。草案 *v1.10*，*2026-08-25*：*v1.9* 之上应用 *DeepSeek* 第 3 轮（2 严重 + 3 警告 + 2 建议）^[289]； Δ_{lag} 初始值 3 *epoch* 低于规范自己声明的正常滞后带上界（ $19.2 < 20$ 分钟）、违反 *r6-2* 设置器不变量，校准为 4 *epoch*， Δ_{stall} 联动为 5 *epoch*^[290]；*f_consumed* 形式化调和为游标对，消除 *r8-2* 双队列后的实现歧义^[291]；退出延迟显式覆盖 $\delta_{slash} + \text{证据提交时延的罚没敞口}$ ^[292]；排班窗口固定对齐分区的形式定义 $w(slot) = \text{floor}(slot / 768)$ ^[293]；附录 A-3 标注 §1 所引成本恒等式^[294]。草案 *v1.9*，*2026-08-25*：*v1.8* 之上应用 *Codex* 第 4 轮三项 *P1*^[295]。草案 *v1.8*，*2026-08-25*：*v1.7* 之上应用 *Codex* 第 3 轮两项 *P1*^[296]。草案 *v1.7*，*2026-08-25*：*v1.6* 之上应用所有者转贴的第 7 轮深度评审（2 阻塞 + 4 高危 + 1 目标层面，全部采纳）：排班种子按窗口单一取值 + $\text{randao_source} \leq H_{snap} \leq \text{最终化 } L1 \text{ 头} < \text{窗口起点一致性公式}$ ^[297]；强制时间界改为积压感知公式 + 桥接保留配额 *C_bridge*^[298]；*f_consumed* 游标四步原子状态转换^[299]；*anchor* 定为系统级隐式交易^[300]；仅强制块 *slot* 唯一化为 $\text{parent.slot} + 1$ ^[301]；迟到计数按开窗期去重、“零误伤”声明降级至豁免机制（升为阻塞级待定项）完成^[302]；用户承诺三档术语 + 验收基准注记^[303]。草案 *v1.6*，*2026-08-25*：*v1.5* 之上应用 *Codex* 第 2 轮两项 *P1*^[304]。草案 *v1.5*，*2026-08-25*：初稿经五轮对抗评审修订^[305]。第 5 轮（*OpenAI*）修复：批次不得落地未来 $\text{slot} + lag$ 下限截断^[306]、跳过（*skip*）更正为“可获利、不可罚没”的诚实定性并分层用户语义^[307]、计次按开窗期去重并加 *G_strike* 宽限^[308]、*L_eq* 外部价值诚实上限与证据幂等^[309]、强制队列游标形式化^[310]；信任模型补明“诚实多数是经济假设、非协议强制”^[311]。机制骨架完整，参数为初始建议值，§12 列出待定项。

附录 E：评审注记索引

本附录汇集正文中以 ^[n] 标出的全部出处注记——它们记录每条规则由哪一轮对抗评审、哪位评审者或所有者的哪次决定促成。正文因此只承载规范性陈述；追溯某条规则的来龙去脉时查阅本附录。完整的逐版本修订记录另见附录 D。

- [1] 前言 其全文与评审记录保留在 git 历史中，可保留结论见 legacy-summary.md
- [2] 前言 评审 r39
- [3] 前言 v1.31 起，含每一轮评审的发现与修复
- [4] §1 r41 option C 修订原”接受即最终”
- [5] §1 评审 r27-1，DeepSeek 第 6 轮 Critical 1
- [6] §1 所有者决定，2026-08-25
- [7] §1 评审 r12-4 的精确化 + r20-1 更正深度
- [8] §1 r42，独立审核第 5 轮高危 1——固定的 W_settle 收盘与 D_anchor_max 新鲜度截止无法只由”最终会纳入”推出稳健
- [9] §1 评审 r5-1 的精确化
- [10] §1 r7-7
- [11] §1 r5-3
- [12] §1 所有者决定：无需协议内解决
- [13] §1 评审 r33，与”诚实”并列的第三条非纯协议依赖
- [14] §3.2 纯函数，评审 r7-1 重写——v1.6 的”每 slot 各自向前 D_rand 取 RANDAO”与 768 slot 前瞻、单一 H_snap 三者无法同时成立：768 个 1 秒 L2 slot 恰好 $\approx$ 2 个 L1 epoch，窗口末端的 per-slot RANDAO 尚未最终、且晚于 H_snap
- [15] §3.2 评审 r10-5 的形式化——”窗口”若被解读为滑动视界，同一 slot 会在不同时刻得到不同排班，破坏 G2 的确定性
- [16] §3.2 评审 r8-1 的修复——v1.7 的 D_snap = 3 epoch 漏了视界项：排班要求提前 H_look 可算，最远端窗口的起点 $\approx$ now + H_look，其快照点 = now + H_look - 3 epoch = now - 1 epoch，仍未最终化、可被重组
- [17] §3.2 r1-9 的唯一性要求由此自动满足
- [18] §3.2 评审 r1-7
- [19] §3.2 所有者建议，此处有意偏离——见附录 A-1
- [20] §4.1 评审 r1-5 的修复
- [21] §4.1 评审 r2-3：双签者在首次罚没生效前还能跨入下一窗口继续签，视界不能只算单窗口
- [22] §4.1 评审 r5-5
- [23] §4.1 评审 r10-4 的显式化；r22-6 澄清措辞
- [24] §4.2 评审 r36 补入字段列表，DeepSeek 警告——§7 的 r30-1 逐块强制义务与 §8 的 m_consumed 均以块头 anchor 为准，故它必须是签名覆盖的块头字段，而非批次级隐式量
- [25] §4.2 所有者 2026-08-25 的修复建议；分析见 §5.4

- [26] §4.2 评审 r1-8 的修复
- [27] §4.2 评审 r6-1 → r32 重做，方向 B 恢复重设计；r35 澄清判据与求值时点，DeepSeek 关键 1/2
- [28] §4.2 DeepSeek 关键 1 的化解——判据是缺口不是落地状态，故不存在“两档都不适用”的父块类别
- [29] §4.2 r42: 仅 F_l1 深不够，provisional 候选收盘前仍可被取代，建在其上会随之搁浅，独立审核第 5 轮一致性 2；等待界 = $\max(W_settle, F_l1) + D_anchor$
- [30] §4.2 r41
- [31] §4.2 DeepSeek 关键 2
- [32] §4.2 评审 r40，独立审核第 3 轮发现 5
- [33] §4.2 方向 B，所有者决定 2026-08-25
- [34] §4.2 评审 r36, Codex line 475 P1——必须明说
- [35] §4.2 r24-4
- [36] §4.3 评审 r1-4 的修复
- [37] §4.3 评审 r39，独立审核第 2 轮发现 5
- [38] §4.3 §6.4/独立审核发现 5
- [39] §4.3 独立审核第 4 轮发现 7 指出其宽限窗内放回填、窗外误杀在途的两难
- [40] §4.3 如实，r42 修正措辞——独立审核第 5 轮指出“不误伤其他构建者”不实
- [41] §5.1 评审 r3-3 Codex 引入例外；r35 更新引用 DeepSeek 警告 1；r36 定为精确谓词 Codex line 517 / DeepSeek
- [42] §5.1 r32；判据是缺口大小、非父块落地状态，r35
- [43] §5.1 r41 补进合法性规则——独立审核第 4 轮一致性 1: v1.40 只在 §4.2 叙述、未列入本节
- [44] §5.1 评审 r40 消除独立审核发现 4：此前“至 C_force 上限”与新增“gas 满即溢出”两套规则互相否定
- [45] §5.1 r9-2
- [46] §5.1 评审 r37
- [47] §5.1 评审 r40 与规则 4 同构，消除发现 4 的“恰好 $\min(C_anchor)$ ”与“gas 满即溢出”矛盾
- [48] §5.1 r36
- [49] §5.2 r42 改窗口语义，删“先落者永久正统”
- [50] §5.2 评审 r11-2 引入，r40 把排序合并进全序、删去与之冲突的旧“块数最多”独立表述，独立审核一致性 1
- [51] §5.2 评审 r12-4，仍成立
- [52] §5.2 评审 r39 引入；r40 修 lane 洗白；r41 起服务于结算窗口的机械比较
- [53] §5.2 r42 修独立审核第 5 轮严重 1——旧“看两链首个相异块决定 lane”是 pairwise 判据，可构造 $A > B > C > A$ 非传递环： $A = [X, a]$ vs $B = [X, b1..b3]$ 在 $a/b1$ 分歧判 content 胜， B vs $C = [Y, c1, c2]$ 在 X/Y 分歧按块数， C vs A 又在 Y/X 分歧按块数——环。修复：lane 是每条候选自身的、相对固定基线 F 的不变标量，与比较对象无关

- [54] §5.2 r42
- [55] §5.2 仍防 r40 发现 3 的洗白：仅强制首块的链无论后面接多少普通块，lane 仍为仅强制
- [56] §5.2 r42 弃用”首个相异子块哈希”这一 pairwise 量，改为候选自身的标量，保证全序
- [57] §5.2 r42
- [58] §5.2 评审 r33，所有者提出；r41 由 §5.6 结算窗口机械背书
- [59] §5.2 r33
- [60] §5.2 r41 由结算窗口化解
- [61] §5.2 v1.39 前”落错即无法纠正”的前提已被窗口消除
- [62] §5.2 r47，所有者确认的表述
- [63] §5.2 含双签分叉，r47 明确
- [64] §5.2 设计取舍，r47
- [65] §5.2 评审 r1-1，必须明说的残留
- [66] §5.2 评审 r16-1 的更正——v1.15 及之前如此声称
- [67] §5.2 r41
- [68] §5.4 见 §4.2/下文的 r20-1 两档界；r22-4 删去此处此前笼统写”封顶在 $G_{\max}$ 以内”的旧句，与下文自相矛盾。
- [69] §5.4 评审 r5-3 更正
- [70] §5.4 评审 r11-2 补明、r20-1 更正深度
- [71] §5.4 §4.2 的 r20-1
- [72] §5.4 评审 r12-4
- [73] §5.4 r20-1 分两档，v1.31 方向 B 增补档 (ii) 的单块经落地头路径
- [74] §5.4 v1.31 新， $G_{\max_landed} = \infty$
- [75] §5.4 评审 r35, DeepSeek 警告 4
- [76] §5.4 评审 r36, Codex line 475 P1
- [77] §5.4 评审 r1-1/r1-6 后的诚实版
- [78] §5.4 软上界，r16-1 更正——见 §5.2 诚实定界
- [79] §5.4 r5-3
- [80] §5.4 r20-1
- [81] §5.4 r11-2，§5.2
- [82] §5.4 r12-4，上文的明码残留
- [83] §5.4 评审 r7-7 采纳
- [84] §5.4 r5-3
- [85] §5.5 评审 r1-6 的修复
- [86] §5.5 评审 r2-1 的一半
- [87] §5.5 r41 更新——option C 撤回”落错链可罚没”，代之以窗口机械竞争
- [88] §5.5 独立审核第 3/4 轮的结论

- [89] **§5.5** 评审 r12-6 更正——此处曾残留 r5-3 之前的”弱执行类不含任何能获利的攻击”旧结论，与 §5.4 直接矛盾
- [90] **§5.6** 评审 r41，所有者决定 option C
- [91] **§5.6** r42 修独立审核第 5 轮严重 2:v1.41 让首个 provisional 候选立即推进 canonical 游标/ 状态，第二个更重候选起始游标对不上、连参赛都不可能——”先落者永久赢”复辟
- [92] **§5.6** r46 澄清，DeepSeek-on-v1.45 W2
- [93] **§5.6** r44,W2；可执行模型的 replay 即此语义，附录 C P5b/P6
- [94] **§6.1** r41 option C 修订
- [95] **§6.1** 评审 r5-2 的修复——阻塞级
- [96] **§6.2** r41:lag 以 provisional 候选计，保证活性判定不被窗口延迟拖累
- [97] **§6.2** r41 option C 的明码成本
- [98] **§6.2** r42，独立审核第 5 轮高危 2/中危 1——混用会先关掉纠错报酬、又把可撤销敞口写小
- [99] **§6.2** r44,DeepSeek-on-v1.43 C1——只拆判定量不拆阈值，兜底窗口会常开
- [100] **§6.2** r10-2
- [101] **§6.3** §6.2 拆分，r42/r44
- [102] **§6.3** r44 清理，DeepSeek W3
- [103] **§6.3** r10-2 等
- [104] **§6.3** 评审 r2-4
- [105] **§6.3** r44 引入、r46 数值重校——阈值必须随判定量一起抬，推导见 §6.2; $\Delta_{\text{lag,prov}}$ 初始 = 4 epoch $\approx$ 25.6 分钟；评审 r10-2 的校准——v1.9 及之前初始值为 3 epoch $\approx$ 19.2 分钟，低于 §6.2 自己声明的 15–20 分钟正常滞后带上界，违反 r6-2 的设置器不变量 $\Delta_{\text{lag}} \geq$ 正常滞后带上界：正常运行中窗口就会打开、误罚诚实聚合者，”零误伤”失效。校准后余量 $\approx$ 5.6 分钟
- [106] **§6.3** r42，独立审核第 5 轮高危 2——若按 provisional lag 判，恶意同伙先落一个短候选把 lag_prov 清零，就能在诚实重候选落进来之前关掉兜底资格、掐断其证明费补偿，理性兜底者退出、坏候选无竞争收盘
- [107] **§6.3** 评审 r5-4 引入去重、r13-1 修正粒度——Codex 第 5 轮指出 v1.12 的”每个开窗期至多一次”是绝对去重：与聚合者合谋的兜底账户在开窗后落一个刻意的微批次吃下该期唯一一次计次，之后持续滴灌微批次让最终性任意缓慢地爬行、开窗期永不关闭，m_agg 永远凑不满，聚合者永不被换——防刷计次的盾牌被反手当成了攻击者的盾牌；微批次每次推进落地头还会作废诚实兜底者按旧链头预生成的证明，使其抢不回落地权
- [108] **§6.3** r5-4 的原始目的
- [109] **§6.3** 评审 r3-1，Codex
- [110] **§6.3** 评审 r4-2，DeepSeek——v1.3 曾写成按开窗计数，与本节的零误伤原则自相矛盾
- [111] **§6.3** 评审 r7-6
- [112] **§6.3** r13-1 把两个计数器的粒度一并从”每开窗期一次”修正过来，否则聚合者用自己的微批次骑线同样可以把 m_agg' 卡在 1
- [113] **§6.3** 评审 r39，独立审核第 2 轮发现 3—

- [114] §6.3 主堵点，r41 §5.6 结算窗口
- [115] §6.3 评审 r39;r40 修等号处仍零罚——独立审核第 3 轮发现 8
- [116] §6.3 r41，独立审核第 4 轮中危 3
- [117] §6.3 评审 r24-4，DeepSeek 第 5 轮 W3
- [118] §6.3 评审 r2-1 的修复——这一条必须明说
- [119] §6.4 v1.31 方向 B 重设计
- [120] §6.4 r23→r24→r25→r28 + 第 7 轮 P1/押金确定性 r29-P2
- [121] §6.4 见附录 D v1.31 条
- [122] §6.4 评审 r39 更正，取代 v1.35 的错误”恢复优先/让位”；独立审核第 1/2 轮发现 1/2
- [123] §6.4 r41
- [124] §6.4 评审 r36,Codex line 916 P1——精确化” ≥ 1 构建者回归”
- [125] §6.4 评审 r35,DeepSeek 关键 2
- [126] §6.4 r41 补 D_anchor 项——独立审核第 4 轮中危 2: 档 (ii) 块头的 final_ref 须 $\leq$ anchor，而 anchor 至少深 D_anchor，故见证父 final 的 L1 块还要再老 D_anchor 才能被引用，最早可签名点 = 父落地 + F_l1 + D_anchor
- [127] §6.4 评审 r38,Codex P1
- [128] §6.4 r40 修正为 $D_anchor_max \geq D_anchor + P_prove,max + T_include,max + 余量$ ——独立审核第 3 轮指出旧写法漏了 anchor 签名时已老 D_anchor 这一项
- [129] §6.4 r44 lag 项升为 final 阈值，罩住兜底授权全程——凡兜底被授权落的尾巴其 anchor 必然仍新鲜
- [130] §6.4 r39 更正 C_anchor 绑定表述——独立审核一致性 2
- [131] §6.4 r39 与下文”持续推进需活性底线”统一——独立审核一致性 6：删去与之冲突的旧” ≥ 1 回归即够”表述
- [132] §6.4 相对 v1.30
- [133] §6.5 评审 r1-10
- [134] §6.5 r47，所有者提出
- [135] §6.5 与所有者”landing 时直接分”的建议有一处偏差，见附录 A-4
- [136] §7 评审 r3-2，Codex——P1
- [137] §7 评审 r4-1，DeepSeek——严重
- [138] §7 评审 r7-2 的修复——引入 C_force 后，固定的” $\leq F_delay + H_force$ ”上界不再成立
- [139] §7 评审 r8-2 的修复——单一 FIFO 游标下”保留配额”形同虚设：桥接条目仍要等它前面的全部普通条目清完才轮到，付费洪泛可无界推迟它
- [140] §7 评审 r5-6 的形式化
- [141] §7 评审 r7-3 的形式化；r10-3 调和为游标对——r8-2 拆双队列后本条仍写着单一游标，而”两游标各自按 r7-3 推进”缺少形式定义，实现会在批次起止校验上分叉
- [142] §7 实现照此一句，评审 r35 澄清，DeepSeek 警告 3——下文 r18-2→r30-1 是演进史，不是并行的第二套绑定

- [143] §7 评审 r18-2 → r30-1 重做
- [144] §7 评审 r9-2, Codex——P1
- [145] §7 评审 r16-2 引入按队列验份额、r19-1 修正普通份额：若桥接条目只按 C_{force} 验， gas 落在 $(C_{bridge}, C_{force}]$ 的条目入队却塞不进 C_{bridge} 桥接前缀、到队头死锁；而普通份额若取 C_{force} ，当每块都有到期桥接条目占去至多 C_{bridge} 时，一个满 C_{force} 的普通队头永远只剩 $C_{force} - C_{bridge}$ 可用、永远塞不下，持续桥接流即可无限期饿死普通强制包含——两条队列的单条目份额都必须取其”扣除对方保留配额后的保证容量”：普通 = $C_{force} - C_{bridge}$ ，桥接 = C_{bridge} ，各自的队头由此在任意对方流量下都能装进本块
- [146] §7 评审 r17-2 取代 v1.16 的借额泄流规则——借额只覆盖 $\leq C_{force}$ 份额的桥接条目，救不了 C_{force} 下调后超出新份额的普通队头条目，强制前缀义务会使所有块非法、死锁重现
- [147] §7 评审 r24-5, DeepSeek 第 5 轮 W5
- [148] §7 v1.31 方向 B 简化：删除双侧 $lag > \Delta_{stall}$ 条件、公告/挑战/ Δ_{ra} 窗口、 s_{base}/s_{ra} 、 Δ_{cont} 续接——这些随 §6.4 episode 一并删除
- [149] §7 r37-2 修正，原”只准档 (ii)”会被 F_{l1} 限速
- [150] §7 评审 r36, Codex line 517 P2 / DeepSeek 警告
- [151] §7 评审 r37-2, Codex ”Allow forced-only blocks to continue after landing” P1——原写”只准档 (ii)”在全卡特尔下被 F_{l1} 限速，与 §7 声称的 积压/ C_{force} 逐块排空矛盾
- [152] §7 r39；落地者跳过内容尾巴去落仅强制链在 §5.6 窗口内被内容候选直接盖掉，r41
- [153] §7 评审 r40 补 H_{force} ，独立审核第 3 轮发现 9：此前谓词只要”到期非空前缀”，在 $due_{slot} = 提交 + F_{delay}$ 就开放任意密钥，比参数表宣称的 $due + H_{force}$ 早 H_{force} ，提前放大任意密钥分叉面
- [154] §7 r41，独立审核第 4 轮中危 1 ——”首块满足即可”会让一个老条目把任意密钥权限无限续杯
- [155] §7 r40
- [156] §7 DeepSeek 警告——任意密钥无双签罚没
- [157] §7 评审 r36, Codex line 1102 P1
- [158] §7 r35, DeepSeek 建议 2
- [159] §8 评审 r37, Codex line 598 P1
- [160] §8 r41，取代 v1.40 的承诺时间豁免——承诺层已删；独立审核第 3 轮发现 1、第 4 轮场景 A
- [161] §8 r44: lag 项是 $\Delta_{lag, final}$ ——兜底以 lag_{final} 开窗，授权时点比旧 Δ_{lag} 更晚，新鲜度窗口必须罩住它，否则 C1 修复会反过来把这条几何解打穿
- [162] §8 r46 随 $\Delta_{lag, final}$ 重校同步抬升
- [163] §8 评审 r32 自审第 5 轮引入，r34 修正绑定对象——Codex 指出绑在 $anchor$ 上会与新鲜度冲突
- [164] §8 评审 r39，独立审核一致性 3：同一 $L1$ 高度可含多条消息，若游标只记高度会在该高度内重复或漏消费、无法唯一证明”最大前缀”；故 $m_{consumed} =$ 入站消息队列的全局序号，或等价的 ($L1$ 高度, tx 序, log 序) 三元组, 与 §7 强制游标同款

- [165] §8 r41 删净”恰好 $\min(\text{条数})$ ”残文——独立审核第 4 轮严重 3 指出该残文与 §5.1 双约束前缀互相否定，可致共识分裂或无合法块
- [166] §8 r41 修”纯条数除法低估 gas 瓶颈”
- [167] §8 Codex line 1031
- [168] §8 评审 r38,DeepSeek 警告 1 ——修正”少消费即块不合法”的过强表述
- [169] §8 配合 Codex line 598 的 $\text{anchor} \leq \text{slot}$ 时间上界，anchor 被夹在 [落地头 - $D_{\text{anchor_max}}$, slot 时间对应高度] 内。
- [170] §8 r34
- [171] §8 评审 r39，独立审核第 1/2 轮发现 1/5——严重
- [172] §8 强制侧沿用 §7 r16-2/r19-1 的按队列份额
- [173] §9 r39 更正——不丢冻结长尾
- [174] §9 自败，r41
- [175] §9 r41
- [176] §9 r35/r41；稳态无此项
- [177] §9 r36
- [178] §9 r24-4：若无人兜底则不计次、可无限停滞
- [179] §9 §1 信任模型，所有者决定不在协议内解决
- [180] §9 r14-1 如实标注；豁免形态拟沿用 v15 §10.4 第 3 级
- [181] §10 r5-3
- [182] §10 r11-2/r20-1
- [183] §10 v1.31/r35，前提是当前落地尖端已 final，见 §5.4/§4.2
- [184] §10 r41 与 §4.2/§5.4 一致化
- [185] §10 r41 option C——”该落什么”被证明无法机械裁决，独立审核第 3/4 轮
- [186] §10 r5-3 更正
- [187] §12 r44 拆分，定义见 §6.2
- [188] §12 v1.31 方向 B 重设计后
- [189] §12 r41 修正本行残文，独立审核第 4 轮一致性 7
- [190] §12 r18-2/r30-1
- [191] §12 评审 r38 引入，r39 修正公式漏项——独立审核第 2 轮发现 4
- [192] §12 r44:lag 项为 $\Delta_{\text{lag,final}}$ ，与 §8 同步
- [193] §12 §1,r42
- [194] §12 r41 option C,§5.6;r42 补上界
- [195] §12 r42 中危 1
- [196] §12 v1.39/v1.40 的 $W_{\text{chal}}/L_{\text{land}}/L_{\text{chal}}/L_{\text{commit}}/\delta_{\text{land}}$ 随挑战与承诺层删除，不再是参数。
- [197] §12 水位线的”下调 vs $O(1)$ ”张力见独立审核一致性 7，列为实现细化
- [198] §12 评审 r38,DeepSeek 警告 4

- [199] §12 r13-1 的工程要求
- [200] §12 阻塞级, r7-6
- [201] §12 评审 r2-8
- [202] §12 阻塞级, r24-5
- [203] §12 评审 r24-4, DeepSeek 第 5 轮
- [204] §12 评审 r27-5, DeepSeek 第 6 轮 W4
- [205] §12 评审 r27-5, DeepSeek 第 6 轮 W5; r44 起覆盖两个阈值
- [206] §12 r46 重校
- [207] §12 评审 r36, DeepSeek 警告
- [208] §12 评审 r36, DeepSeek 警告
- [209] §12 评审 r36 提出; r42 定为阻塞; r43 前半、r44 后半已交付
- [210] §12 最终判定 = 所有者 + 人类安全评审, 本门不自动放行
- [211] §12 r42
- [212] §12 r42 阻塞级, 实现前必须完成
- [213] §12 当前散文重度依赖评审注记与交叉引用, P1 级的父块/fork-choice/落地者交互易被读者漏看
- [214] §12 评审 r24-建议——单位与不变量集中一处, 散落数值易读错
- [215] §12 r35
- [216] §12 r46 重校, 8 epoch 违反本公式
- [217] §12 r44, DeepSeek C1
- [218] §12 r42 中危 2; 随 L1 重组回滚
- [219] §12 ≈ 420 , r46 随 $\Delta_{lag, final}$ 重校抬升
- [220] §12 r42 全文统一最强式, r44 lag 项升 final
- [221] §12 r47
- [222] 附录 A 待所有者确认
- [223] 附录 A r10-6 补标注
- [224] 附录 B §5.6, r41
- [225] 附录 B §4.2, v1.31/r35
- [226] 附录 B §7/§8, r9-2
- [227] 附录 B §5.2, r42
- [228] 附录 B r41/r42
- [229] 附录 B §5.6, r41 option C
- [230] 附录 B r41
- [231] 附录 C 前半 r43, 后半 r44
- [232] 附录 C 对照独立审核第 5 轮的阻塞/要求
- [233] 附录 C r44
- [234] 附录 C r46 非空化, 推导式断言恒真无检验力

- [235] 附录 C DeepSeek W1/建议 3
- [236] 附录 C r44
- [237] 附录 C DeepSeek W2
- [238] 附录 C r47
- [239] 附录 D 评审 r39
- [240] 附录 D 独立审核第 5 轮高危 1——…
- [241] 附录 D 所有者指令：五轮自审，零规范语义变更
- [242] 附录 D 含评审括注的阅读约定
- [243] 附录 D v1.41” 无额外状态机” 措辞更正
- [244] 附录 D v1.39 §5.6 + v1.40 §5.7, W_chal/L_land/L_chal/L_commit/ δ_land 全部退场
- [245] 附录 D 对象 v1.34/v1.38
- [246] 附录 D 取代 v1.35 错误规则
- [247] 附录 D 同批 line 855/1036 两项 = v1.35 已修的 cHR4b/cHR4c 在新 head 上的重发，已回复 resolve。
- [248] 附录 D Codex $3 \times P1 + 1 \times P2$; DeepSeek 1 Critical + 6 警告 + 4 建议
- [249] 附录 D Codex line 822
- [250] 附录 D Codex line 1031 / DeepSeek
- [251] 附录 D Codex line 475
- [252] 附录 D Codex line 517 / DeepSeek
- [253] 附录 D DeepSeek
- [254] 附录 D 其评审对象是 v1.32 旧 head
- [255] 附录 D r35-1
- [256] 附录 D r35-2
- [257] 附录 D r34-1
- [258] 附录 D 所有者 r33 提问
- [259] 附录 D r30-1，非冻结范围——普通强制包含路径的确定性
- [260] 附录 D 评审 v1.25
- [261] 附录 D 我 r24 的 §6.4 修复未同步到其余规范位置
- [262] 附录 D r25-1
- [263] 附录 D r24-1
- [264] 附录 D 终结 r18/r20/r21/r23 的重锚 slot-timing 一族，并含 DeepSeek W2
- [265] 附录 D DeepSeek W1
- [266] 附录 D DeepSeek W4
- [267] 附录 D DeepSeek W3
- [268] 附录 D DeepSeek W5
- [269] 附录 D r23-1
- [270] 附录 D DeepSeek 指出的 附录 B 旧词条已在 v1.21/r21-1 更新。

- [271] 附录 D r21-1
- [272] 附录 D r20-1，重要
- [273] 附录 D r19-1
- [274] 附录 D 对象 head v1.17，2 P1 + 1 P2，全部成立采纳
- [275] 附录 D r16-1：双签合谋的可孤立深度曾写”被 Δ_{lag} 硬性封顶”——开窗只是授权、不强制接受，如实界为 Δ_{lag} + 兜底响应时间的软上界 + 明示假设；r16-2：桥接条目按 C_force 验 gas 但只能按 C_bridge 消费，(C_bridge, C_force] 区间条目到队头即死锁——入队验证改按所属队列配额计，加遗留超额条目的确定性借额泄流规则
- [276] 附录 D r15-1
- [277] 附录 D r12-1
- [278] 附录 D r13-1
- [279] 附录 D 对象 v1.11，1 阻塞 + 5 高危 + 2 中危，全部成立并采纳
- [280] 附录 D r12-1
- [281] 附录 D r12-3
- [282] 附录 D r12-2
- [283] 附录 D r12-4
- [284] 附录 D r12-5
- [285] 附录 D r12-6
- [286] 附录 D 评审对象为 v1.6，10 项中 8 项已由 r7-r10 修复，逐项对照见 PR 回复
- [287] 附录 D r11-2
- [288] 附录 D r11-4
- [289] 附录 D 其严重 1 与 Codex r9-3 重复、v1.9 已修
- [290] 附录 D r10-2
- [291] 附录 D r10-3
- [292] 附录 D r10-4
- [293] 附录 D r10-5
- [294] 附录 D r10-6
- [295] 附录 D r9-1/r9-3：删除”落地时 $\text{lag} > \Delta_{\text{stall}}$ 即豁免 G_{max} ”的即时恢复例外——该条件只看 L1 落地头、看不见 P2P 尾巴，扣落地的聚合者 + 一个恶意排班构建者 + 合谋落地者可借它免双签孤立无界诚实尾巴；且真实大缺口的诚实恢复被迫等满 Δ_{stall} ——统一改为 §6.4 重锚流程：公告 + 存在挑战窗口 + lag 追平即取消，仅强制块落地同规，r2-2 的停摆残留竞速一并收窄；r9-2：强制条目加两道确定性防线——入队验证 + 消费时弃置，堵死”一条畸形付费条目使包含义务与执行合法性两难、链永久停摆”
- [296] 附录 D r8-1：D_snap 补上前瞻视界项，= $H_{\text{look}} + F_{\text{final}} + \text{余量} = 5 \text{ L1 epoch}$ ，并立为设置器不变量；r8-2：强制队列拆为普通/桥接双 FIFO 双游标，C_bridge 保留配额在单一 FIFO 下形同虚设的问题修复
- [297] 附录 D r7-1，修复 768 slot $\approx 2 \text{ L1 epoch}$ 的三方矛盾
- [298] 附录 D r7-2

- [299] 附录 D r7-3
- [300] 附录 D r7-4
- [301] 附录 D r7-5
- [302] 附录 D r7-6
- [303] 附录 D r7-7
- [304] 附录 D r6-1：父块缺口上限 $G_{\max}$ + 停摆恢复例外，封死免双签的深重组路径；r6-2：仅强制阀的开关阈值从 H_{force} 改为 $\Delta_{\text{stall}} > \Delta_{\text{lag}}$ ，修复其与正常最终性滞后带的数值冲突
- [305] 附录 D 一轮内部评审、DeepSeek 两轮、Codex 一轮、所有者转贴的 OpenAI 评审一轮，共 31 项发现全部修复或明确定性；正文以”评审 rN-M”标注各修复位置，评审原文见 PR 记录
- [306] 附录 D r5-2，阻塞级
- [307] 附录 D r5-3
- [308] 附录 D r5-4
- [309] 附录 D r5-5
- [310] 附录 D r5-6
- [311] 附录 D r5-1，所有者决定的精确化